

## Unit 1:

### Data Structure overview and Arrays

A computer system can perform different tasks with different information and records. Every information or record is represented in computer system in a form of binary digits or values and a meaningful set of binary sequence is called data. Another way each and every information or records are a group of data. There are different forms of data can accomplish under a computer system. These data can be used to process different forms of information such as numerical values, image files, audio, video etc.

#### 1.1 Concept of Data Types:

Different information or records has their own different characteristics and formation behavior. Some information such as alphabets or symbols has one type of information. As well as numerical values, image files, audios, videos have some another kind of characteristic based information. To handle this variety of data, we adapted different kinds of data types that are represented according to the memory space occupied by.

##### 1.1.1 Basic data types

As per the characteristics of different data and memory requirements to hold the data, there are four basic categories of data types:

- Character (char) represents the alphabets and other symbols
- Integer (int) represents the integer numbers (i.e. 0, 1, 2, ... -1, 2, .... Etc.)
- Floating Point (float) represents the real numbers (i.e. 2.5, 23.0045, ... etc.)
- Double (double) represents the double precision numbers (i.e.  $2.5 \times 10^{34}$ ,  $2.0 \times 10^{-5}$  etc.)

**Character:** Under character data type, all alphabetic symbols and other symbols that are present in the key board are included. In general, all character data are stored in one byte of memory space.

**Integer:** under integer data type, all integer numbers both positive and negative are included. In programming in C, an integer data can occupy 2-bytes of memory spaces and the integers in the range of  $-2^{15}$  to  $2^{15} - 1$  are included.

**Floating point:** Floating point data type is used to represent real number values such as 2.5, 23.305, etc. It occupies 4-bytes of memory spaces and represent values from  $-2^{31}$  to  $2^{31} - 1$ .

**Double:** The data type double is used to represent double precision numbers such as  $2.5 \times 10^{15}$ ,  $3.5 \times 10^{-31}$  etc. It uses 8-byte of memory spaces.

##### 1.1.2 Derived Data types

Beyond the basic data types, C language has used some other derived data types, such as array, pointer, structure and union. They are basically derived from basic or fundamental data types. In other word, the data types that are derived from primitive data types are called derived data type.

##### 1.1.3 Abstract Data Types

An abstract data type is an abstraction of data and operations associated with. It defines a combination of a set of values and a set of operations. These ADTs have been implemented using "void" pointer by some modern C libraries such as `#include <lib/ADT.h>`, `#include <vector.h>`, `#include <RedBlackTree.h>`, `#include <stack.h>` etc. example of some Abstract Data Types are vector, stack, queue, graph etc.

##### 1.1.4 Importance of Data Structure

A Data Structure is a collection of data values and relations that organizes them. To organize a large scale of information, we need some well-defined structure through which we can manage and access our information in easy way. In computer Science, Data structures are categorized by Linear and Non-Linear ways. Linear Data structures are those where the data are stored sequentially in memory. Such as arrays, stacks, queues etc. again in non-linear data structure, data and its relationship between them are strictly not linear. Such as Tree, Graph etc.

Why Data structure is important?

Data Structures play a crucial role in many software developments. There are so many reasons that defines why it is important and some of them are as follows:

1. Performance: It increases the accessibility of data in an efficient way by choosing a reliable data structure.
2. Memory management: It helps to optimize memory uses, preventing wastes of spaces and improve program efficiency.
3. Reusability: A well-defined data structure can give the flexibility to reuse of codes across different parts of our program.
4. Data abstraction: It provides a way to abstract the underlying complexity of the problem etc.

## 1.2 Arrays:

Array is a structured derived data type that is used to store variables that are of same data type at contiguous locations. In other word an array is a data structure that stores a collection of elements, all of the same data type, in a contiguous block of memory. Each element in the array is identified by an index, which is a numerical value that represents the position of the element in the array. The first element in the array has an index of 0, the second element has an index of 1, and so on. Arrays are defined by a manner given below:

Type Array\_Name[Size];

Here-

- |            |                                                                    |
|------------|--------------------------------------------------------------------|
| Type       | is the data type of the elements that are to be stored             |
| Array-Name | is the name of the array though which the elements are accessible. |
| Size       | is the Maximum number of elements that the array holds.            |

There are mainly three types of array:

- a) One dimensional array
- b) Two-dimensional array
- c) Three-dimensional array

One-dimensional array:

One dimensional array is defined with a single subscript bracket as follows:

Type Array\_Name[Size];

Example:

char Name[20];

int A[50];

Two-dimensional array:

They are also known as Matrix array and defined with two subscript brackets as follows:

Int M[4][5];     a m x n array

Int SM[3][3];     a n x n square matrix array

Multi-dimensional array:

The arrays with more than two subscript brackets are known as multi-dimensional array.

Example:

```
Int space[4][5][5];
```

```
Float V[3][4][4];
```

#### 1.2.1 Characteristics:

The different characteristics of arrays are as follows:

1. Fixed size: array has a fixed size collection of elements.
2. Homogeneous: the elements of an array are of same data type.
3. Indexing: each array elements are accessed by indexed values associated with it.
4. Dimension: arrays has different numbers of dimensions.
5. Random Access: array elements can access randomly at any time.
6. The name of the array is related with pointer etc.

#### 1.2.2 Advantages:

There are lots of merits to use arrays in C and some of them are as follows:

1. Searching and accessing process an element is very easy.
2. It stores multiple values of same data type in a way of consecutive sequence.
3. C provides a set of library functions for manipulation.
4. C supports multiple dimensions of arrays.
5. Easily convertible to a pointer.

#### 1.2.3 Disadvantages

There are some demerits of array structure in C. such as:

1. Array size is fixed.
2. It is homogeneous, i.e. only one type of value can be store in the array.
3. Array is contiguous blocks of memory.
4. Insertion and deletion are not easy.

### 1.3 Address translation function

In an array structure, the array name holds the base address (first element address) itself. The addresses are calculated by different formulas using the base address for different types of arrays. The address calculation formula for 1-D array is:

$$\text{Address of } A[I] = B + W * (I - LB)$$

Where,

B        Base Address

W        Storage size of one element stored in the array in byte

I        index of the element whose address is to be calculate

LB       lower bound/limit of index. If not specified then assumed 0

Example: calculate the address of the array element A[5] of the array of integer. Consider the base address be 2235.

Solution:    Given base address B = 2235

Storage size of one element W = 4 bytes

Lower bound index LB = 0

Element index I = 5 (given)

$$\begin{aligned}\text{Therefor Address of } A[5] &= 2235 + 4 * (5 - 0) \\ &= 2235 + 20 \\ &= 2255\end{aligned}$$

Ans: 2255.

2-dimensional arrays are considered as array of arrays and are organized as matrices wich can be represented as the collection of rows and columns as array[M][N] where M is the number of rows and N is the number of columns.

To find the address of any element in a 2-D array, there are two different ways

1. Row major order
2. Column major order

### 1.3.1 Row Major ordering

To find the address of an element using row-major ordering uses the formula:

$$\text{Address of } A[I][J] = B + W * ((I - LR) * N + (J - LC))$$

Where,

|    |                                                           |
|----|-----------------------------------------------------------|
| I  | row index of the element whose address to be found        |
| J  | column index of the element whose element to be found     |
| B  | base address                                              |
| W  | storage size of one element stored in the array (in byte) |
| LR | Lower limit / starting row index of the array             |
| LC | lower limit / starting column index of the array          |
| N  | number of columns given in the matrix                     |

Example: For given an array of integer M of size 4 X 5 in row-major ordering, find the address of the element M[3][2]. Consider the base address is 2243 and starting element is at M[0][0].

Solution: Given,

Row index I = 3

Column index J = 2

Base address B = 2243

Size of integer W = 4 bytes

Starting index for row LR = 0

Starting index for column = 0

Number of column N = 5

$$\begin{aligned} \text{Therefore Address of } M[3][2] &= 2243 + 4 * ((3 - 0) * 5 + (2 - 0)) \\ &= 2243 + 4 * (15 + 2) \\ &= 2243 + 4 * 17 \\ &= 2243 + 68 \\ &= 2311 \end{aligned}$$

Answer: 2311

### 1.3.2 Column Major Ordering

If the elements of an array are stored in column-major ordering then the address of the elements are calculated using the formula:

$$\text{Address of } A[I][J] = B + W * ((J - LC) * M + (I - LR))$$

Where,

|    |                                                           |
|----|-----------------------------------------------------------|
| I  | row index of the element whose address to be found        |
| J  | column index of the element whose element to be found     |
| B  | base address                                              |
| W  | storage size of one element stored in the array (in byte) |
| LR | Lower limit / starting row index of the array             |
| LC | lower limit / starting column index of the array          |
| M  | number of rows given in the matrix                        |

Example: For given an array of integer M of size 4 X 5 in column-major ordering, find the address of the element M[3][2]. Consider the base address is 2243 and starting element is at M[0][0].

Solution: Given,

Row index I = 3

Column index J = 2

Base address B = 2243

Size of integer  $W = 4$  bytes

Starting index for row  $LR = 0$

Starting index for column  $= 0$

Number of rows  $M = 4$

$$\begin{aligned}\text{Therefore Address of } M[3][2] &= 2243 + 4 * ((2 - 0) * 4 + (3 - 0)) \\ &= 2243 + 4 * (8 + 3) \\ &= 2243 + 4 * 11 \\ &= 2243 + 44 \\ &= 2287\end{aligned}$$

Answer: 2287

#### 1.4 Insertion and deletion in Arrays

Insertion and Deletion operations in arrays are done by various ways in various situations. We can access the array elements in random way, so we can insert or delete an element at position. In this chapter, we will discuss both the operations for different cases.

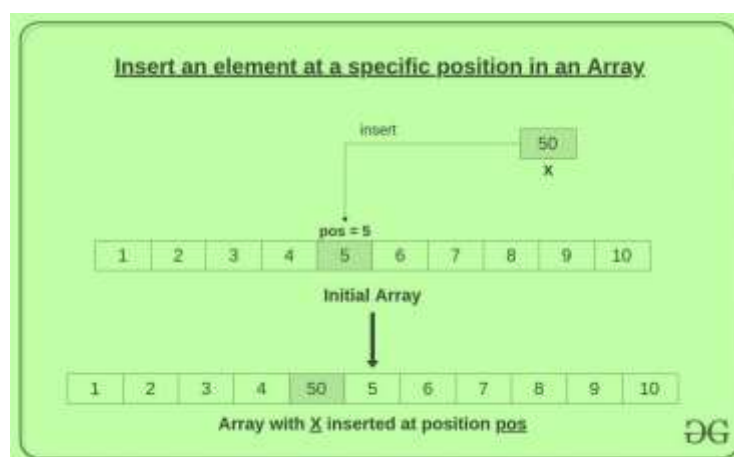
##### 1. Insertion at the end of the array:

In this case, we directly have to move to the first empty position and insert the element.



##### 2. Insertion at any position:

In this case we have to find the position first. After that, move the succeeding elements to the right by one place for each. Then insert it.



#C program to insert at any position:

```
#include <stdio.h>
```

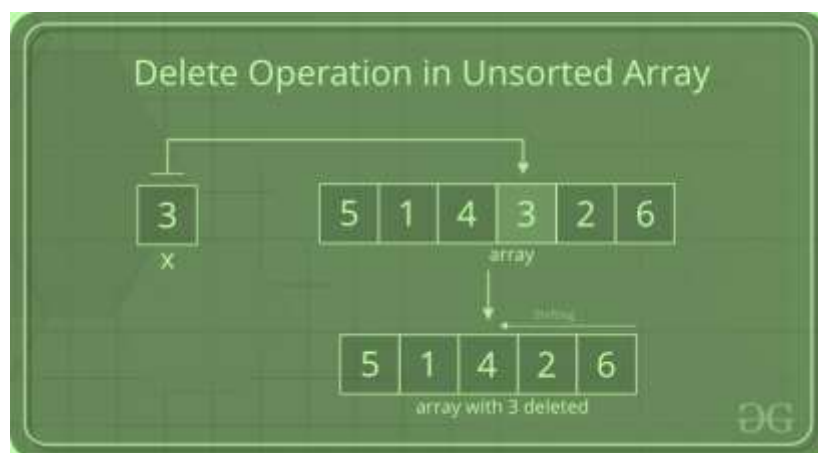
```
// Function to insert element at a specific position
void insertElement(int arr[], int n, int x, int pos)
{
    // shift elements to the right which are on the right side of pos
    for (int i = n - 1; i >= pos; i--)
        arr[i + 1] = arr[i];
    arr[pos] = x;
}

int main()
{
    int arr[15] = { 2, 4, 1, 8, 5 };
    int n = 5;

    printf("Before insertion : ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
    int x = 10, pos = 2;
    // Function call
    insertElement(arr, n, x, pos);
    n++;
    printf("After insertion : ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

### 3. Delete an element from the array:

In deletion operation, the element to be deleted is searched using linear search, and then succeeding elements can be shifted left by one place and decreased the number of element in the array by one.



#C program to delete an element

```

#include <stdio.h>
    // To search a key to be deleted
int findElement(int arr[], int n, int key);
    // Function to delete an element
int deleteElement(int arr[], int n, int key)
{
    // Find position of element to be deleted
    int pos = findElement(arr, n, key);
    if (pos == -1) {
        printf("Element not found");
        return n;
    }
    // Deleting element
    int i;
    for (i = pos; i < n - 1; i++)
        arr[i] = arr[i + 1];
    return n - 1;
}

    // Function to implement search operation
int findElement(int arr[], int n, int key)
{
    int i;
    for (i = 0; i < n; i++)
        if (arr[i] == key)
            return i;
    return -1;
}

    // Driver's code
int main()
{
    int i;
    int arr[] = { 10, 50, 30, 40, 20 };
    int n = sizeof(arr) / sizeof(arr[0]);
    int key = 30;
    printf("Array before deletion\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    // Function call
    n = deleteElement(arr, n, key);
    printf("\nArray after deletion\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}

```

### 1.5 Use of arrays for large number representation:

In C, an integer is typically 32 bits, of which 31 bits can be used for positive integer numbers up to  $2^{31} - 1$  (2 times  $10^9$ ). Some compilers offer a “long long” type, which gives 64 bits capable of representing about 9 times of  $10^{18}$ . But some applications require many more digits than this, such as a number with 50 digits long or  $50!$  =

30,414,093,201,713,378,043,612,608,166,064,768,844,377,641,568,960,512,000,000,000,000. Which will not fit into a C integer, not even a 64 bit one.

So, we must have to move to a different representation of non-negative numbers. For such cases, we can represent the number as a sequence of digits stored in an array of integer and can write functions to perform operations on that as large as we want.

In our new representation, we have an array of "digits" (integers) in some base  $b$ . Typically,  $b = 10$ , our normal decimal number system; that makes things easy to print. The 0'th array element is the 1's place, the #1 element is the ten's place, the #2 element is the hundred's place, and so forth (really, the  $b^0$ 's place, the  $b^1$  place, the  $b^2$  place, etc.).

Let's look at some algorithms doing arithmetic on our new "big" integers. We'll let  $N$  be the number of digits we will represent. If we need any more, we just increase  $N$ . Let BASE be the base of our number system, understood to be 10 typically, but can be changed if we like.

First, we need a way of making a "normal" integer into a "big" integer; we'd like a function called `make_int` such that e.g. `make_int (A, 123)` would put 3 in `A[0]`, 2 in `A[1]`, 1 in `A[2]`, and zeros in `A[3..N-1]`. We'll do these in C rather than pseudocode because the code works out very easily:

```
/* put the normal int n into the big int A */
void make_int (int A[], int n) {
    int i;
    /* start indexing at the 0's place */
    i = 0;
    /* while there is still something left to the number, we're encoding... */
    while (n) {
        /* put the least significant digit of n into A[i] */
        A[i++] = n % BASE;
        /* get rid of the least significant digit, i.e., shift right once */
        n /= BASE;
    }
    /* fill the rest of the array up with zeros */
    while (i < N) A[i++] = 0;
}
```

Note: Another way to do this is by reading the sequence of digits in a character array and store it in an array of integer starting from last character into first array element position.

## Unit 2: Linked Lists

### 2.1 Structure:



Structure is a user-defined data type that can be used to group and store the collection of different data types. The **struct** keyword is used to define the structure in C. Each elements of a structure are called a member. In addition, the values of a structure are stored sequentially in memory. In C, structure are defined as follows:

```
struct structure_name
{
    data_type member1;

    data_type member2;

    .....

    .....

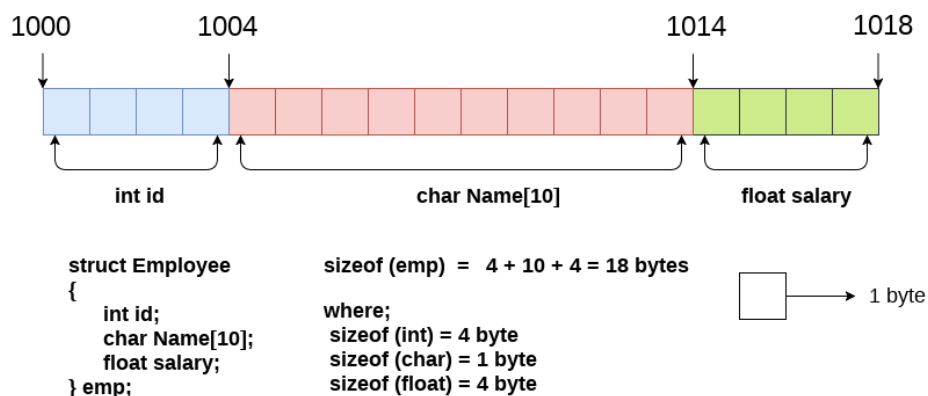
    data_type member;
};
```

Example:

```
struct employee
{
    int id;

    char name[20];

    float salary;
};
```



We can declare a structure variable for the structure in two ways;

1. Within main() by struct keyword

Example: struct employee e1, e2;

The variable e1 and e2 can be used to access the value stored in the structure by using dot (.) operator.

e1.id = 123;

e1.name = "Rabin";

2. by declaring a variable at the time of defining the structure

Example:

```

        struct employee
    {
        int id;
        char name[20];
        float salary;
    }e1,e2;

```

Example of C structure:

```

#include<stdio.h>
#include <string.h>
struct employee    //declaring structure
{   int id;
    char name[50];
}e1; //declaring e1 variable for structure
int main( )
{
    //store first employee information
    e1.id=101;
    strcpy(e1.name, "Sonoo Jaiswal"); //copying string into char array
    //printing first employee information
    printf( "employee 1 id : %d\n", e1.id);
    printf( "employee 1 name : %s\n", e1.name);
return 0;
}

```

### Output:

```

employee 1 id: 101
employee 1 name: Sonoo Jaiswal

```

#### 2.1.1 Self-referential Structure:

The structure that refer the same structure itself is known as self-referential structure. It contains one or more pointers that points the same structure itself.

Example:

```

struct node
{
    int data1;

```

```

char data2;

struct node *link;

};

```

**Note:** It plays a vital role in linked-lists, trees, graphs and many other data structures.

## 2.2 Pointer

Pointer is a derived data type that can be used to store the Address of other variables or a memory location. It is declared by a referencing operator (\*) with a proper data type as follow:

```
datatype *ptr;
```

Where,

ptr is the name of the pointer

datatype is the type of data it is pointing to

Pointers are initialized by using assignment operator (=) along with the Addressof operator (&) to get the memory address of a variable.

Example:

```

int var = 10;
int *ptr;
ptr = &var;

```

// C program to explain use of pointer

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int number=50;
```

```
    int *p;
```

```
    p=&number;           //stores the address of number variable
```

```
    // p contains the address of the number therefore printing p gives the address
    of number.
```

```
    printf("Address of p variable is %x \n",p);
```

```
    // As we know that * is used to dereference a pointer therefore if we print *p,
    we will get the value stored at the address contained by p.
```

```
    printf("Value of p variable is %d \n",*p);
```

```
    return 0;
```

```
}
```

## 2.3 Initialization and implementation of structure:

There are two ways to access a structure member;

1. Using **Dot (.)** operator (for structure variable)
2. Using **Pointer (->)** operator (for structure pointer)

To illustrate these, let we discuss two programs in C.

Program 1: For a structure variable that use **Dot (.)** operator

```
#include<stdio.h>
#include <string.h>
struct employee
{   int id;
    char name[50];
    float salary;
}e1,e2;    //declaring e1 and e2 variables for structure
int main( )
{
    //store first employee information
    e1.id=101;
    strcpy(e1.name, "Sonoo Jaiswal");    //copying string into char array
    e1.salary=56000;
    //store second employee information
    e2.id=102;
    strcpy(e2.name, "James Bond");
    e2.salary=126000;
    //printing first employee information
    printf( "employee 1 id : %d\n", e1.id);
    printf( "employee 1 name : %s\n", e1.name);
    printf( "employee 1 salary : %f\n", e1.salary);
    //printing second employee information
    printf( "employee 2 id : %d\n", e2.id);
    printf( "employee 2 name : %s\n", e2.name);
    printf( "employee 2 salary : %f\n", e2.salary);
    return 0;
}
```

**Output:**

```
employee 1 id : 101
employee 1 name : Sonoo Jaiswal
employee 1 salary : 56000.000000
employee 2 id : 102
employee 2 name : James Bond
employee 2 salary : 126000.000000
```

Program 2: For a structure pointer that use **pointer (->)** operator

```
#include <stdio.h>
#include <string.h>
```

```

// Creating Structure Student
struct Student {
    int roll_no;
    char name[30];
    char branch[40];
    int batch;
};
// variable of structure with pointer defined
struct Student s, *ptr;
int main()
{
    ptr = &s;
    // Taking inputs
    printf("Enter the Roll Number of Student\n");
    scanf("%d", &ptr->roll_no);
    printf("Enter Name of Student\n");
    scanf("%s", &ptr->name);
    printf("Enter Branch of Student\n");
    scanf("%s", &ptr->branch);
    printf("Enter batch of Student\n");
    scanf("%d", &ptr->batch);
    // Displaying details of the student
    printf("\nStudent details are: \n");
    printf("Roll No: %d\n", ptr->roll_no);
    printf("Name: %s\n", ptr->name);
    printf("Branch: %s\n", ptr->branch);
    printf("Batch: %d\n", ptr->batch);
    return 0;
}

```

#### **Output:**

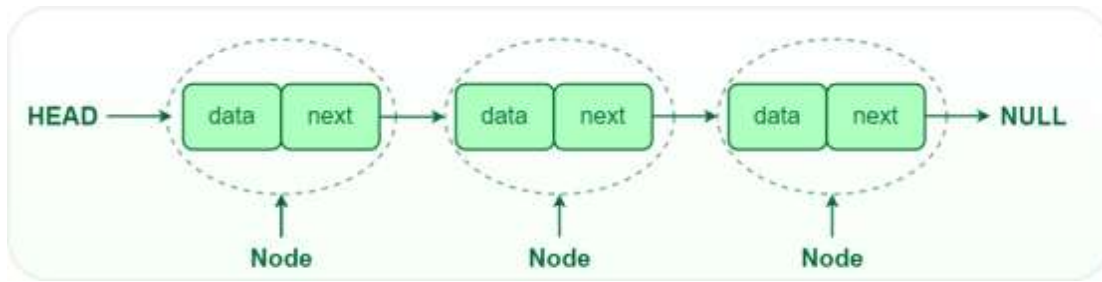
```

Enter the Roll Number of Student
27
Enter Name of Student
Kamlesh_Joshi
Enter Branch of Student
Computer_Science_And_Engineering
Enter batch of Student
2019
Student details are:
Roll No: 27
Name: Kamlesh_Joshi
Branch: Computer_Science_And_Engineering
Batch: 2019

```

## **2.4 Introduction to Link Lists:**

**Linked List** is a linear data structure, in which elements are not stored at a contiguous location, rather they are linked using pointers. Linked List forms a series of connected nodes, where each node stores the data and the address of the next node.



**Node Structure:** A node in a linked list typically consists of two components:

**Data:** It holds the actual value or data associated with the node.

**Next Pointer:** It stores the memory address (reference) of the next node in the sequence.

**Head and Tail:** The linked list is accessed through the head node, which points to the first node in the list. The last node in the list points to NULL or nullptr, indicating the end of the list. This node is known as the tail node.

Some advantages of Linked-Lists are as follows:

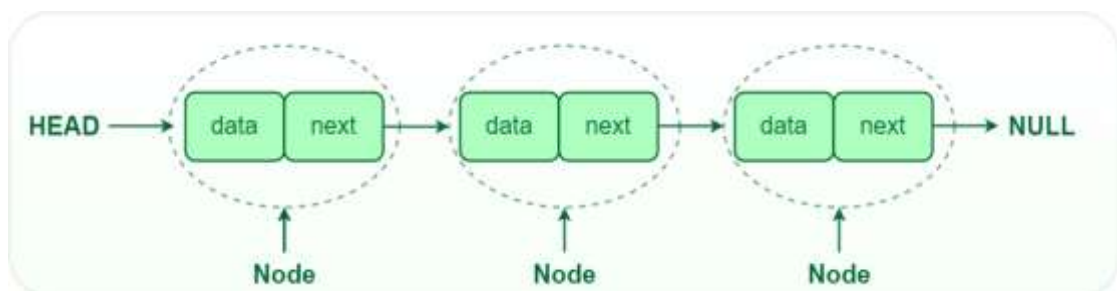
- **Dynamic Data structure:** The size of memory can be allocated or de-allocated at run time based on the operation insertion or deletion.
- **Ease of Insertion/Deletion:** The insertion and deletion of elements are simpler than arrays since no elements need to be shifted after insertion and deletion, Just the address needed to be updated.
- **Efficient Memory Utilization:** As we know Linked List is a dynamic data structure the size increases or decreases as per the requirement so this avoids the wastage of memory.
- **Implementation:** Various advanced data structures can be implemented using a linked list like a stack, queue, graph, hash maps, etc.

There are mainly three types of Linked-List;

1. Single-linked list
2. Double linked list
3. Circular linked list

#### 2.4.1 Singly Linked List:

In a singly linked list, each node contains a reference to the next node in the sequence. Traversing a singly linked list is done in a forward direction.



## Single-linked list

```
struct node
{
    int data;
    struct node *next;
} head;
```

### 2.4.2 Doubly Linked List

In a doubly linked list, each node contains references to both the next and previous nodes. This allows for traversal in both forward and backward directions, but it requires additional memory for the backward reference.

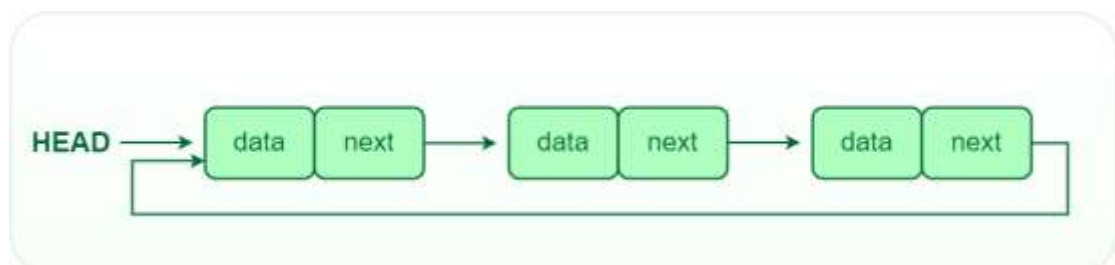


## Double-linked list

```
struct node
{
    int data;
    struct node *next;
    struct node *prev;
}head;
```

### 2.4.3 Circular Linked List

In a circular linked list, the last node points back to the head node, creating a circular structure. It can be either singly or doubly linked.



## 2.5 Operations on Linked Lists

There are various operations associated on Linked-Lists.

### 2.5.1 Creation:

The nodes of a linked-Lists are created by the keyword **struct** as follows;

```
struct node
{
    int data;
    struct node *next;
} head;
```

Here,

node – is the name of the Linked-List

head – is the starting/first element of the linked-list

data – is the data field which to be used to hold the value

next – is a pointer that points to next node of the linked-list

### 2.5.2 Insertion:

Insertion is the process of adding a new node to a linked-list and it involves adjusting the pointer of the existing nodes to maintain the proper sequence. Insertion can be performed at the beginning, end or any other position within the list.

// inserting a node at the end of list

```
int Insert_Node(int new_value)
{
    struct node *new_node, *temp = head;

    new_node->data=new_value;
    new_node->next = NULL;
    if(temp)
    {
        while(temp->next)
            temp = temp->next;
        temp->next = new_node;
    }
    else
        head = new_node;
    return(0);
}
```



```
}
```

### 2.5.3 Deletion:

Removing a node from a linked list requires adjusting the pointers of the neighbouring nodes to bridge the gap left by the deleted node. Deletion can be performed at the beginning, end, or any position within the list.

// removing a node from a singly linked-list

```
int delete_node(int key)
{
    struct node *temp = head, *p;
    while(temp->data != key)
    {
        P = temp;
        temp = temp->next;
    }
    if(temp == head)
        head = head->next;
    else
        p->next = temp->next;
    free(temp);
    return(temp->data);
}
```

### 2.5.4 Traversal:

Searching for a specific value in a linked list involves traversing the list from the head node until the value is found or the end of the list is reached.

// traversing for an element in the linked-list and return the position where it exists.

```
int search_node(int key)
{
    struct node *temp = head;
    int pos = 0;
    do{
        pos ++;
```

```

        if(temp->data == key)
            break;
        temp = temp->next;
    while(temp);
    return(pos);
}

```

#### 2.5.5 Merging:

We can merge two linked-list to get a new list by simply joining the second list at the end of first list.

```

// merging two linked-lists
struct List Merging(struct List *head1, struct List *head2)
{
    struct List *temp = head1;
    while(temp->next)
        temp = temp->next;
    temp->next = head2;
    return(temp);
}

```

#### 2.5.6 Splitting:

Splitting a linked-list in two lists of size ratio p:q; i. e. the first list contains first p nodes and the second list contains the rest of the q nodes. Here, first we need to find the length of the linked-list. If the total ratio sum exceeds the actual length then dividing the list is not possible so print -1. If it is possible to divide the list then simply traverse the list up to length p and break it into the ratio p:q. Next node will be the head of the second list.

```

// splitting a linked-list in the ratio of p:q
struct List split(node *head, int p, int q)
{
    int n = 0;
    node *temp;
    temp = head;
    // Find the length of the list
    while (temp != NULL)
    {
        n += 1;
        temp = temp->next;
    }
    // If ratio exceeds the actual length

```

```

if (p + q > n)
{
    Printf("..... Split operation not possible..... ");
    return(NULL);
}
temp = head;
while (p > 1)
{
    temp = temp->next;
    p -= 1;
}
// second head node after splitting
node *head2 = temp->next;
temp->next = NULL;
//returning the second new list head to the calling function
return(head2);
}

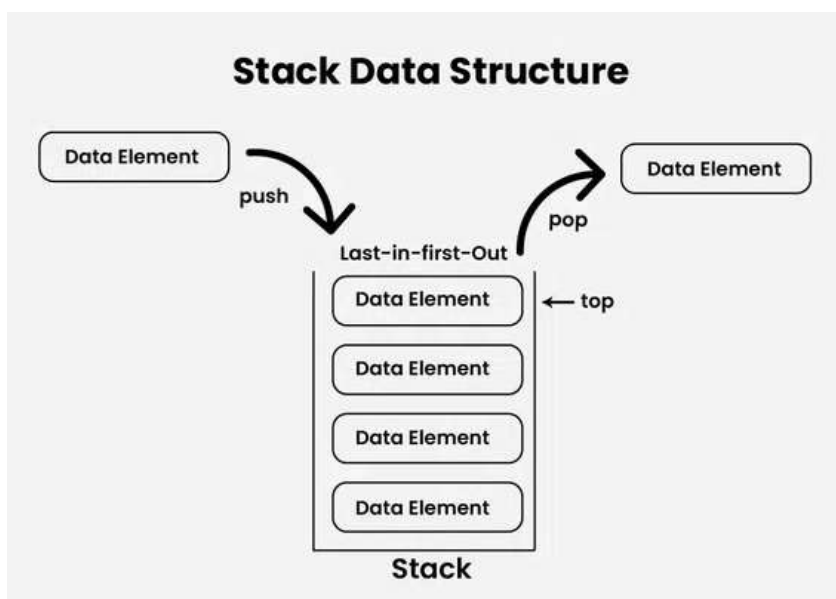
```

### Unit 3:

#### Stacks and queues

##### 3.1. Stack:

A stack is a linear data structure that follows the principle of Last In First Out (LIFO). This means the last element inserted in the stack is removed first.



We can construct a stack in C as follows;

```
#define Max_Size 100;
typedef struct
{
    int arr[Max_Size];
    int top;
} Stack;
```

Here, we have defined a structure called **Stack**, which consist of an array of integer named **arr** and an integer **top** used to identify the element that is now at the top. The **Max\_Size** constant denotes the stack's maximum capacity. The insertion and deletion operations are performed only at **top** position and initially the integer **top** is assigned by -1 (Empty stack).

### 3.1.1 Representation of stack using array:

A stack can be represented both using array and linked-list. In array representation, the stack is formed by using the array and all the operations regarding the stack are performed using array. It can be implemented with step-by-step approach as follows:

1. Initialize an array to represent the stack
2. Use the end of the array to represent the top of the stack
3. Implement push(add element) and pop (remove element), and peak (check end) operations, ensuring to handle full and empty stack conditions.

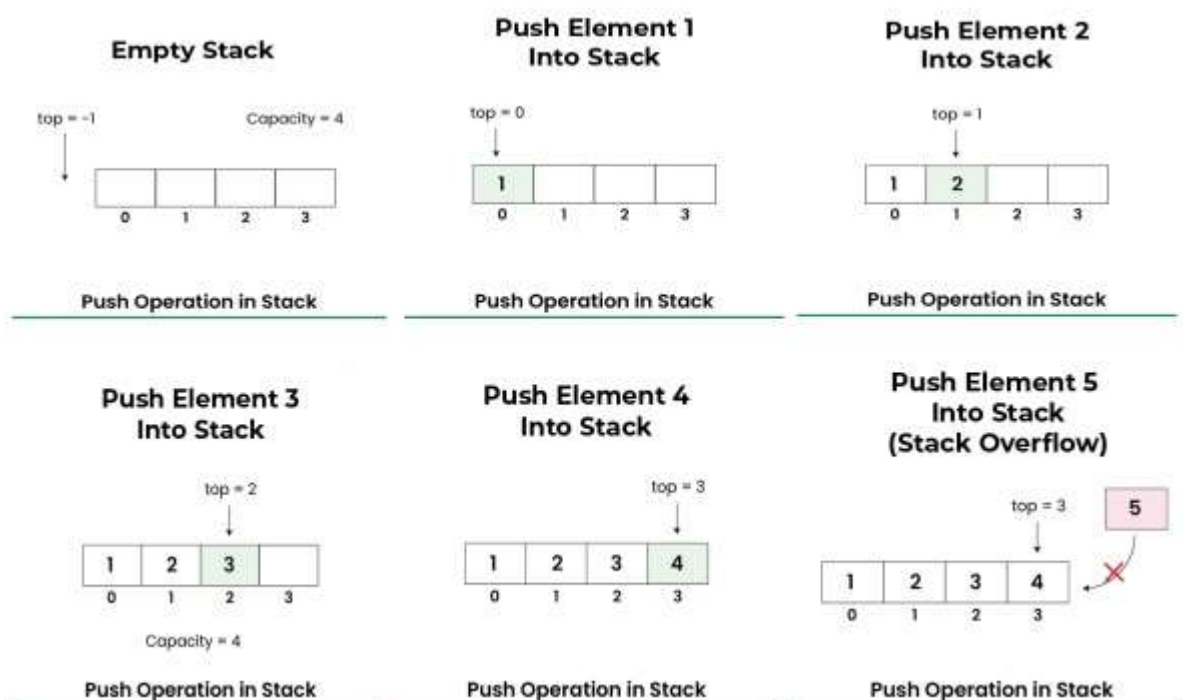
### 3.1.2 Operations on stack on the basis of array representation:

The different operations performed on a stack are as follows:

- a. **Push Operation:** It is used to add an item to the stack. If the stack is already full, then it is said to be an overflow condition.

**Algorithm for Push Operation:**

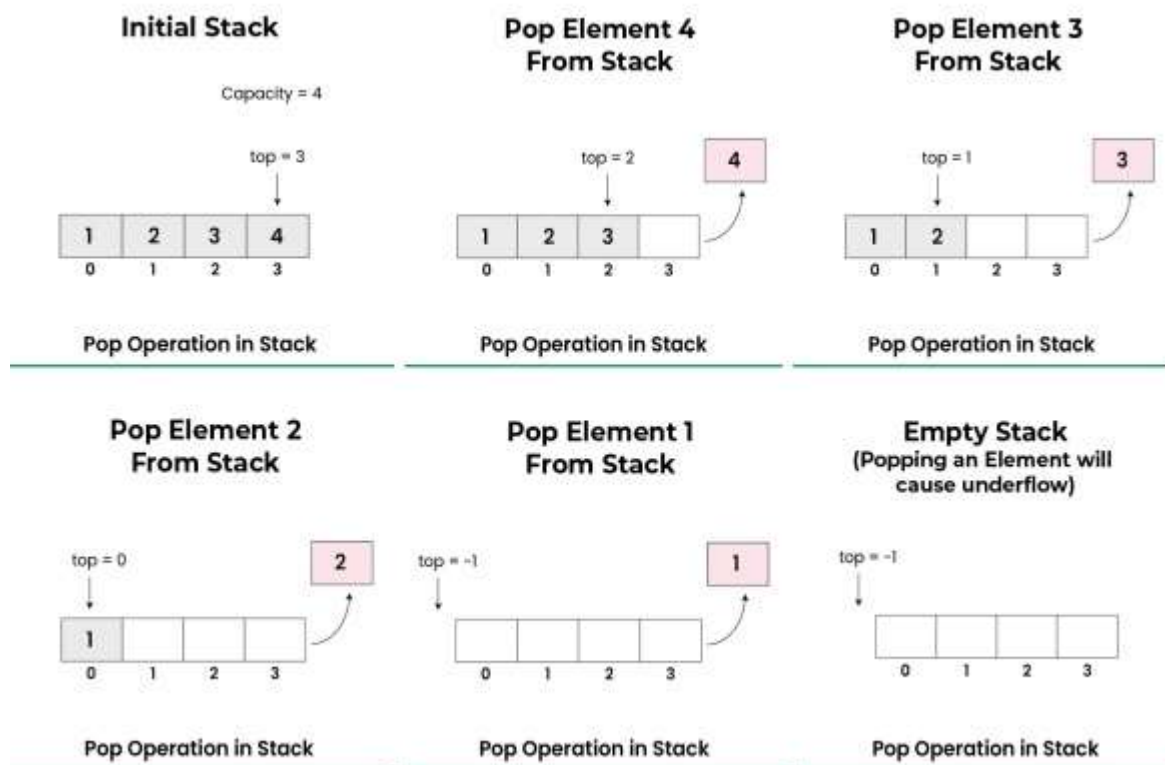
1. Before pushing the element to the stack, we check if the stack is **full**.
2. If the stack is full (**top == capacity-1**), then **Stack Overflows** and we cannot insert the element to the stack.
3. Otherwise, we increas the value of top by 1 (**top = top + 1**) and the new value is inserted at **top position**.
4. The elements can be pushed into the stack till we reach the **capacity** of the stack.



- b. **Pop Operation:** This operation is used to remove an item from the stack. The items are popped in the reversed order in which they are pushed. If the stack is empty, then it is said to be an underflow condition.

**Algorithm for pop operation:**

1. Before popping the element from the stack, we check if the stack is **empty**.
2. If the stack is empty ( $top == -1$ ), then **Stack Underflows** and we cannot remove any element from the stack.
3. Otherwise, we store the value at  $top$ , decrement the value of  $top$  by 1 ( $top = top - 1$ ) and return the stored  $top$  value.



- c. **Top or Peek Operation:** It returns the top element of the stack.

**Algorithm:**

1. Before returning the top element from the stack, we check if the stack is empty.
2. If the stack is empty ( $top == -1$ ), we simply print "Stack is empty, Underflow condition".
3. Otherwise, we return the top element stored in **index = top**.

- d. **isEmpty operation:** It returns **true** if the stack is empty, else return **false**

**Algorithm:**

1. check for the value of  $top$  in stack.

2. If (top == -1), then the stack is empty so return **true**.
3. Otherwise, the stack is not empty so return **false**.

e. isFull operation: It is used to check whether the stack is **full** or not. It returns **true** if the stack is full, else **false**.

Algorithm:

1. check for the value of top in stack.
2. If (top == capacity - 1), then the stack is **full** so return **true**.
3. Otherwise, the stack is not full so return **false**.

```
// C program for array implementation of stack
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>

// A structure to represent a stack
struct Stack {
    int top;
    unsigned capacity;
    int* array;
};

// function to create a stack of given capacity. It initializes size of
// stack as 0
struct Stack* createStack(unsigned capacity)
{
    struct Stack* stack = (struct Stack*)malloc(sizeof(struct Stack));
    stack->capacity = capacity;
    stack->top = -1;
    stack->array = (int*)malloc(stack->capacity * sizeof(int));
    return stack;
}

// Stack is full when top is equal to the last index
int isFull(struct Stack* stack)
{
    return stack->top == stack->capacity - 1;
}

// Stack is empty when top is equal to -1
int isEmpty(struct Stack* stack)
{
    return stack->top == -1;
}

// Function to add an item to stack. It increases top by 1
void push(struct Stack* stack, int item)
{
    if (isFull(stack))
        return;
    stack->array[++stack->top] = item;
```

```

    printf("%d pushed to stack\n", item);
}

// Function to remove an item from stack. It decreases top by 1
int pop(struct Stack* stack)
{
    if (isEmpty(stack))
        return INT_MIN;
    return stack->array[stack->top--];
}

// Function to return the top from stack without removing it
int peek(struct Stack* stack)
{
    if (isEmpty(stack))
        return INT_MIN;
    return stack->array[stack->top];
}

// Driver program to test above functions
int main()
{
    struct Stack* stack = createStack(100);

    push(stack, 10);
    push(stack, 20);
    push(stack, 30);

    printf("%d popped from stack\n", pop(stack));

    return 0;
}

```

## Output

```

10 pushed into stack
20 pushed into stack
30 pushed into stack
30 Popped from stack
Top element is : 20
Elements present in stack : 20 10

```

### Advantages of array representation:

1. Easy to implement.
2. Memory is saved as pointers are not involved.

### Disadvantages of array representation:

1. It is not dynamic.
2. The total size of the stack must be defined beforehand.

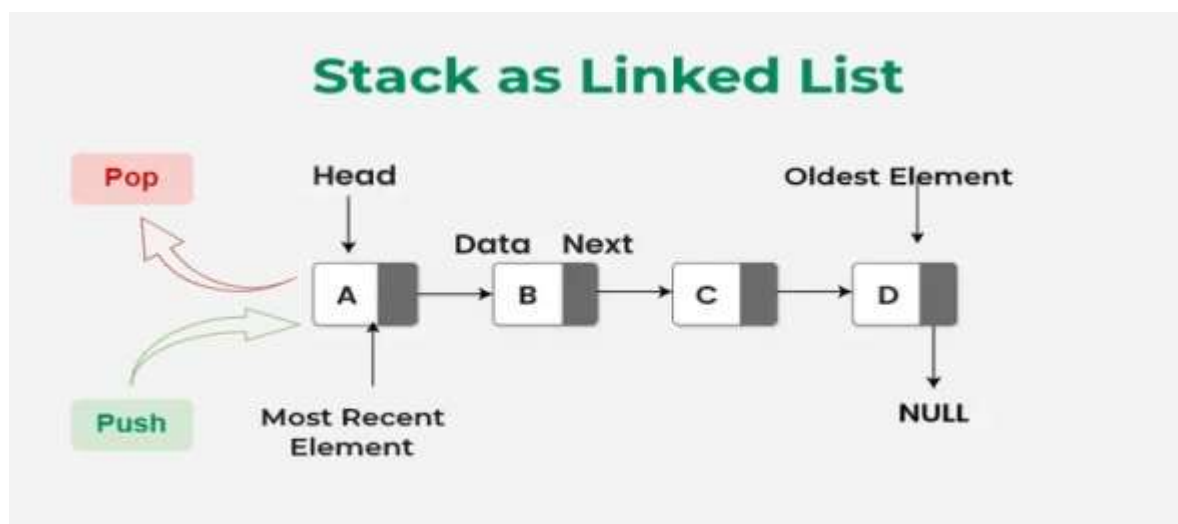
### 3.1.3 Representation of Stack using linked-List:

Stack is generally implemented using an array but the limitation of this kind of stack is that the memory occupied by the array is fixed no matter what are the number of elements in the stack. In the stack implemented using linked list implementation, the size occupied by the linked list will be equal to the number of elements in the stack. Moreover, its size is dynamic. It means that the size is gonna change automatically according to the elements present.

In C, the stack that is implemented using a linked list can be represented by the pointer to the head node of the linked list. Each node in that linked list represents the element of the stack. The type of linked list here is a singly linked list in which each node consists of a data field and the next pointer.

```
struct Node
{
    type data;
    struct Node *next;
};
```

The type of the data can be defined according to the requirement.



#### 3.1.4 Operations of stack on the basis of linked-list representation:

Following are the basic operations of stack in linked-list representation:

**Push Operation:** It is used to add a new element into the stack. The new elements are inserted at the beginning of the linked-list. At every new insertion, the newly inserted element is the head.

**Algorithm:**

3. Create a new node with the given data.
4. Insert the node before the head of the linked-list.
5. Update the head of the linked-list to the new node.

Here, the stack overflow will only occur if there is some error allocating the memory in the heap so the check will be done in the creation of the new node.



**Pop Operation:** It is used to remove the most recent element from the stack which is pointed by the head node. Here, we need to check for the stack underflow first.

**Algorithm:**

1. Check if the stack is empty.
2. If not empty, store the top node in a temporary variable.
3. Update the head pointer to the next node.
4. Free the temporary node.

**Peek Operation:** the peek operation will return the topmost element of the stack if the stack is not empty. The topmost element means the element at the head.

**Algorithm:**

1. Check if the stack is empty.
2. If it is empty, return -1.
3. Else return the head->data.

**IsEmpty Operation:** this operation is used to check if the stack is empty or not. It returns **true** if the stack is empty, otherwise, it returns **false**.

**Algorithm:**

1. Check if the top pointer of the stack is NULL.
2. If NULL, return **true**, indicating the stack is empty.
3. Otherwise return **false** indicating the stack is not empty.

```
// C program to implement a stack using linked list
#include <stdio.h>
#include <stdlib.h>

// _____LINKED LIST UTILITY FUNCITON_____

// Define the structure for a node of the linked list
typedef struct Node {
    int data;
    struct Node* next;
} node;

// linked list utility function
node* createNode(int data)
{
    // allocating memory
    node* newNode = (node*)malloc(sizeof(node));

    // if memory allocation is failed
    if (newNode == NULL)
        return NULL;
```

```

    // putting data in the node
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// fuction to insert data before the head node
int insertBeforeHead(node** head, int data)
{
    // creating new node
    node* newNode = createNode(data);
    // if malloc fail, return error code
    if (!newNode)
        return -1;
    // if the linked list is empty
    if (*head == NULL) {
        *head = newNode;
        return 0;
    }
    newNode->next = *head;
    *head = newNode;
    return 0;
}

// deleting head node
int deleteHead(node** head)
{
    // no need to check for empty stack as it is already
    // being checked in the caller function
    node* temp = *head;
    *head = (*head)->next;
    free(temp);
    return 0;
}

// _____STACK IMPLEMENTATION STARTS HERE_____
// Function to check if the stack is empty or not
int isEmpty(node** stack) { return *stack == NULL; }
// Function to push elements to the stack
void push(node** stack, int data)
{
    // inserting the data at the beginning of the linked
    // list stack
    // if the insertion function returns the non - zero
    // value, it is the case of stack overflow
    if (insertBeforeHead(stack, data)) {
        printf("Stack Overflow!\n");
    }
}

// Function to pop an element from the stack
int pop(node** stack)
{
    // checking underflow condition
    if (isEmpty(stack)) {
        printf("Stack Underflow!\n");
        return -1;
    }
}

```

```

    }
    // deleting the head.
    deleteHead(stack);
}
// Function to return the topmost element of the stack
int peek(node** stack)
{
    // check for empty stack
    if (!isEmpty(stack))
        return (*stack)->data;
    else
        return -1;
}
// Function to print the Stack
void printStack(node** stack)
{
    node* temp = *stack;
    while (temp != NULL) {
        printf("%d-> ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
/ driver code
int main()
{
    // Initialize a new stack top pointer
    node* stack = NULL;

    // Push elements into the stack
    push(&stack, 10);
    push(&stack, 20);
    push(&stack, 30);
    push(&stack, 40);
    push(&stack, 50);
    // Print the stack
    printf("Stack: ");
    printStack(&stack);
    // Pop elements from the stack
    pop(&stack);
    pop(&stack);
    // Print the stack after deletion of elements
    printf("\nStack: ");
    printStack(&stack);
    return 0;
}

```

## Output

Stack: 50-> 40-> 30-> 20-> 10->

Stack: 30-> 20-> 10->

Benefits of Linked-list use for stack:

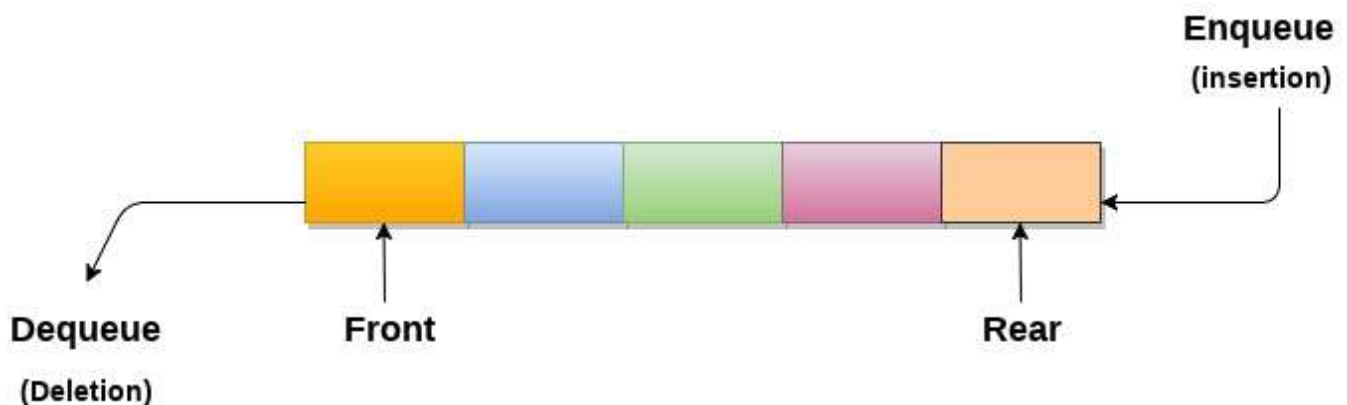
1. The dynamic memory management of linked-list provide dynamic size to the stack that changes with the change in the number of elements.
2. Rarely reaches the condition of the stack overflow.

### 3.1.5 Applications of stack:

1. **Function calls:** Stacks are used to keep track of the return addresses of function calls, allowing the program to return to the correct location after a function has finished executing.
2. **Recursion:** Stacks are used to store the local variables and return addresses of recursive function calls, allowing the program to keep track of the current state of the recursion.
3. **Expression evaluation:** Stacks are used to evaluate expressions in postfix notation (Reverse Polish Notation).
4. **Syntax parsing:** Stacks are used to check the validity of syntax in programming languages and other formal languages.
5. **Memory management:** Stacks are used to allocate and manage memory in some operating systems and programming languages.

### 3.2 Queue:

A queue is a linear data structure that follows the **First-In-First-Out (FIFO)** principle. It operates like a line where elements are added at one end called **rear** and removed from the other end called **front**.



#### 3.2.1 Representation of queue using array: To implement a queue using an array:

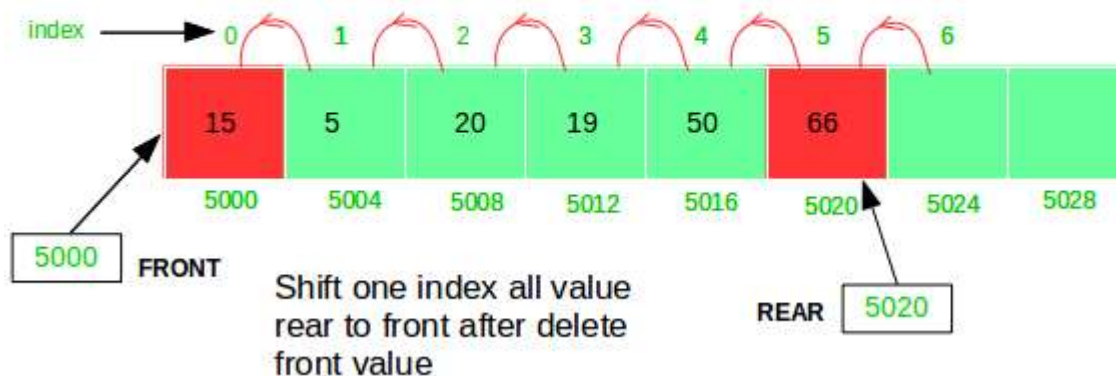
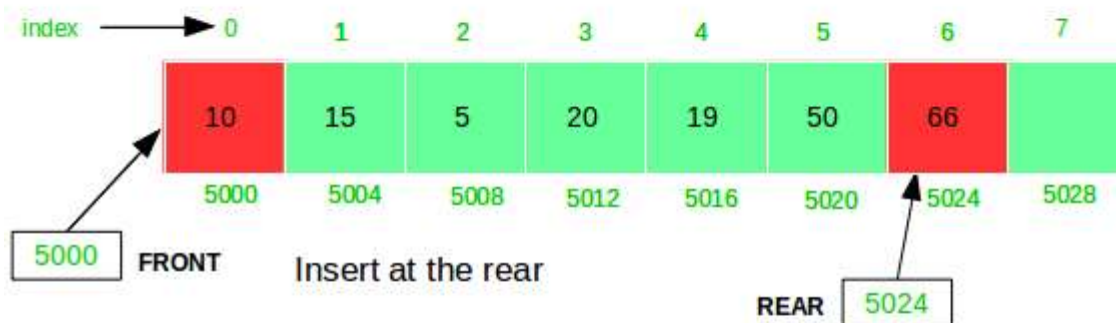
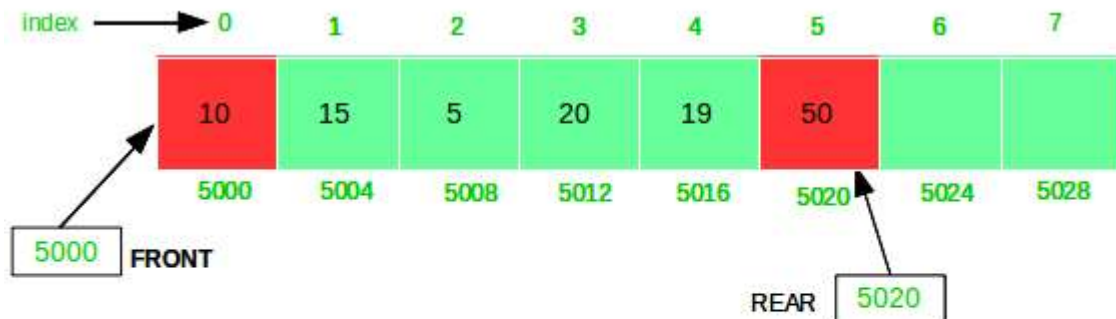
1. Create an array arr of size n,
2. Take two variables front and rear both of which will be initialised to -1 which means the queue is currently empty,
3. Element
  - ✓ Rear is the index up to which the elements are stored in the array including the rear index itself. By incrementing this value we can add an item.
  - ✓ Front is the index of the first element of the array and we remove the element at this index.

#### 3.2.2 Some operations of queue are as follows:

1. **Enqueue:** Addition of an element to the queue. Adding an element will be performed after checking whether the queue is full or not. If  $\text{rear} == \text{capacity} - 1$ , then it is said to be an

Overflow condition as the array is full. If array is not full, then store the element at  $\text{arr}[\text{rear}]$  and increment rear by 1.

2. **Dequeue:** Removal of an element from the queue. An element can only be deleted when there is at least an element to delete. If  $\text{rear} == -1$ , then it is said to be an Underflow condition as the array is empty. If the element at  $\text{arr}[\text{front}]$  can be deleted then move all the remaining elements to the left by one position in order to maintain the FIFO (the first element must be deleted).



```
// C Program to implement queue using arrays
#include <stdio.h>
// Define the maximum size for the queue
#define MAX_SIZE 100

// Define a structure for the queue
struct Queue {
    int queue[MAX_SIZE];
    int front;
    int rear;
};
```

```

};

// Function to initialize the queue
void initializeQueue(struct Queue *q) {
    q->front = -1;
    q->rear = -1;
}

// Function to check if the queue is empty
int isEmpty(struct Queue *q) {
    return (q->front == -1);
}

// Function to check if the queue is full
int isFull(struct Queue *q) {
    return (q->rear == MAX_SIZE - 1);
}

// Function to insert an element into the queue
void enqueue(struct Queue *q, int data) {
    if (isFull(q)) {
        printf("Queue is full\n");
        return;
    }
    if (isEmpty(q)) {
        q->front = 0;
    }
    q->rear++;
    q->queue[q->rear] = data;
    printf("Enqueued %d in queue\n", data);
}

// Function to remove an element from the queue
int dequeue(struct Queue *q) {
    if (isEmpty(q)) {
        printf("Queue is empty\n");
        return -1;
    }
    int data = q->queue[q->front];
    // If the queue is empty reset the pointers
    if (q->front == q->rear) {
        q->front = -1;
        q->rear = -1;
    } else {
        q->front++;
    }
    printf("Deleted element: %d\n", data);
    return data;
}

```

```
// Function to display the elements of the queue
void display(struct Queue *q) {
    if (isEmpty(q)) {
        printf("Queue is empty\n");
        return;
    }
    for (int i = q->front; i <= q->rear; i++) {
        printf("%d ", q->queue[i]);
    }
    printf("\n");
}

int main() {
    // Initialize a queue
    struct Queue q;
    initializeQueue(&q);

    enqueue(&q, 1);
    enqueue(&q, 2);
    enqueue(&q, 3);
    printf("Elements in the queue after enqueue operation: ");
    display(&q);

    dequeue(&q);
    printf("Elements in the queue after dequeue operation: ");
    display(&q);

    return 0;
}
```

## Output

```
Enqueued 1 in queue
Enqueued 2 in queue
Enqueued 3 in queue
Elements in the queue after enqueue operation: 1 2 3
Deleted element: 1
Elements in the queue after dequeue operation: 2 3
```

### 3.2.3 Representation of queue using Linked-list:

In linked-list representation, we can define a queue as follows:

```
// Node structure representing a single node in the Linked List
typedef struct Node {
    int data;
    struct Node* next;
} Node;
```

```

// Function to create a new node
Node* createNode(int new_data)
{
    Node* new_node = (Node*)malloc(sizeof(Node));
    new_node->data = new_data;
    new_node->next = NULL;
    return new_node;
}

// Structure to implement queue operations using a linked list
typedef struct Queue {

    // Pointer to the front and the rear of the linked list
    Node *front, *rear;
} Queue;

// Function to create a queue
Queue* createQueue()
{
    Queue* q = (Queue*)malloc(sizeof(Queue));
    q->front = q->rear = NULL;
    return q;
}

```

```

// C program to implement the queue data structure using
// linked list
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>

// Node structure representing a single node in the linked
// list
typedef struct Node {
    int data;
    struct Node* next;
} Node;

// Function to create a new node
Node* createNode(int new_data)
{
    Node* new_node = (Node*)malloc(sizeof(Node));
    new_node->data = new_data;
    new_node->next = NULL;
    return new_node;
}

// Structure to implement queue operations using a linked
// list
typedef struct Queue {

    // Pointer to the front and the rear of the linked list

```



```

    Node *front, *rear;
} Queue;

// Function to create a queue
Queue* createQueue()
{
    Queue* q = (Queue*)malloc(sizeof(Queue));
    q->front = q->rear = NULL;
    return q;
}

// Function to check if the queue is empty
int isEmpty(Queue* q)
{
    // If the front and rear are null, then the queue is
    // empty, otherwise it's not
    if (q->front == NULL && q->rear == NULL) {
        return 1;
    }
    return 0;
}

// Function to add an element to the queue
void enqueue(Queue* q, int new_data)
{
    // Create a new Linked List node
    Node* new_node = createNode(new_data);

    // If queue is empty, the new node is both the front
    // and rear
    if (q->rear == NULL) {
        q->front = q->rear = new_node;
        return;
    }

    // Add the new node at the end of the queue and
    // change rear
    q->rear->next = new_node;
    q->rear = new_node;
}

// Function to remove an element from the queue
void dequeue(Queue* q)
{
    // If queue is empty, return
    if (isEmpty(q)) {
        printf("Queue Underflow\n");
        return;
    }
}

```

```

// Store previous front and move front one node
// ahead
Node* temp = q->front;
q->front = q->front->next;

// If front becomes null, then change rear also
// to null
if (q->front == NULL)
    q->rear = NULL;

// Deallocate memory of the old front node
free(temp);
}

// Function to get the front element of the queue
int getFront(Queue* q)
{
    // Checking if the queue is empty
    if (isEmpty(q)) {
        printf("Queue is empty\n");
        return INT_MIN;
    }
    return q->front->data;
}

// Function to get the rear element of the queue
int getRear(Queue* q)
{
    // Checking if the queue is empty
    if (isEmpty(q)) {
        printf("Queue is empty\n");
        return INT_MIN;
    }
    return q->rear->data;
}

// Driver code
int main()
{
    Queue* q = createQueue();

    // Enqueue elements into the queue
    enqueue(q, 10);
    enqueue(q, 20);

    printf("Queue Front: %d\n", getFront(q));
    printf("Queue Rear: %d\n", getRear(q));

    // Dequeue elements from the queue
    dequeue(q);
    dequeue(q);
}

```

```

// Enqueue more elements into the queue
enqueue(q, 30);
enqueue(q, 40);
enqueue(q, 50);

// Dequeue an element from the queue
dequeue(q);

printf("Queue Front: %d\n", getFront(q));
printf("Queue Rear: %d\n", getRear(q));

return 0;
}

```

## Output

```

Queue Front: 10
Queue Rear: 20
Queue Front: 40
Queue Rear: 50

```

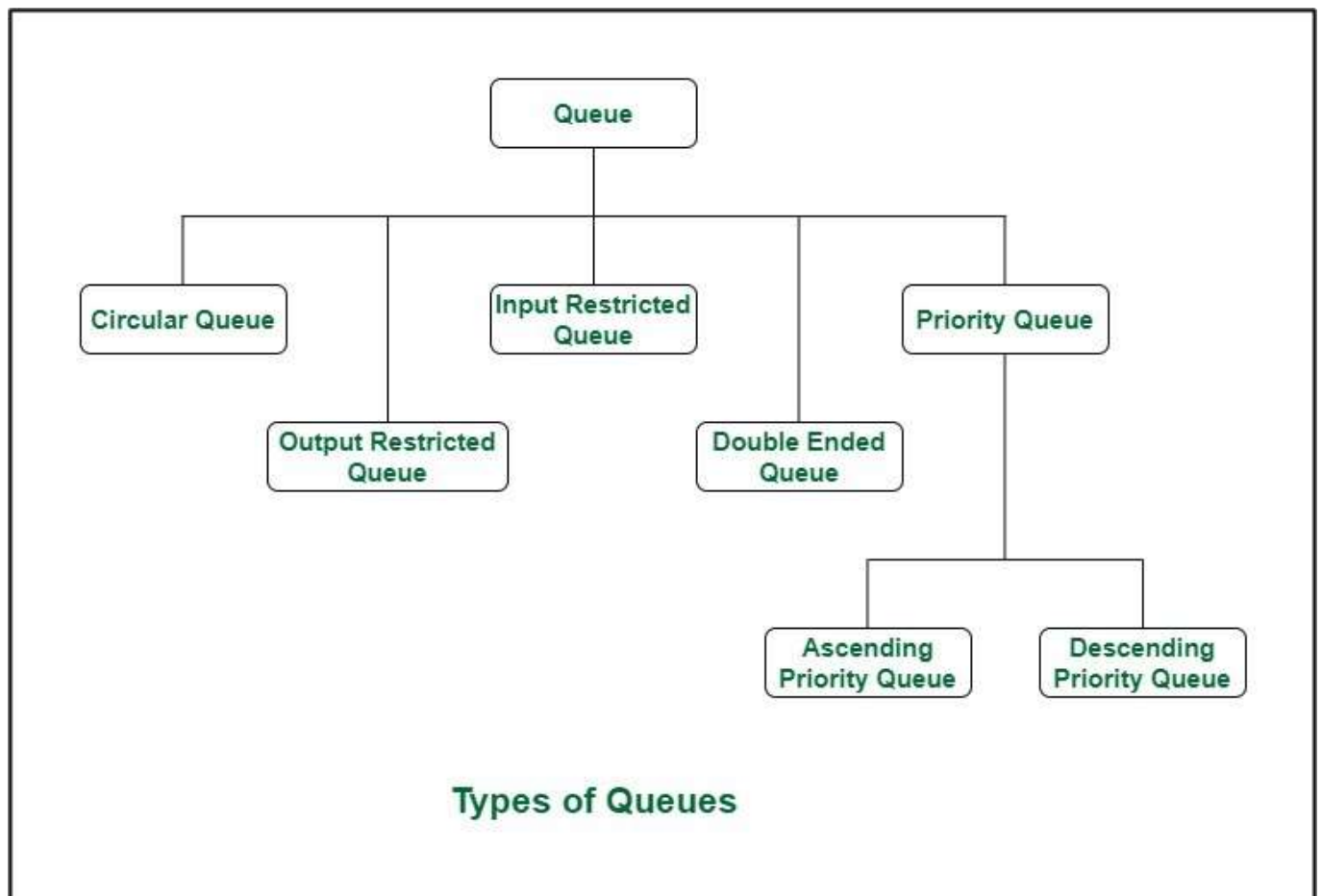
### 3.2.4 Applications of queue:

1. Task scheduling in operating system
2. Data transfer in network communication
3. Simulation of real-world system
4. Priority queues for event processing

### 3.2.4 Types of Queue:

There are **five different types of queues** that are used in different scenarios. They are:

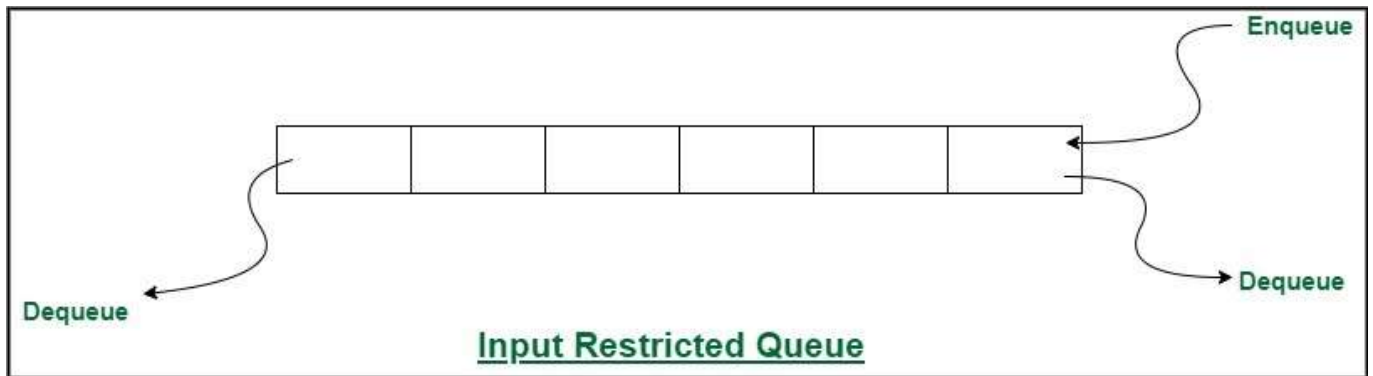
1. Input Restricted Queue (this is a Simple Queue)
2. Output Restricted Queue (this is also a Simple Queue)
3. Circular Queue
4. Double Ended Queue (Deque)
5. Priority Queue
  - Ascending Priority Queue
  - Descending Priority Queue



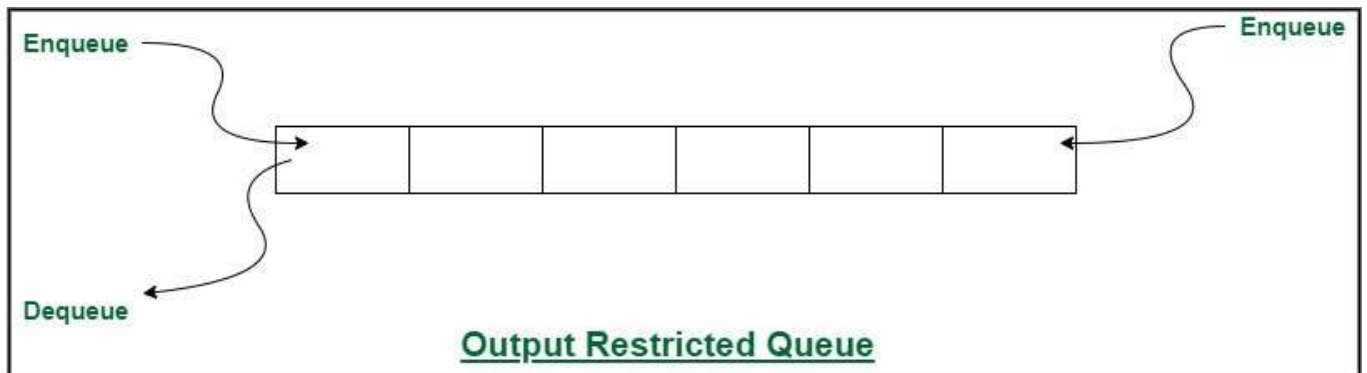
**1. Circular Queue:** Circular Queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle and the last position is connected back to the first position to make a circle. It is also called '**Ring Buffer**'. This queue is primarily used in the following cases:

- ✓ **Memory Management:** The unused memory locations in the case of ordinary queues can be utilized in circular queues.
- ✓ **Traffic system:** In a computer-controlled traffic system, circular queues are used to switch on the traffic lights one by one repeatedly as per the time set.
- ✓ **CPU Scheduling:** Operating systems often maintain a queue of processes that are ready to execute or that are waiting for a particular event to occur.

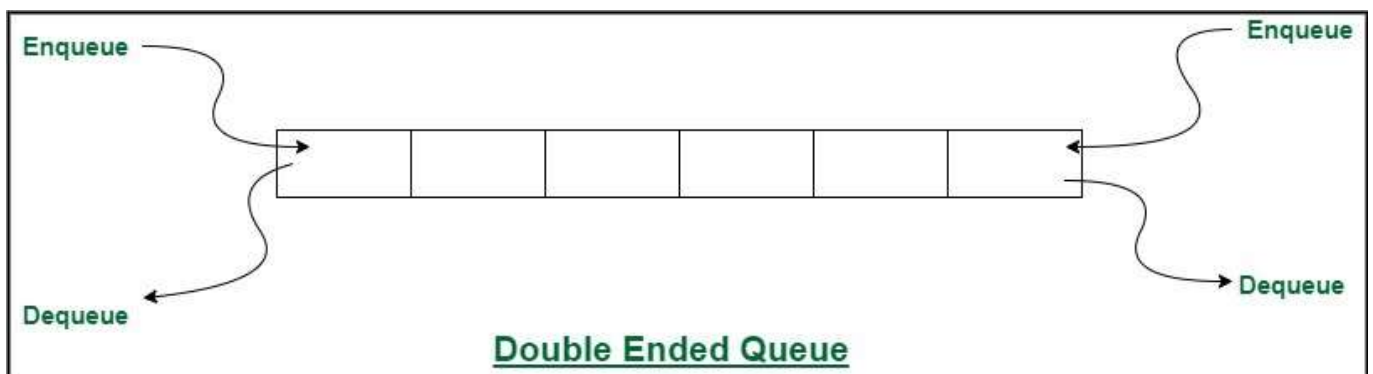
**2. Input restricted Queue:** In this type of Queue, the input can be taken from one side only(rear) and deletion of elements can be done from both sides(front and rear). This kind of Queue does not follow FIFO(first in first out). This queue is used in cases where the consumption of the data needs to be in FIFO order but if there is a need to remove the recently inserted data for some reason and one such case can be irrelevant data, performance issue, etc.



- 3. Output restricted Queue:** In this type of Queue, the input can be taken from both sides(rear and front) and the deletion of the element can be done from only one side(front). This queue is used in the case where the inputs have some priority order to be executed and the input can be placed even in the first place so that it is executed first.



- 4. Double ended Queue:** Double Ended Queue is also a Queue data structure in which the insertion and deletion operations are performed at both the ends (front and rear). That means, we can insert at both front and rear positions and can delete from both front and rear positions. Since Deque supports both stack and queue operations, it can be used as both. The Deque data structure supports clockwise and anticlockwise rotations in  $O(1)$  time which can be useful in certain applications. Also, the problems where elements need to be removed and or added both ends can be efficiently solved using Deque.



**5. Priority Queue:** A priority queue is a special type of queue in which each element is associated with a priority and is served according to its priority. There are two types of Priority Queues. They are:

- ✓ **Ascending Priority Queue:** Element can be inserted arbitrarily but only smallest element can be removed. For example, suppose there is an array having elements 4, 2, 8 in the same order. So, while inserting the elements, the insertion will be in the same sequence but while deleting, the order will be 2, 4, 8.
- ✓ **Descending priority Queue:** Element can be inserted arbitrarily but only the largest element can be removed first from the given Queue. For example, suppose there is an array having elements 4, 2, 8 in the same order. So, while inserting the elements, the insertion will be in the same sequence but while deleting, the order will be 8, 4, 2.

### 3.5 Conversion of infix arithmetic expression:

An arithmetic expression can express in three different ways, Infix, Postfix, and Prefix. Postfix and Prefix expressions are faster to execute for the compiler than simple Infix expression, as the compiler doesn't have to care about operator precedence in case of Postfix and Prefix.

The three components of an arithmetic expression are

- ✓ **Operation:** any expression of algebraic format
- ✓ **Operands:** The constant symbols (A, B. etc.) used for a value or the numbers (5, 7, 34, etc.)
- ✓ **Operators:** the arithmetic operators such as +, -, \*, %, / etc.

*Note that the parentheses are only used in infix expressions.*

#### Infix Notation

Infix is the day to day notation that we use of format  $A + B$  type. The general form can be classified as **(a op b)** where **a** and **b** are operands(variables) and **op** is Operator.

- Example 1 :

#### Postfix Notation

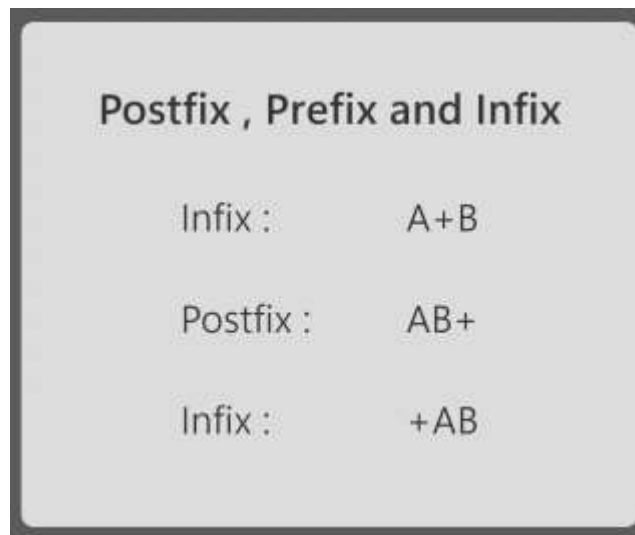
Postfix is notation that compiler uses/converts to while reading left to right and is of format **AB+** type. The general form can be classified as **(ab op)** where **a** and **b** are operands(variables) and **op** is Operator.

- Example 1 : AB+
- Example 2 : AB\*CD/+

#### Prefix Notation

Prefix is notation that compiler uses/converts to while reading right to left (some compilers can also read prefix left to right) and is of format **+AB** type. The general form can be classified as **(op ab)** where **a** and **b** are operands(variables) and **op** is Operator.

- Example 1 : +AB
- Example 2 : +\*AB/CD



### Why Postfix/Prefix Expressions are faster than Infix?

For Infix Expression which is format  $A+B*C$ , if the compiler is reading left to right then it can't evaluate  $A+B$  first until it has read whole expression and knows expression is actually  $A + (B*C)$  i.e.  $B * C$  needs to be implemented first

Postfix for above infix is  $ABC*+$ . Now, as soon as compiler sees two operands followed by operator it can implement it without caring for precedence.

- Assume  $ABC*+$
- $ABC*+$  ( $BC*$  is implemented as  $B*C$  and result is put back)
- $AX+$  (Assuming  $X$  stores result of  $BC*$  i.e.  $B*C$ )
- Now finally  $AX+$  can be implemented as  $A+X$

*Note: Compiler converts our Infix Expression into postfix or Prefix as expressions are operated as stacks and postfix and prefix are faster to implement as compiler doesn't need to care about precedence at all.*

Compiler converts infix expression to postfix/prefix at compile time, so at runtime your calculations are always happening in post-prefix. A website's code maybe compiled once a week but it may need to run 1 million times a week.

### 3.5.1 Conversion of Infix Expression into Postfix Expression:

- **Infix:**  $a + b * c + d$  can be written as  $a + (b * c) + d$
- Now, for all  $+ - / *$  [associativity](#) is left to right we will write it as
- $(a + (b * c)) + d$  and thus further  $((a + (b * c)) + d)$
- Solving and converting innermost bracket to postfix

Step 1:  $((a + bc*) + d)$

Step 2: Consider  $bc*$  as separate operand  $x$  the innermost bracket now looks like  $((a + x) + d)$

- ✓ Applying postfix, it looks like  $-(ax+ + d)$ , replacing  $x$  here  $(abc*+ + d)$

Step 3: Considering  $abc*+as$  separate operand  $z$ , the final bracket looks like  $-(z + d)$  the result would be  $zd+$

✓ Replacing  $z$  value =  $abc*+d+$

### Algorithm:

1. First Start scanning the expression from left to right
2. If the scanned character is an operand, output it, i.e. print it
3. Else
  - ✓ If the precedence of the scanned operator is higher than the precedence of the operator in the stack (or stack is empty or has '('), then push operator in the stack
  - ✓ Else, Pop all the operators, that have greater or equal precedence than the scanned operator. Once you pop them push this scanned operator. (If we see a parenthesis while popping then stop and push scanned operator in the stack)
4. If the scanned character is an '(', push it to the stack.
5. If the scanned character is an ')', pop the stack and output it until an '(' is encountered, and discard both the parenthesis.
6. Now, we should repeat the steps 2 – 6 until the whole infix i.e. whole characters are scanned.
7. Print output
8. Do the pop and output (print) until stack is not empty

| Infix to Postfix                              |       |              |          |
|-----------------------------------------------|-------|--------------|----------|
| Expression : $A + B * C / D - F + A \wedge E$ |       |              |          |
| Scanned Symbol                                | Stack | Output       | Reason   |
| A                                             |       | A            | Step 2   |
| +                                             | +     | A            | Step 3.1 |
| B                                             | +     | AB           | Step 2   |
| *                                             | +     | AB           | Step 3.1 |
| C                                             | +     | ABC          | Step 2   |
| /                                             | +/    | ABC*         | Step 3.2 |
| D                                             | +/    | ABC*D        | Step 2   |
| -                                             | -     | ABC*D/+      | Step 3.2 |
| F                                             | -     | ABC*D/+F     | Step 2   |
| +                                             | +     | ABC*D/+F+    | Step 3.2 |
| A                                             | +     | ABC*D/+F-A   | Step 2   |
| ^                                             | ^^    | ABC*D/+F-A   | Step 2   |
| E                                             | ^^    | ABC*D/+F-AE  | Step 2   |
| (Empty)                                       |       | ABC*D/+F-A+* | Step 8   |

### 3.5.2 conversion of Infix expression into Prefix Expression:

- **Infix:**  $(a / b + c) - (d + e * f)$  can be written as  $((a / b) + c) - (d + (e * f))$
- Now, we have done the above according to associativity
- Solving and converting innermost bracket to prefix



- Step 1:  $(/ab + c) - (d + *ef)$
- Step 2: Consider  $/ab$  and  $*ef$  as separate operand  $x$  and  $y$
- the innermost bracket now looks like  $(x + c) - (d + y)$ 
  - ✓ Applying prefix, it looks like  $-(+xc - +dy)$  replacing  $x$  and  $y$  here  $(+/abc - +d*ef)$
- Step 3: Considering  $+/abc$  and  $+d*ef$  as separate operand  $z$  and  $w$ , the final bracket looks like  $-(z - w)$  the result would be  $-zw$ 
  - ✓ Replacing  $z$  and  $w$  value =  $-+/abc+d*ef$

**Algorithm:**

**Step 1:** Reverse the infix string. Note that while reversing the string you must interchange left and right parentheses.

**Step 2:** Obtain the postfix expression of the expression obtained from Step 1.

**Step 3:** Reverse the postfix expression to get the prefix expression

**Example:** Given Infix:  $((a/b)+c)-(d+(e*f))$

1. **String after reversal:**  $)f*e(+d(-)c+)b/a(($
2. **String after interchanging right and left parenthesis:**  $((f*e)+d)-(c+(b/a))$
3. **Apply postfix Conversion**
4. **Reverse Postfix Expression (Given After the image below)**

| Sr. No. | Expression | Stack | Prefix  |
|---------|------------|-------|---------|
| 0       | (          | (     |         |
| 1       | (          | ((    |         |
| 2       | f          | ((    | f       |
| 3       | *          | ((*   | f       |
| 4       | e          | ((*   | fe      |
| 5       | )          | (     | fe*     |
| 6       | +          | (+    | fe*     |
| 7       | d          | (+    | fe*d    |
| 8       | )          |       | fe*d+   |
| 9       | -          | -     | fe*d+   |
| 10      | (          | -(    | fe*d+   |
| 11      | c          | -(    | fe*d+c  |
| 12      | +          | -(+   | fe*d+c  |
| 13      | (          | -(+(  | fe*d+c  |
| 14      | b          | -(+(  | fe*d+cb |
| 15      | /          | -(+(/ | fe*d+cb |

| Sr. No. | Expression | Stack | Prefix      |
|---------|------------|-------|-------------|
| 16      | a          | -(+(/ | fe*d+cba    |
| 17      | )          | -(+   | fe*d+cba/   |
| 18      | )          | -     | fe*d+cba/+  |
| 19      |            |       | fe*d+cba/+- |

Final prefix: -+/abc+d\*ef

### 3.6 Evaluation of postfix expression

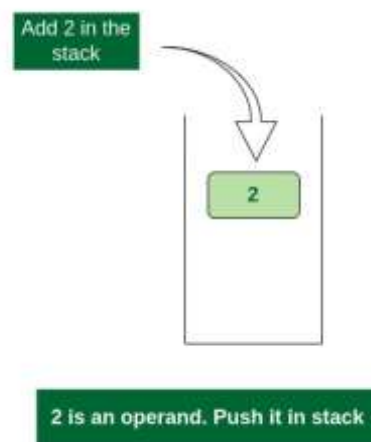
To evaluate a postfix expression, we can use a stack and it be done by a manner given below:

- Iterate the expression from left to Right and keep on storing the operands into the stack.
- Once an operator is received, pop the two topmost elements and evaluate them and push the result in the stack again.

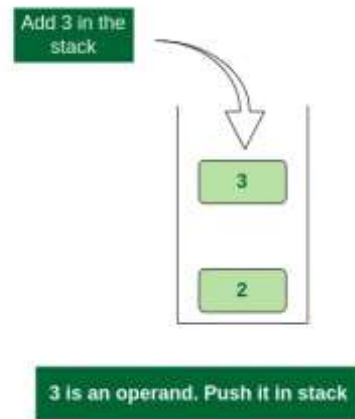
Let us consider an example to illustrate the process of evaluation.

Given Postfix Expression: "2 3 1 \* + 9 -"

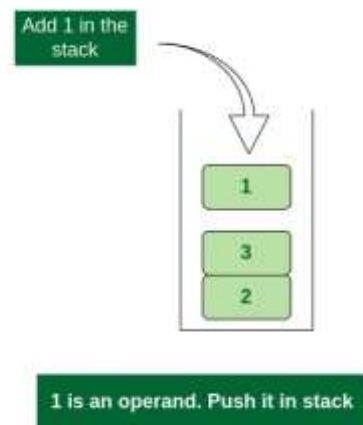
- ✓ Scan 2, it is a number (i.e. operand), so push it into stack. Now, stack contains '2'.



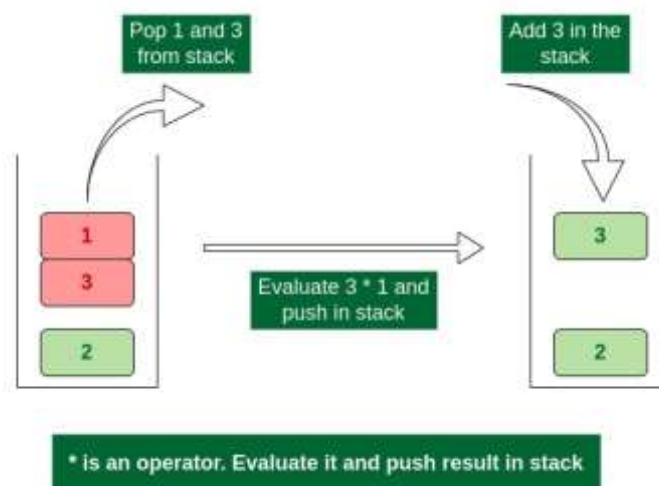
- ✓ Scan 3, again a number, push it into the stack, stack now contains '2 3'.



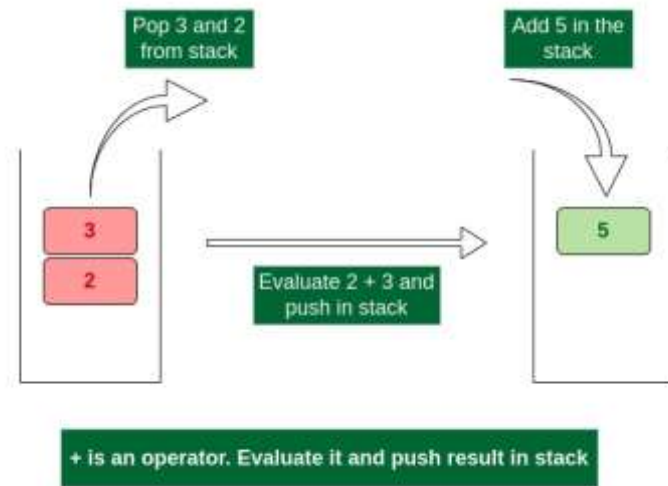
- ✓ Scan 1, again a number, push it to stack, stack now contains '2 3 1'.



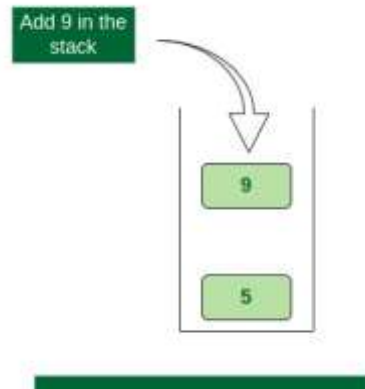
- ✓ Scan \*, it is an operator. Pop two operands from the stack, apply \* operator on the operands. We get  $3 * 1$  which result in 3. We push the result 3 to stack. The stack now contains '2 3'.



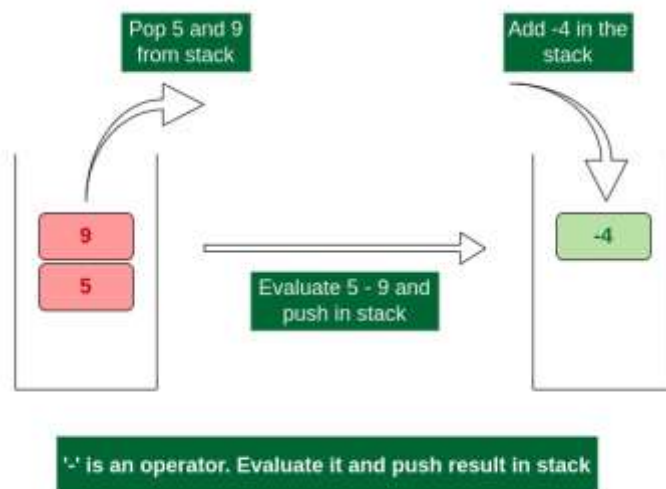
- ✓ Scan +, it is an operator. Pop two operands from stack, apply the + operator on operands. We get  $2 + 3$  which results in 5. We push the result 5 to stack. The stack now contains '5'.



- ✓ Scan 9, it is a number. So, we push it to stack. Now the stack contains '5 9'.



- ✓ Scan -, it is an operator, pop two operands from stack, apply the - operator on operands, we get 5 - 9 which results in -4. We push the result -4 to the stack. The stack now becomes '-4'.



- ✓ There are no more elements to scan, we return the top element from the stack which is the only element left in the stack.

So, the result becomes -4.

**Algorithm:**

1. Create a stack to store operands (or values).
2. Scan the given expression from left to right and do the following for every scanned element.
  - a. If the element is a number, push it into the stack.
  - b. If the element is an operator, pop two operands for the operator from the stack. Evaluate the operator and push the result back to the stack.
3. When the expression is ended, the number in the stack is the final answer.

```
// C program to evaluate value of a postfix expression
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Stack type
struct Stack {
    int top;
    unsigned capacity;
    int* array;
};

// Stack Operations
struct Stack* createStack(unsigned capacity)
{
    struct Stack* stack
        = (struct Stack*)malloc(sizeof(struct Stack));

    if (!stack)
        return NULL;

    stack->top = -1;
    stack->capacity = capacity;
    stack->array
        = (int*)malloc(stack->capacity * sizeof(int));

    if (!stack->array)
        return NULL;

    return stack;
}

int isEmpty(struct Stack* stack)
{
    return stack->top == -1;
}

char peek(struct Stack* stack)
{
    return stack->array[stack->top];
}
```

```

char pop(struct Stack* stack)
{
    if (!isEmpty(stack))
        return stack->array[stack->top--];
    return '$';
}

void push(struct Stack* stack, char op)
{
    stack->array[++stack->top] = op;
}

// The main function that returns value
// of a given postfix expression
int evaluatePostfix(char* exp)
{
    // Create a stack of capacity equal to expression size
    struct Stack* stack = createStack(strlen(exp));
    int i;

    // See if stack was created successfully
    if (!stack)
        return -1;

    // Scan all characters one by one
    for (i = 0; exp[i]; ++i) {

        // If the scanned character is an operand
        // (number here), push it to the stack.
        if (isdigit(exp[i]))
            push(stack, exp[i] - '0');

        // If the scanned character is an operator,
        // pop two elements from stack apply the operator
        else {
            int val1 = pop(stack);
            int val2 = pop(stack);
            switch (exp[i]) {
                case '+':
                    push(stack, val2 + val1);
                    break;
                case '-':
                    push(stack, val2 - val1);
                    break;
                case '*':
                    push(stack, val2 * val1);
                    break;
                case '/':
                    push(stack, val2 / val1);
                    break;
            }
        }
    }
    return pop(stack);
}

// Driver code
int main()
{
    char exp[] = "231*+9-";

```

```

// Function call
printf("postfix evaluation: %d", evaluatePostfix(exp));
return 0;
}

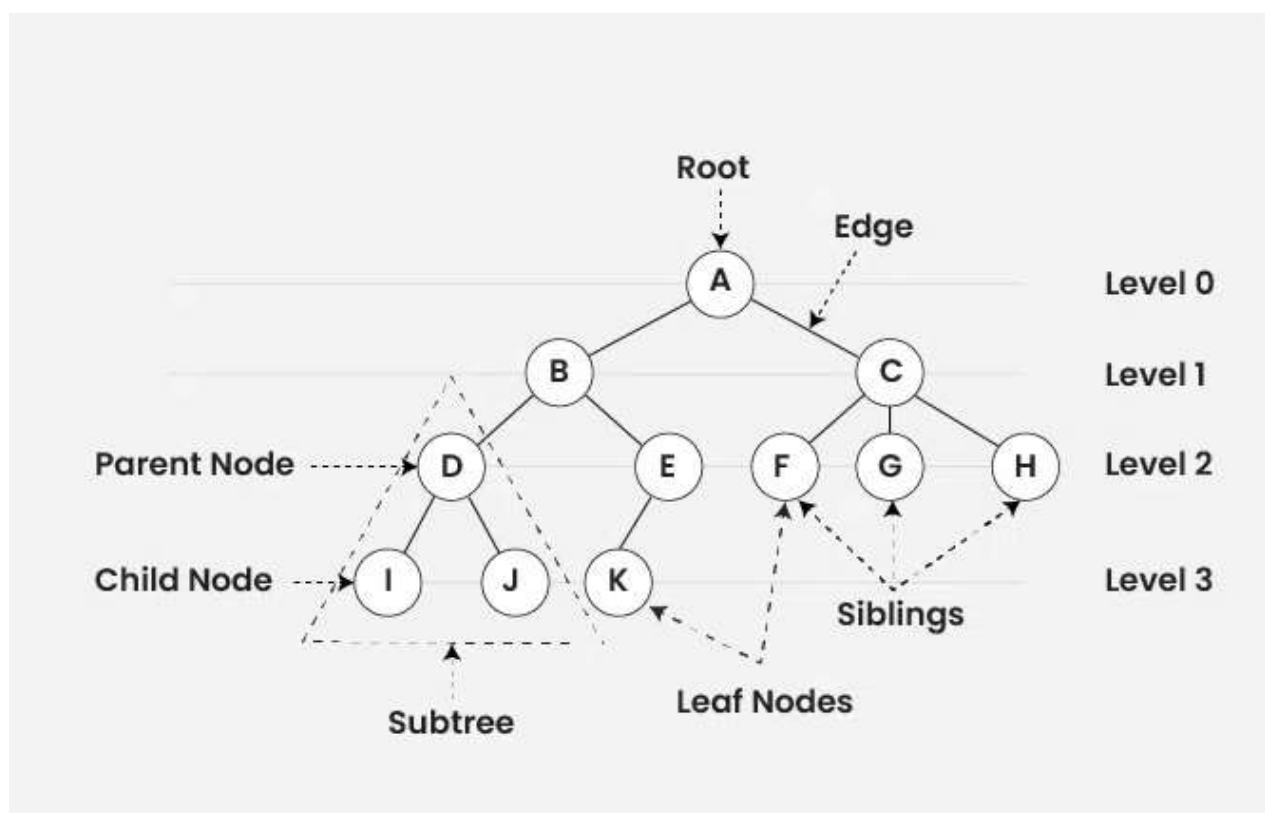
```

## Unit 4:

### Binary trees

#### 4.1 Definition of Trees

A tree is defined as a set of one or more nodes where one node is designated as a root of the tree and all other remaining nodes can be partitioned into non-empty sets each of which is a sub-set of the root. In the Figure ... shows a tree where node A is the root; nodes B, C, D are children of the root node and form subtree rooted at node A.



#### 4.1.1 Basic Terminology

- **Parent Node:** The node which is a predecessor of a node is called the parent node of that node. {B} is the parent node of {D, E}.
- **Child Node:** The node which is the immediate successor of a node is called the child node of that node. Examples: {D, E} are the child nodes of {B}.
- **Root Node:** The topmost node of a tree or the node which does not have any parent node is called the root node. {A} is the root node of the tree. A non-empty tree must contain exactly one root node and exactly one path from the root to all other nodes of the tree.
- **Leaf Node or External Node:** The nodes which do not have any child nodes are called leaf nodes. {I, J, K, F, G, H} are the leaf nodes of the tree.
- **Ancestor of a Node:** Any predecessor nodes on the path of the root to that node are called Ancestors of that node. {A,B} are the ancestor nodes of the node {E}
- **Descendant:** A node x is a descendant of another node y if and only if y is an ancestor of x.
- **Sibling:** Children of the same parent node are called siblings. {D,E} are called siblings.
- **Level of a node:** The count of edges on the path from the root node to that node. The root node has level 0.
- **Internal node:** A node with at least one child is called Internal Node.
- **Neighbour of a Node:** Parent or child nodes of that node are called neighbors of that node.
- **Subtree:** Any node of the tree along with its descendant.
- **Depth of node x:** The depth of node x can be defined as the length of the path from the root to the node x. One edge contributes one-unit length in the path. So, the depth of node x can also be defined as the number of edges between the root node and the node x. The root node has 0 depth.
- **Height of node x:** The height of node x can be defined as the longest path from the node x to the leaf node.
- **Height of the Tree:** The height of a tree is the length of the longest path from the root of the tree to a leaf node of the tree.
- **Degree of a Node:** The total count of subtrees attached to that node is called the degree of the node. The degree of a leaf node must be 0. The degree of a tree is the maximum degree of a node among all the nodes in the tree.

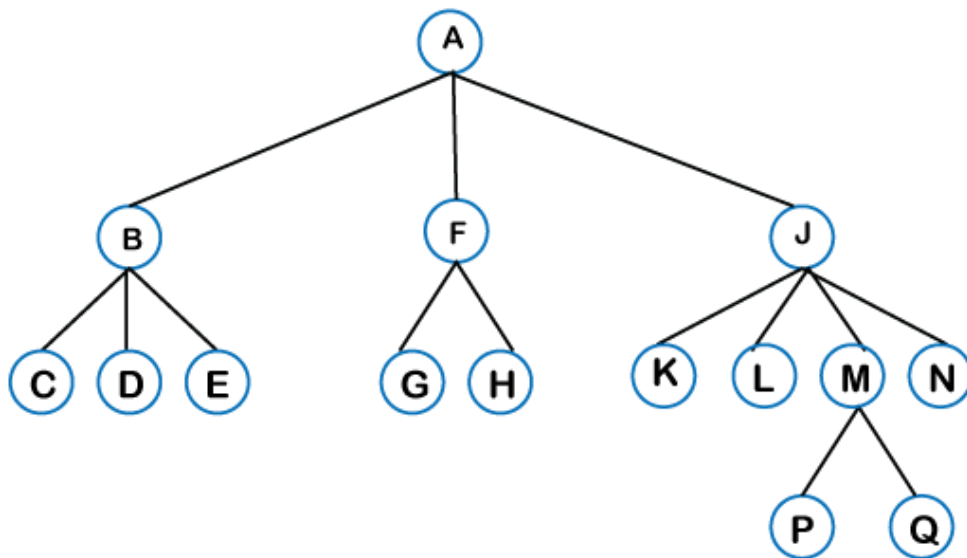
## 4.2 Types of tree

There are different types of tree data structure, such as:

### a. General Tree:

**General tree:** The general tree is one of the types of tree data structure. In the general tree, a node can have either 0 or maximum n number of nodes. There is no restriction imposed on the degree of the node (the number of nodes that a node can contain). The topmost node in a general tree is known as a root node. The children of the parent node are known as *subtrees*.





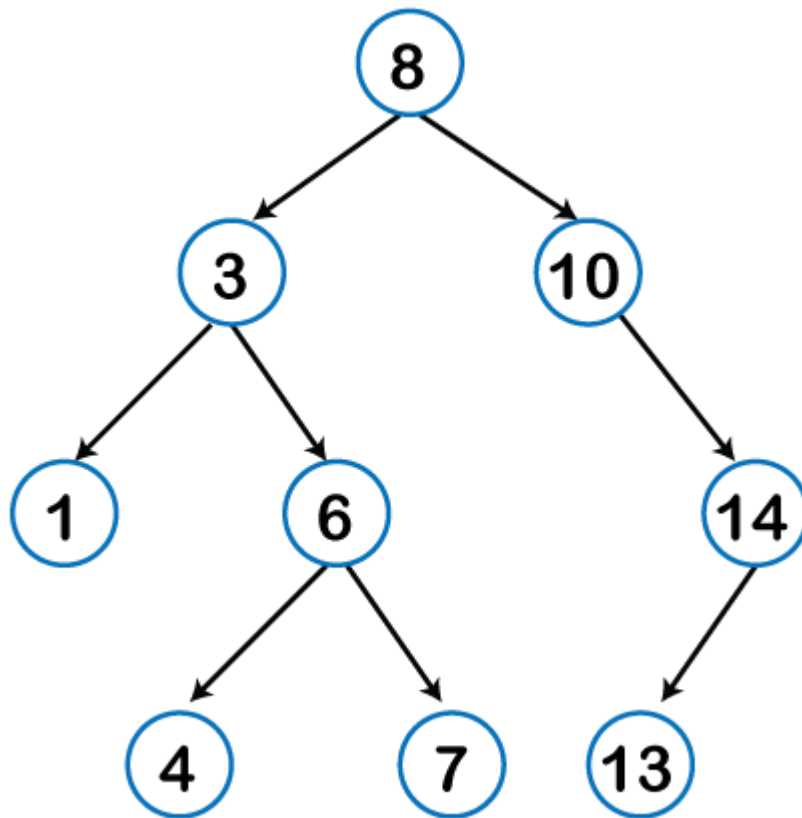
There can be  $n$  number of subtrees in a general tree. In the general tree, the subtrees are unordered as the nodes in the subtree cannot be ordered.

Every non-empty tree has a downward edge, and these edges are connected to the nodes known as *child nodes*. The root node is labeled with level 0. The nodes that have the same parent are known as *siblings*.

b. Forest:

c. Binary Tree:

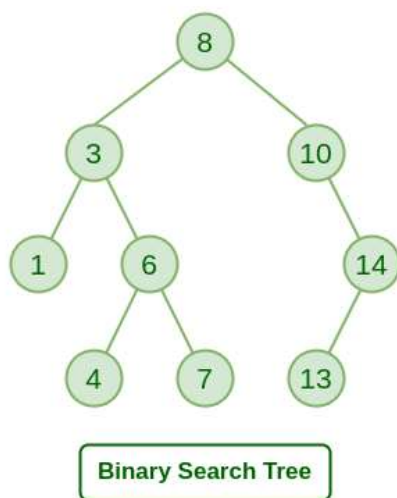
Here, binary name itself suggests two numbers, i.e., 0 and 1. In a binary tree, each node in a tree can have utmost two child nodes. Here, utmost means whether the node has 0 nodes, 1 node or 2 nodes.



d. Binary Search Tree:

**Binary Search Tree** is a node-based binary tree data structure which has the following properties:

1. The left subtree of a node contains only nodes with keys lesser than the node's key.
2. The right subtree of a node contains only nodes with keys greater than the node's key.
3. The left and right subtree each must also be a binary search tree.



e. AVL Tree:

It is one of the types of the binary tree, or we can say that it is a variant of the binary search tree. AVL tree satisfies the property of the *binary tree* as well as of the *binary search tree*. It is a self-balancing binary search tree that was invented by *Adelson Velsky Lindas*. Here, self-balancing means that balancing the heights of left subtree and right subtree. This balancing is measured in terms of the *balancing factor*. We can consider a tree as an AVL tree if the tree obeys the binary search tree as well as a balancing factor. The balancing factor can be defined as the *difference between the height of the left subtree and the height of the right subtree*. The balancing factor's value must be either 0, -1, or 1; therefore, each node in the AVL tree should have the value of the balancing factor either as 0, -1, or 1.

f. Red-Black Tree:

**The red-Black tree** is the binary search tree. The prerequisite of the Red-Black tree is that we should know about the binary search tree. In a binary search tree, the value of the left-subtree should be less than the value of that node, and the value of the right-subtree should be greater than the value of that node. As we know that the time complexity of binary search in the average case is  $\log_2 n$ , the best case is  $O(1)$ , and the worst case is  $O(n)$ .

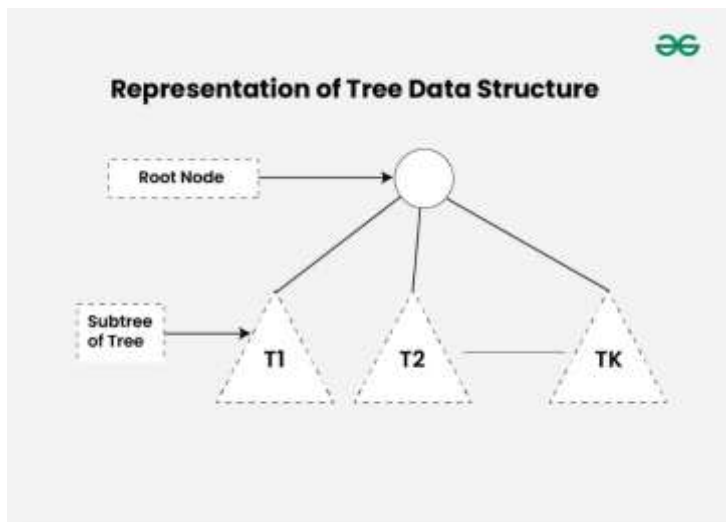
g. B-Tree

h. Splay Tree:

The splay tree data structure is also binary search tree in which recently accessed element is placed at the root position of tree by performing some rotation operations. Here, *splaying* means the recently accessed node. It is a *self-balancing* binary search tree having no explicit balance condition like **AVL** tree.

#### 4.3 Representation of Trees

A tree consists of a root node, and zero or more subtrees  $T_1, T_2, \dots, T_k$  such that there is an edge from the root node of the tree to the root node of each subtree. Subtree of a node  $X$  consists of all the nodes which have node  $X$  as the ancestor node.



4.4 Binary tree

4.5 Traversal Methods

4.6 Binary Search Tree

4.7 Operations on BST

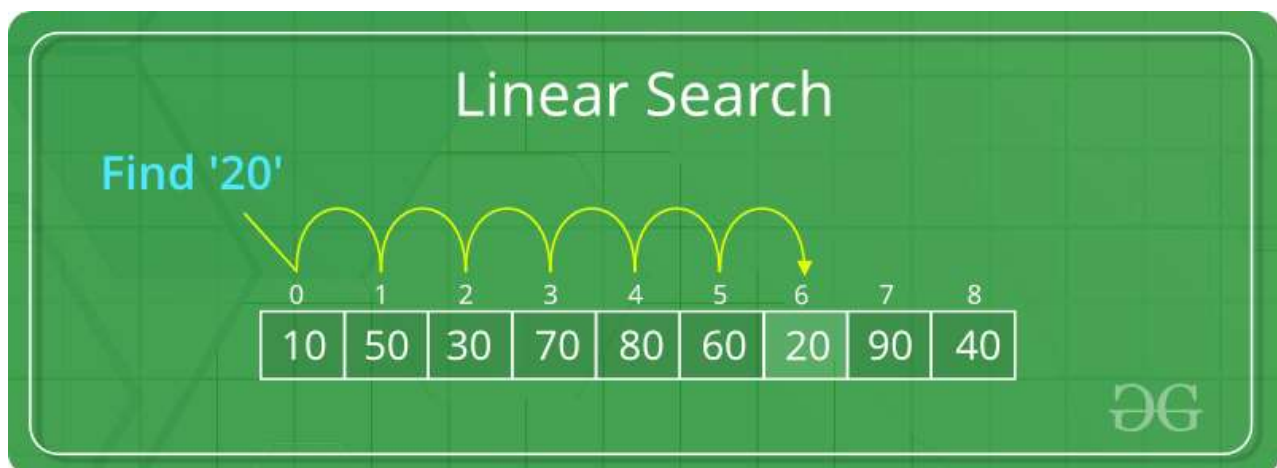
4.8 Threaded Binary Tree

4.9 Heap

Unit 5;

Searching and Sorting

**Linear Search** is defined as a sequential [search algorithm](#) that starts at one end and goes through each element of a list until the desired element is found, otherwise the search continues till the end of the data set.



## Linear Search Algorithm

### How Does Linear Search Algorithm Work?

In Linear Search Algorithm,

- Every element is considered as a potential match for the key and checked for the same.
- If any element is found equal to the key, the search is successful and the index of that element is returned.
- If no element is found equal to the key, the search yields “No match found”.

**For example:** Consider the array `arr[] = {10, 50, 30, 70, 80, 20, 90, 40}` and `key = 30`

**Step 1:** Start from the first element (index 0) and compare `key` with each element (`arr[i]`).

- Comparing key with first element `arr[0]`. Since not equal, the iterator moves to the next element as a potential match.

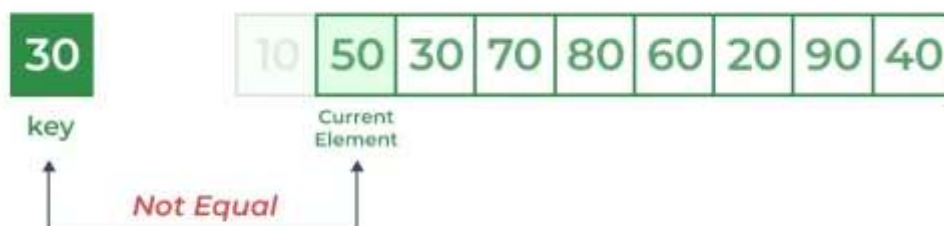


Linear Search Algorithm



Compare key with `arr[0]`

- Comparing key with next element `arr[1]`. Since not equal, the iterator moves to the next element as a potential match.

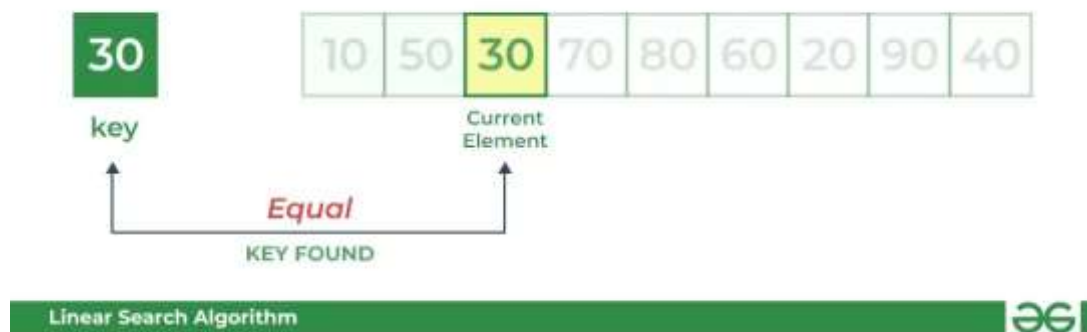


Linear Search Algorithm



Compare key with `arr[1]`

**Step 2:** Now when comparing arr[2] with key, the value matches. So the Linear Search Algorithm will yield a successful message and return the index of the element when key is found (here 2).



Compare key with arr[2]

Implementation of Linear Search Algorithm:

Below is the implementation of the linear search algorithm:

// C code to linearly search x in arr[].

```
#include <stdio.h>
```

```
int search(int arr[], int N, int x)
```

```
{  
    for (int i = 0; i < N; i++)  
        if (arr[i] == x)  
            return i;  
    return -1;  
}
```

```
// Driver code
```

```
int main(void)
```

```
{  
    int arr[] = { 2, 3, 4, 10, 40 };  
    int x = 10;
```

```

int N = sizeof(arr) / sizeof(arr[0]);

// Function call
int result = search(arr, N, x);

(result == -1)
    ? printf("Element is not present in array")
    : printf("Element is present at index %d", result);

return 0;
}

```

## Output

Element is present at index 3

## Complexity Analysis of Linear Search:

### Time Complexity:

- **Best Case:** In the best case, the key might be present at the first index. So the best case complexity is  $O(1)$
- **Worst Case:** In the worst case, the key might be present at the last index i.e., opposite to the end from which the search has started in the list. So the worst-case complexity is  $O(N)$  where  $N$  is the size of the list.
- **Average Case:**  $O(N)$

**Auxiliary Space:**  $O(1)$  as except for the variable to iterate through the list, no other variable is used.

## Advantages of Linear Search:

- Linear search can be used irrespective of whether the array is sorted or not. It can be used on arrays of any data type.
- Does not require any additional memory.
- It is a well-suited algorithm for small datasets.

## Drawbacks of Linear Search:

- Linear search has a time complexity of  $O(N)$ , which in turn makes it slow for large datasets.
- Not suitable for large arrays.

## When to use Linear Search?

- When we are dealing with a small dataset.
- When you are searching for a dataset stored in contiguous memory.

## Binary Search

**Binary Search** is defined as a [searching algorithm](#) used in a sorted array by **repeatedly dividing the search interval in half**. The idea of binary search is to use the information that the array is sorted and reduce the time complexity to  $O(\log N)$ .

| Binary Search                        |     |   |   |    |     |          |     |     |    |     |
|--------------------------------------|-----|---|---|----|-----|----------|-----|-----|----|-----|
|                                      | 0   | 1 | 2 | 3  | 4   | 5        | 6   | 7   | 8  | 9   |
| Search 23                            | 2   | 5 | 8 | 12 | 16  | 23       | 38  | 56  | 72 | 91  |
|                                      | L=0 |   |   |    | M=4 |          |     |     |    | H=9 |
| 23 > 16<br>take 2 <sup>nd</sup> half | 2   | 5 | 8 | 12 | 16  | 23       | 38  | 56  | 72 | 91  |
|                                      | 0   | 1 | 2 | 3  | 4   | L=5      | 6   | M=7 | 8  | H=9 |
| 23 < 56<br>take 1 <sup>st</sup> half | 2   | 5 | 8 | 12 | 16  | 23       | 38  | 56  | 72 | 91  |
|                                      | 0   | 1 | 2 | 3  | 4   | L=5, M=5 | H=6 | 7   | 8  | 9   |
| Found 23,<br>Return 5                | 2   | 5 | 8 | 12 | 16  | 23       | 38  | 56  | 72 | 91  |

Example of Binary Search Algorithm

Conditions for when to apply Binary Search in a Data Structure:

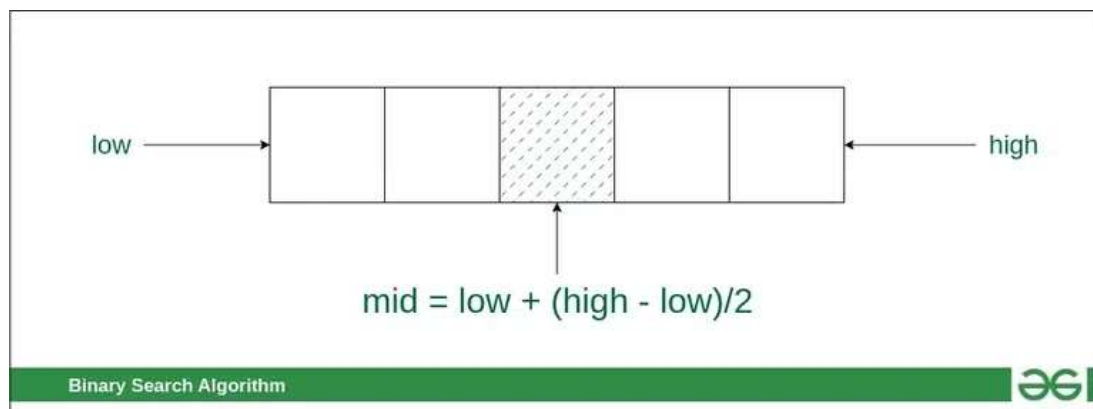
To apply Binary Search algorithm:

- The data structure must be sorted.
- Access to any element of the data structure takes constant time.

Binary Search Algorithm:

In this algorithm,

- Divide the search space into two halves by [finding the middle index “mid”](#).





## Finding the middle index “mid” in Binary Search Algorithm

- Compare the middle element of the search space with the key.
- If the key is found at middle element, the process is terminated.
- If the key is not found at middle element, choose which half will be used as the next search space.
  - If the key is smaller than the middle element, then the left side is used for next search.
  - If the key is larger than the middle element, then the right side is used for next search.
- This process is continued until the key is found or the total search space is exhausted.

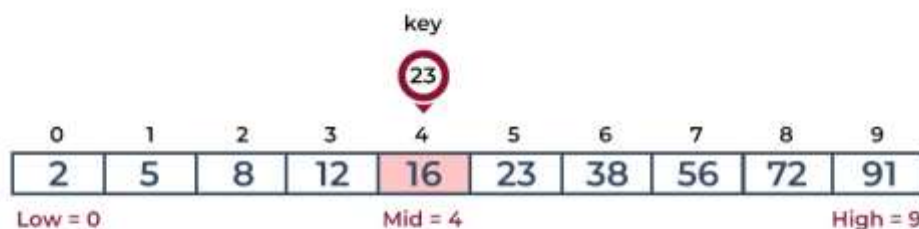
## How does Binary Search work?

To understand the working of binary search, consider the following illustration:

Consider an array `arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91}`, and the **target = 23**.

**First Step:** Calculate the mid and compare the mid element with the key. If the key is less than mid element, move to left and if it is greater than the mid then move search space to the right.

- Key (i.e., 23) is greater than current mid element (i.e., 16). The search space moves to the right.

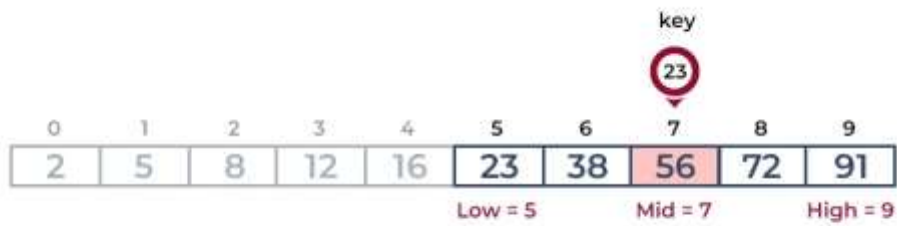


Binary search



Binary Search Algorithm : Compare key with 16

- Key is less than the current mid 56. The search space moves to the left.



Binary search



Binary Search Algorithm : Compare key with 56

**Second Step:** If the key matches the value of the mid element, the element is found and stop search.



Binary search



Binary Search Algorithm : Key matches with mid

### How to Implement Binary Search?

The **Binary Search Algorithm** can be implemented in the following two ways

- Iterative Binary Search Algorithm
- Recursive Binary Search Algorithm

Given below are the pseudocodes for the approaches.

#### 1. Iterative Binary Search Algorithm:

Here we use a while loop to continue the process of comparing the key and splitting the search space in two halves.

Implementation of Iterative Binary Search Algorithm:.

// C++ program to implement iterative Binary Search

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// An iterative binary search function.
int binarySearch(int arr[], int l, int r, int x)
{
    while (l <= r) {
        int m = l + (r - l) / 2;

        // Check if x is present at mid
        if (arr[m] == x)
            return m;

        // If x greater, ignore left half
        if (arr[m] < x)
            l = m + 1;

        // If x is smaller, ignore right half
        else
            r = m - 1;
    }

    // If we reach here, then element was not present
    return -1;
}

// Driver code
int main(void)
{
    int arr[] = { 2, 3, 4, 10, 40 };
    int x = 10;
    int n = sizeof(arr) / sizeof(arr[0]);
    int result = binarySearch(arr, 0, n - 1, x);
```

```

(result == -1)
    ? cout << "Element is not present in array"
    : cout << "Element is present at index " << result;

return 0;
}

```

## Output

Element is present at index 3

**Time Complexity:**  $O(\log N)$

**Auxiliary Space:**  $O(1)$

## 2. Recursive Binary Search Algorithm:

Create a recursive function and compare the mid of the search space with the key. And based on the result either return the index where the key is found or call the recursive function for the next search space.

Implementation of Recursive Binary Search Algorithm:

// C++ program to implement recursive Binary Search

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// A recursive binary search function. It returns
```

```
// location of x in given array arr[l..r] is present,
```

```
// otherwise -1
```

```
int binarySearch(int arr[], int l, int r, int x)
```

```
{
```

```
    if (r >= l) {
```

```
        int mid = l + (r - l) / 2;
```

```
        // If the element is present at the middle
```

```
        // itself
```

```
        if (arr[mid] == x)
```

```
            return mid;
```

```

// If element is smaller than mid, then
// it can only be present in left subarray
if (arr[mid] > x)
    return binarySearch(arr, l, mid - 1, x);

// Else the element can only be present
// in right subarray
return binarySearch(arr, mid + 1, r, x);
}

// We reach here when element is not
// present in array
return -1;
}

// Driver code
int main()
{
    int arr[] = { 2, 3, 4, 10, 40 };
    int x = 10;
    int n = sizeof(arr) / sizeof(arr[0]);
    int result = binarySearch(arr, 0, n - 1, x);
    (result == -1)
        ? cout << "Element is not present in array"
        : cout << "Element is present at index " << result;
    return 0;
}

```

## Output

Element is present at index 3

## Complexity Analysis of Binary Search:

- **Time Complexity:**

- Best Case:  $O(1)$
  - Average Case:  $O(\log N)$
  - Worst Case:  $O(\log N)$
- **Auxiliary Space:**  $O(1)$ , If the recursive call stack is considered then the auxiliary space will be  $O(\log N)$ .

#### Advantages of Binary Search:

- Binary search is faster than linear search, especially for large arrays.
- More efficient than other searching algorithms with a similar time complexity, such as interpolation search or exponential search.
- Binary search is well-suited for searching large datasets that are stored in external memory, such as on a hard drive or in the cloud.

#### Drawbacks of Binary Search:

- The array should be sorted.
- Binary search requires that the data structure being searched be stored in contiguous memory locations.
- Binary search requires that the elements of the array be comparable, meaning that they must be able to be ordered.

#### Applications of Binary Search:

- Binary search can be used as a building block for more complex algorithms used in machine learning, such as algorithms for training neural networks or finding the optimal hyperparameters for a model.
- It can be used for searching in computer graphics such as algorithms for ray tracing or texture mapping.
- It can be used for searching a database.

#### Comparison of Linear and Binary Search

| Linear Search                                                                                                 | Binary Search                                                        |
|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| In linear search input data need not to be in sorted. In binary search input data need to be in sorted order. |                                                                      |
| It is also called sequential search.                                                                          | It is also called half-interval search.                              |
| The time complexity of linear search <b><math>O(n)</math></b> .                                               | The time complexity of binary search <b><math>O(\log n)</math></b> . |
| Multidimensional array can be used.                                                                           | Only single dimensional array is used.                               |
| Linear search performs equality comparisons                                                                   | Binary search performs ordering comparisons                          |
| It is less complex.                                                                                           | It is more complex.                                                  |

It is very slow process.

It is very fast process.

#### 5.1 Linear Search

#### 5.2 Binary Search

#### 5.3 Indexed Search

#### 5.4 Hashing

**Hashing** refers to the process of generating a fixed-size output from an input of variable size using the mathematical formulas known as hash functions. This technique determines an index or location for the storage of an item in a data structure.

#### Need for Hash data structure

Every day, the data on the internet is increasing multifold and it is always a struggle to store this data efficiently. In day-to-day programming, this amount of data might not be that big, but still, it needs to be stored, accessed, and processed easily and efficiently. A very common data structure that is used for such a purpose is the Array data structure.

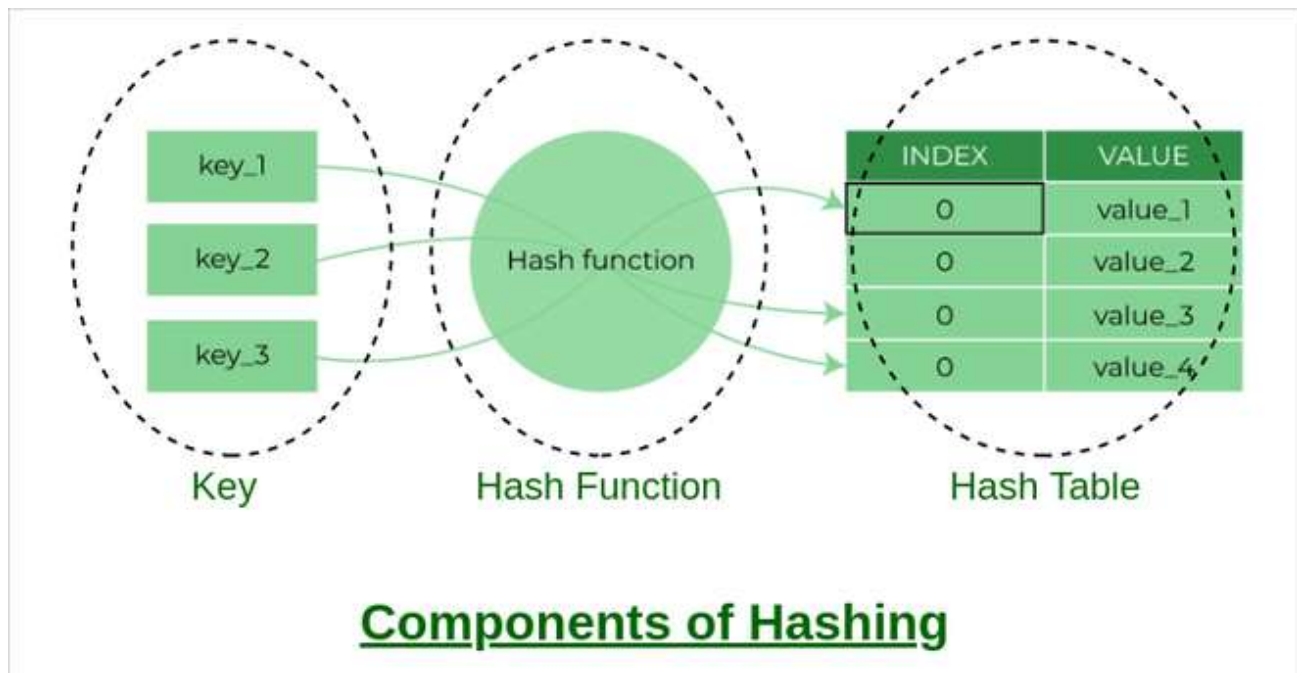
Now the question arises if Array was already there, what was the need for a new data structure! The answer to this is in the word “**efficiency**“. Though storing in Array takes  $O(1)$  time, searching in it takes at least  $O(\log n)$  time. This time appears to be small, but for a large data set, it can cause a lot of problems and this, in turn, makes the Array data structure inefficient.

So now we are looking for a data structure that can store the data and search in it in constant time, i.e. in  $O(1)$  time. This is how Hashing data structure came into play. With the introduction of the Hash data structure, it is now possible to easily store data in constant time and retrieve them in constant time as well.

#### Components of Hashing

There are majorly three components of hashing:

1. **Key:** A **Key** can be anything string or integer which is fed as input in the hash function the technique that determines an index or location for storage of an item in a data structure.
2. **Hash Function:** The **hash function** receives the input key and returns the index of an element in an array called a hash table. The index is known as the **hash index**.
3. **Hash Table:** Hash table is a data structure that maps keys to values using a special function called a hash function. Hash stores the data in an associative manner in an array where each data value has its own unique index.



## Components of Hashing

### How does Hashing work?

Suppose we have a set of strings {"ab", "cd", "efg"} and we would like to store it in a table.

Our main objective here is to search or update the values stored in the table quickly in  $O(1)$  time and we are not concerned about the ordering of strings in the table. So the given set of strings can act as a key and the string itself will act as the value of the string but how to store the value corresponding to the key?

- **Step 1:** We know that hash functions (which is some mathematical formula) are used to calculate the hash value which acts as the index of the data structure where the value will be stored.
- **Step 2:** So, let's assign
  - "a" = 1,
  - "b" = 2, .. etc, to all alphabetical characters.
- **Step 3:** Therefore, the numerical value by summation of all characters of the string:
  - "ab" =  $1 + 2 = 3$ ,
  - "cd" =  $3 + 4 = 7$ ,
  - "efg" =  $5 + 6 + 7 = 18$
- **Step 4:** Now, assume that we have a table of size 7 to store these strings. The hash function that is used here is the sum of the characters in **key mod Table size**. We can compute the location of the string in the array by taking the **sum(string) mod 7**.
- **Step 5:** So we will then store
  - "ab" in  $3 \bmod 7 = 3$ ,
  - "cd" in  $7 \bmod 7 = 0$ , and
  - "efg" in  $18 \bmod 7 = 4$ .



| 0  | 1 | 2 | 3  | 4   | 5 | 6 |
|----|---|---|----|-----|---|---|
| cd |   |   | ab | efg |   |   |

Mapping key with indices of array

The above technique enables us to calculate the location of a given string by using a simple hash function and rapidly find the value that is stored in that location. Therefore the idea of hashing seems like a great way to store (key, value) pairs of the data in a table.

[What is a Hash function?](#)

A **Hash Function** is a function that converts a given numeric or alphanumeric key to a small practical integer value. The mapped integer value is used as an index in the hash table. In simple terms, a hash function **maps** a significant number or string to a small integer that can be used as the **index** in the hash table.

The pair is of the form **(key, value)**, where for a given key, one can find a value using some kind of a “function” that maps keys to values. The key for a given object can be calculated using a function called a hash function. For example, given an array A, if i is the key, then we can find the value by simply looking up A[i].

The [hash function](#) creates a mapping between key and value, this is done through the use of mathematical formulas known as hash functions. The result of the hash function is referred to as a hash value or hash. The hash value is a representation of the original string of characters but usually smaller than the original.

For example: Consider an array as a Map where the key is the index and the value is the value at that index. So for an array A if we have index **i** which will be treated as the key then we can find the value by simply looking at the value at A[i].  
simply looking up A[i].

### Types of Hash functions:

There are many hash functions that use numeric or alphanumeric keys. This article focuses on discussing [different hash functions](#):

1. [Division Method.](#)
2. [Mid Square Method.](#)
3. [Folding Method.](#)
4. [Multiplication Method](#)

#### **1. Division Method:**

This is the most simple and easiest method to generate a hash value. The hash function divides the value k by M and then uses the remainder obtained.

**Formula:**

$$h(K) = k \bmod M$$

Here,

**k** is the key value, and

**M** is the size of the hash table.

It is best suited that **M** is a prime number as that can make sure the keys are more uniformly distributed. The hash function is dependent upon the remainder of a division.

**Example:**

$$k = 12345$$

$$M = 95$$

$$\begin{aligned} h(12345) &= 12345 \bmod 95 \\ &= 90 \end{aligned}$$

$$k = 1276$$

$$M = 11$$

$$\begin{aligned} h(1276) &= 1276 \bmod 11 \\ &= 0 \end{aligned}$$

**Pros:**

1. This method is quite good for any value of **M**.
2. The division method is very fast since it requires only a single division operation.

**Cons:**

1. This method leads to poor performance since consecutive keys map to consecutive hash values in the hash table.
2. Sometimes extra care should be taken to choose the value of **M**.

**2. Mid Square Method:**

The mid-square method is a very good hashing method. It involves two steps to compute the hash value-

1. Square the value of the key **k** i.e.  $k^2$
2. Extract the middle **r** digits as the hash value.

**Formula:**

$$h(K) = h(k \times k)$$

Here,

**k** is the key value.

The value of **r** can be decided based on the size of the table.

**Example:**

Suppose the hash table has 100 memory locations. So  $r = 2$  because two digits are required to map the key to the memory location.

$$\begin{aligned}k &= 60 \\k \times k &= 60 \times 60 \\&= 3600 \\h(60) &= 60\end{aligned}$$

The hash value obtained is 60

**Pros:**

1. The performance of this method is good as most or all digits of the key value contribute to the result. This is because all digits in the key contribute to generating the middle digits of the squared result.
2. The result is not dominated by the distribution of the top digit or bottom digit of the original key value.

**Cons:**

1. The size of the key is one of the limitations of this method, as the key is of big size then its square will double the number of digits.
2. Another disadvantage is that there will be collisions but we can try to reduce collisions.

**3. Digit Folding Method:**

This method involves two steps:

1. Divide the key-value **k** into a number of parts i.e. **k1, k2, k3, ..., kn**, where each part has the same number of digits except for the last part that can have lesser digits than the other parts.
2. Add the individual parts. The hash value is obtained by ignoring the last carry if any.

**Formula:**

$$\begin{aligned}k &= k1, k2, k3, k4, \dots, kn \\s &= k1 + k2 + k3 + k4 + \dots + kn \\h(K) &= s\end{aligned}$$

Here,  
**s** is obtained by adding the parts of the key **k**

**Example:**

$$\begin{aligned}k &= 12345 \\k1 &= 12, k2 = 34, k3 = 5 \\s &= k1 + k2 + k3 \\&= 12 + 34 + 5 \\&= 51 \\h(K) &= 51\end{aligned}$$

**Note:**

The number of digits in each part varies depending upon the size of the hash table. Suppose for example the size of the hash table is 100, then each part must have two digits except for the last part which can have a lesser number of digits.

**4. Multiplication Method**

This method involves the following steps:

1. Choose a constant value  $A$  such that  $0 < A < 1$ .
2. Multiply the key value with  $A$ .
3. Extract the fractional part of  $kA$ .
4. Multiply the result of the above step by the size of the hash table i.e.  $M$ .
5. The resulting hash value is obtained by taking the floor of the result obtained in step 4.

**Formula:**

$$h(K) = \text{floor} (M (kA \bmod 1))$$

Here,

$M$  is the size of the hash table.

$k$  is the key value.

$A$  is a constant value.

**Example:**

$$k = 12345$$

$$A = 0.357840$$

$$M = 100$$

$$\begin{aligned} h(12345) &= \text{floor}[ 100 (12345 * 0.357840 \bmod 1)] \\ &= \text{floor}[ 100 (4417.5348 \bmod 1) ] \\ &= \text{floor}[ 100 (0.5348) ] \\ &= \text{floor}[ 53.48 ] \\ &= 53 \end{aligned}$$

**Pros:**

The advantage of the multiplication method is that it can work with any value between 0 and 1, although there are some values that tend to give better results than the rest.

**Cons:**

The multiplication method is generally suitable when the table size is the power of two, then the whole process of computing the index by the key using multiplication hashing is very fast.

Commonly used hash functions:

Hash functions are widely used in computer science and cryptography for a variety of purposes, including data integrity, digital signatures, password storage, and more.

There are many types of hash functions, each with its own strengths and weaknesses. Here are a few of the most common types:

**1. SHA (Secure Hash Algorithm):** SHA is a family of cryptographic hash functions designed by the National Security Agency (NSA) in the United States. The most widely used SHA algorithms are SHA-1, SHA-2, and SHA-3. Here's a brief overview of each:

- **SHA-1:** SHA-1 is a 160-bit hash function that was widely used for digital signatures and other applications. However, it is no longer considered secure due to known vulnerabilities.
- **SHA-2:** SHA-2 is a family of hash functions that includes SHA-224, SHA-256, SHA-384, and SHA-512. These functions produce hash values of 224, 256, 384, and 512 bits, respectively. SHA-2 is widely used in security protocols such as SSL/TLS and is considered secure.
- **SHA-3:** SHA-3 is the latest member of the SHA family and was selected as the winner of the NIST hash function competition in 2012. It is designed to be faster and more secure than SHA-2 and produces hash values of 224, 256, 384, and 512 bits.

**2. CRC (Cyclic Redundancy Check):** CRC is a non-cryptographic hash function used primarily for error detection in data transmission. It is fast and efficient but is not suitable for security purposes. The basic idea behind CRC is to append a fixed-length check value, or checksum, to the end of a message. This checksum is calculated based on the contents of the message using a mathematical algorithm, and is then transmitted along with the message.

When the message is received, the receiver can recalculate the checksum using the same algorithm, and compare it with the checksum transmitted with the message. If the two checksums match, the receiver can be reasonably certain that the message was not corrupted during transmission.

The specific algorithm used for CRC depends on the application and the desired level of error detection. Some common CRC algorithms include CRC-16, CRC-32, and CRC-CCITT.

**3. MurmurHash:** MurmurHash is a fast and efficient non-cryptographic hash function designed for use in hash tables and other data structures. It is not suitable for security purposes as it is vulnerable to collision attacks.

**4. BLAKE2:** BLAKE2 is a cryptographic hash function designed to be fast and secure. It is an improvement over the popular SHA-3 algorithm and is widely used in applications that require high-speed hashing, such as cryptocurrency mining.

BLAKE2 is available in two versions: BLAKE2b and BLAKE2s. BLAKE2b is optimized for 64-bit platforms and produces hash values of up to 512 bits, while BLAKE2s is optimized for 8- to 32-bit platforms and produces hash values of up to 256 bits.

**5. Argon2:** Argon2 is a memory-hard password hashing function designed to be resistant to brute-force attacks. It is widely used for password storage and is recommended by the Password Hashing Competition. The main goal of Argon2 is to make it difficult for attackers to crack passwords by using techniques such as brute force attacks or dictionary attacks. It achieves this by using a computationally-intensive algorithm that makes it difficult for attackers to perform large numbers of password guesses in a short amount of time.

Argon2 has several key features that make it a strong choice for password hashing and key derivation:

- **Resistance to parallel attacks:** Argon2 is designed to be resistant to parallel attacks, meaning that it is difficult for attackers to use multiple processing units, such as GPUs or ASICs, to speed up password cracking.
- **Memory-hardness:** Argon2 is designed to be memory-hard, meaning that it requires a large amount of memory to compute the hash function. This makes it more difficult for attackers to use specialized hardware to crack passwords.
- **Customizable:** Argon2 is highly customizable, and allows users to adjust parameters such as the memory usage, the number of iterations, and the output length to meet their specific security requirements.

**Resistance to side-channel attacks:** Argon2 is designed to be resistant to side-channel attacks, such as timing attacks or power analysis attacks, that could be used to extract information about the password being hashed.

**6. MD5 (Message Digest 5):** MD5 is a widely-used cryptographic hash function that produces a 128-bit hash value. It is fast and efficient but is no longer recommended for security purposes due to known vulnerabilities. The basic idea behind MD5 is to take an input message of any length, and produce a fixed-length output, known as the hash value or message digest. This hash value is unique to the input message, and is generated using a mathematical algorithm that involves a series of logical operations, such as bitwise operations, modular arithmetic, and logical functions.

MD5 is widely used in a variety of applications, including digital signatures, password storage, and data integrity checks. However, it has been shown to have weaknesses that make it vulnerable to attacks. In particular, it is possible to generate two different messages with the same MD5 hash value, a vulnerability known as a collision attack.

There are many other types of hash functions, each with its own unique features and applications. The choice of hash function depends on the specific requirements of the application, such as speed, security, and memory usage.

### **Properties of a Good hash function:**

A hash function that maps every item into its own unique slot is known as a perfect hash function. We can construct a perfect hash function if we know the items and the collection will never change but the problem is that there is no systematic way to construct a perfect hash function given an arbitrary collection of items. Fortunately, we will still gain performance efficiency even if the hash function isn't perfect. We can achieve a perfect hash function by increasing the size of the hash table so that every possible value can be accommodated. As a result, each item will have a unique slot. Although this approach is feasible for a small number of items, it is not practical when the number of possibilities is large.

So, We can construct our hash function to do the same but the things that we must be careful about while constructing our own hash function.

A good hash function should have the following properties:

1. Efficiently computable.

2. Should uniformly distribute the keys (Each table position is equally likely for each.
3. Should minimize collisions.
4. Should have a low load factor(number of items in the table divided by the size of the table).

### Complexity of calculating hash value using the hash function

- Time complexity:  $O(n)$
- Space complexity:  $O(1)$

### Problem with Hashing:

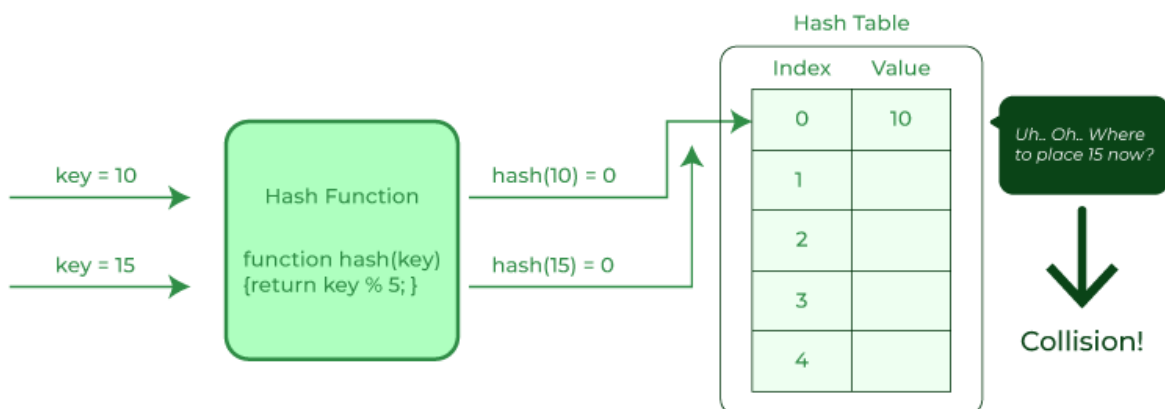
If we consider the above example, the hash function we used is the sum of the letters, but if we examined the hash function closely then the problem can be easily visualized that for different strings same hash value is begin generated by the hash function.

For example: {"ab", "ba"} both have the same hash value, and string {"cd", "be"} also generate the same hash value, etc. This is known as **collision** and it creates problem in searching, insertion, deletion, and updating of value.

### What is collision?

The hashing process generates a small number for a big key, so there is a possibility that two keys could produce the same value. The situation where the newly inserted key maps to an already occupied, and it must be handled using some collision handling technology.

## Collision in Hashing

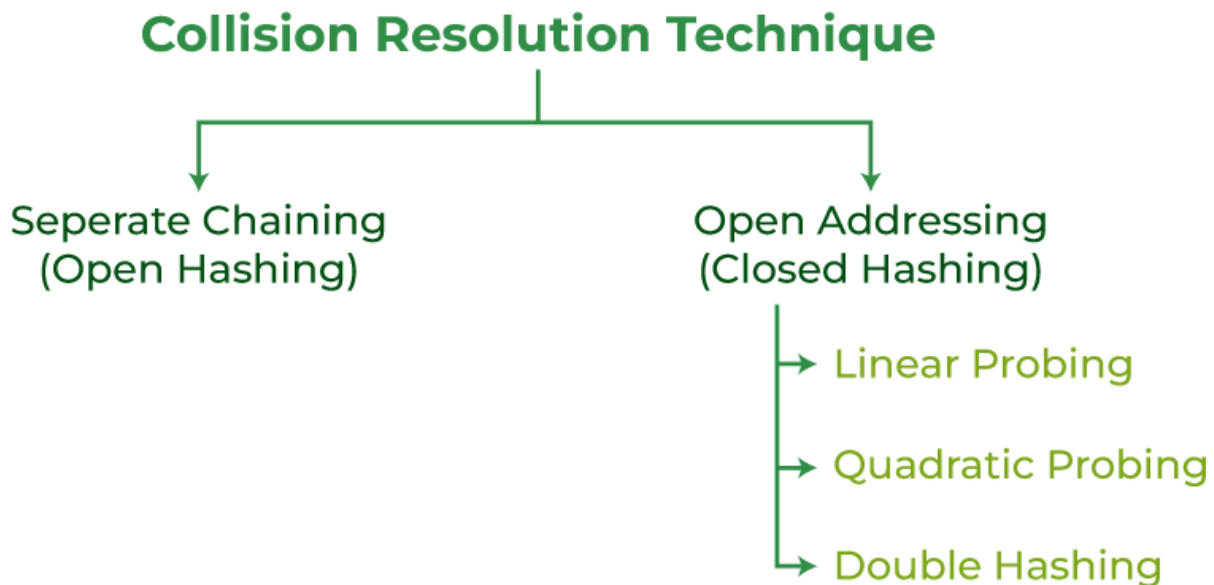


### What is Collision in Hashing

### How to handle Collisions?

There are mainly two methods to handle collision:

1. Separate Chaining:
2. Open Addressing:



Collision resolution technique

### 1) Separate Chaining:

The idea is to make each cell of the hash table point to a linked list of records that have the same hash function value. Chaining is simple but requires additional memory outside the table.

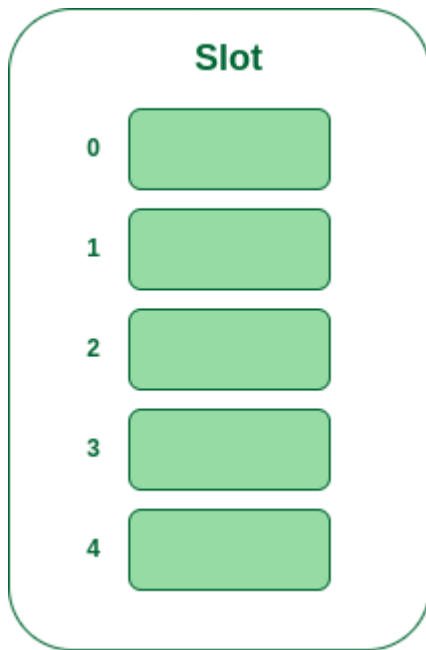
Example: We have given a hash function and we have to insert some elements in the hash table using a separate chaining method for collision resolution technique.

Hash function =  $\text{key} \% 5$ ,  
Elements = 12, 15, 22, 25 and 37.

Let's see step by step approach to how to solve the above problem:

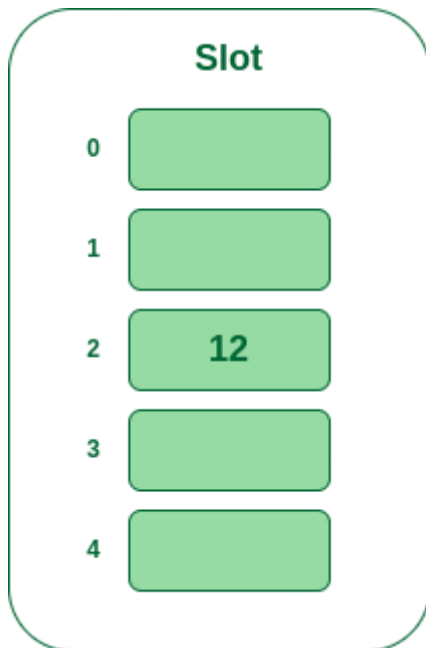
- **Step 1:** First draw the empty hash table which will have a possible range of hash values from 0 to 4 according to the hash function provided.





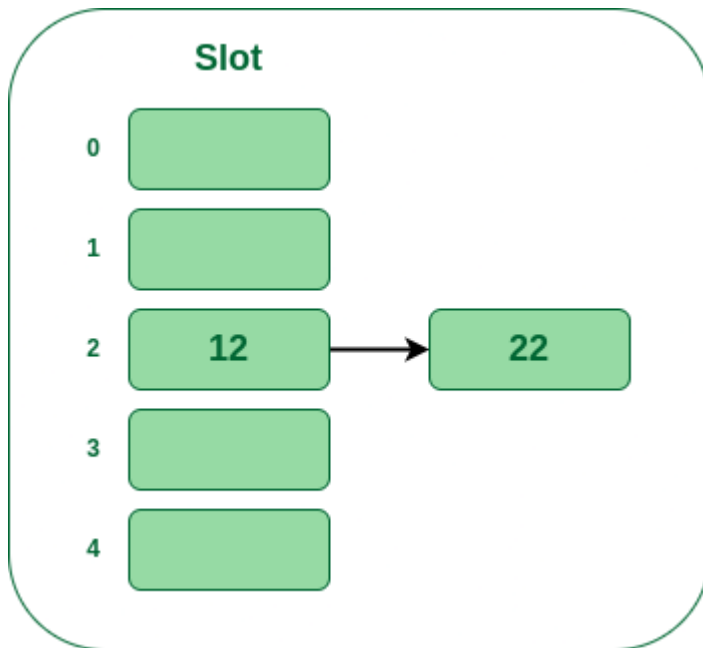
Hash table

- **Step 2:** Now insert all the keys in the hash table one by one. The first key to be inserted is 12 which is mapped to bucket number 2 which is calculated by using the hash function  $12\%5=2$ .



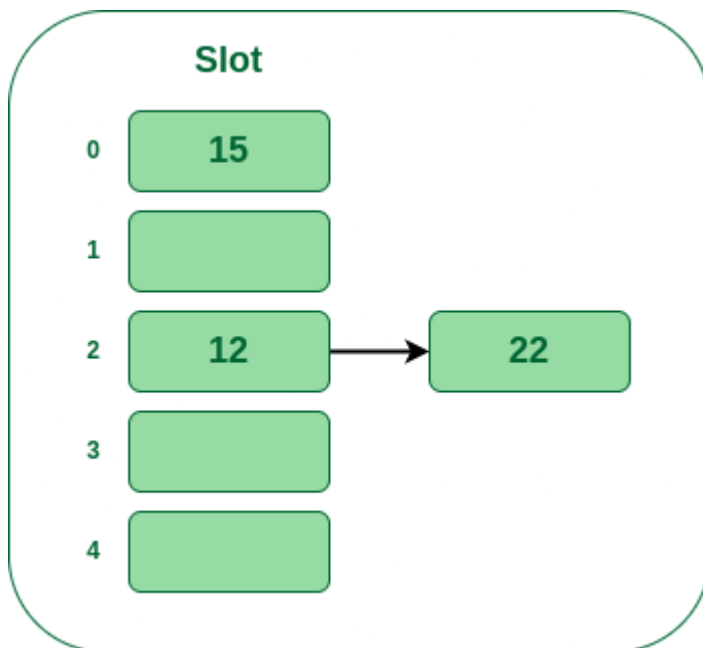
Insert 12

- **Step 3:** Now the next key is 22. It will map to bucket number 2 because  $22\%5=2$ . But bucket 2 is already occupied by key 12.



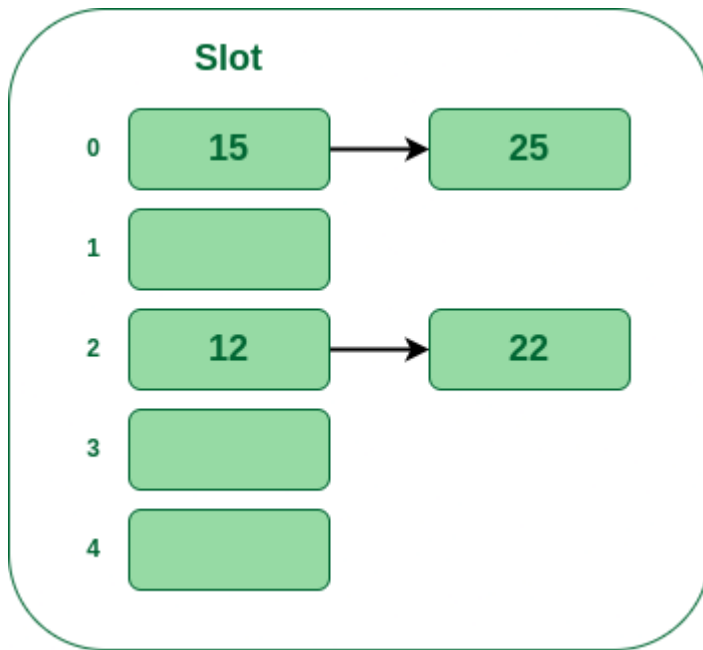
Insert 22

- **Step 4:** The next key is 15. It will map to slot number 0 because  $15\%5=0$ .



Insert 15

- **Step 5:** Now the next key is 25. Its bucket number will be  $25\%5=0$ . But bucket 0 is already occupied by key 25. So separate chaining method will again handle the collision by creating a linked list to bucket 0.



Insert 25

Hence In this way, the separate chaining method is used as the collision resolution technique.

## 2) Open Addressing:

In open addressing, all elements are stored in the hash table itself. Each table entry contains either a record or NIL. When searching for an element, we examine the table slots one by one until the desired element is found or it is clear that the element is not in the table.

### 2.a) Linear Probing

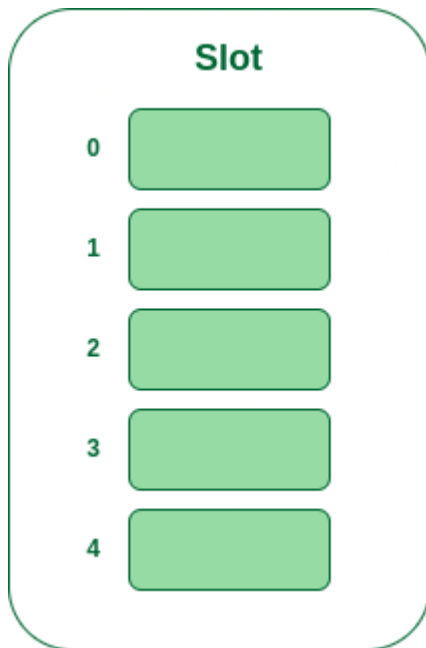
In linear probing, the hash table is searched sequentially that starts from the original location of the hash. If in case the location that we get is already occupied, then we check for the next location.

#### Algorithm:

1. Calculate the hash key. i.e. **key = data % size**
2. Check, if **hashTable[key]** is empty
  - store the value directly by **hashTable[key] = data**
3. If the hash index already has some value then
  1. check for next index using **key = (key+1) % size**
4. Check, if the next index is available hashTable[key] then store the value. Otherwise try for next index.
5. Do the above process till we find the space.

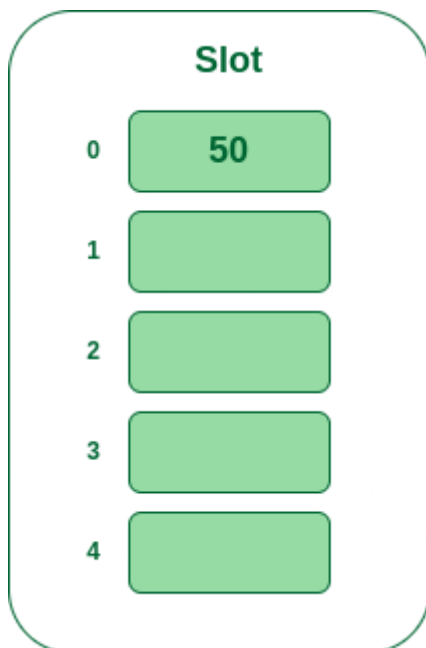
**Example:** Let us consider a simple hash function as “key mod 5” and a sequence of keys that are to be inserted are 50, 70, 76, 85, 93.

- **Step 1:** First draw the empty hash table which will have a possible range of hash values from 0 to 4 according to the hash function provided.



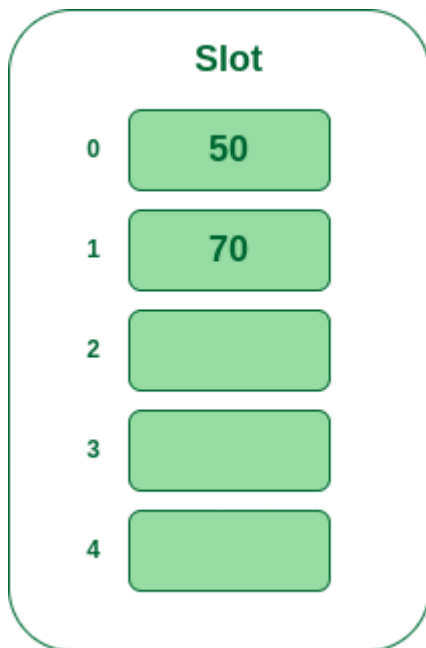
Hash table

- **Step 2:** Now insert all the keys in the hash table one by one. The first key is 50. It will map to slot number 0 because  $50 \% 5 = 0$ . So insert it into slot number 0.



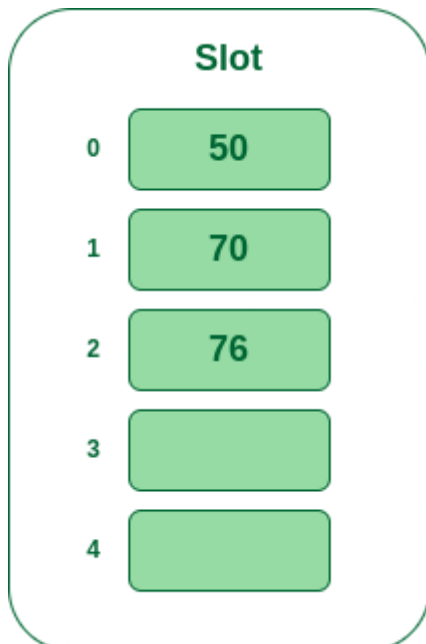
Insert 50 into hash table

- **Step 3:** The next key is 70. It will map to slot number 0 because  $70\%5=0$  but 50 is already at slot number 0 so, search for the next empty slot and insert it.



Insert 70 into hash table

- **Step 4:** The next key is 76. It will map to slot number 1 because  $76\%5=1$  but 70 is already at slot number 1 so, search for the next empty slot and insert it.



Insert 76 into hash table

- **Step 5:** The next key is 93 It will map to slot number 3 because  $93\%5=3$ , So insert it into slot number 3.

| Slot |    |
|------|----|
| 0    | 50 |
| 1    | 70 |
| 2    | 76 |
| 3    | 93 |
| 4    |    |

Insert 93 into hash table

## 2.b) Quadratic Probing

Quadratic probing is an open addressing scheme in computer programming for resolving hash collisions in hash tables. Quadratic probing operates by taking the original hash index and adding successive values of an arbitrary quadratic polynomial until an open slot is found.

An example sequence using quadratic probing is:

$$H + 1^2, H + 2^2, H + 3^2, H + 4^2, \dots, H + k^2$$

This method is also known as the mid-square method because in this method we look for  $i^2$ 'th probe (slot) in  $i$ 'th iteration and the value of  $i = 0, 1, \dots, n - 1$ . We always start from the original hash location. If only the location is occupied then we check the other slots.

Let  $\text{hash}(x)$  be the slot index computed using the hash function and  $n$  be the size of the hash table.

If the slot  $\text{hash}(x) \% n$  is full, then we try  $(\text{hash}(x) + 1^2) \% n$ .

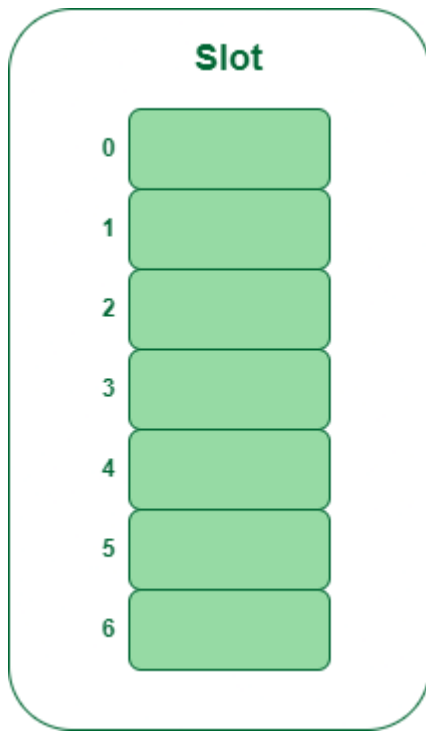
If  $(\text{hash}(x) + 1^2) \% n$  is also full, then we try  $(\text{hash}(x) + 2^2) \% n$ .

If  $(\text{hash}(x) + 2^2) \% n$  is also full, then we try  $(\text{hash}(x) + 3^2) \% n$ .

This process will be repeated for all the values of  $i$  until an empty slot is found

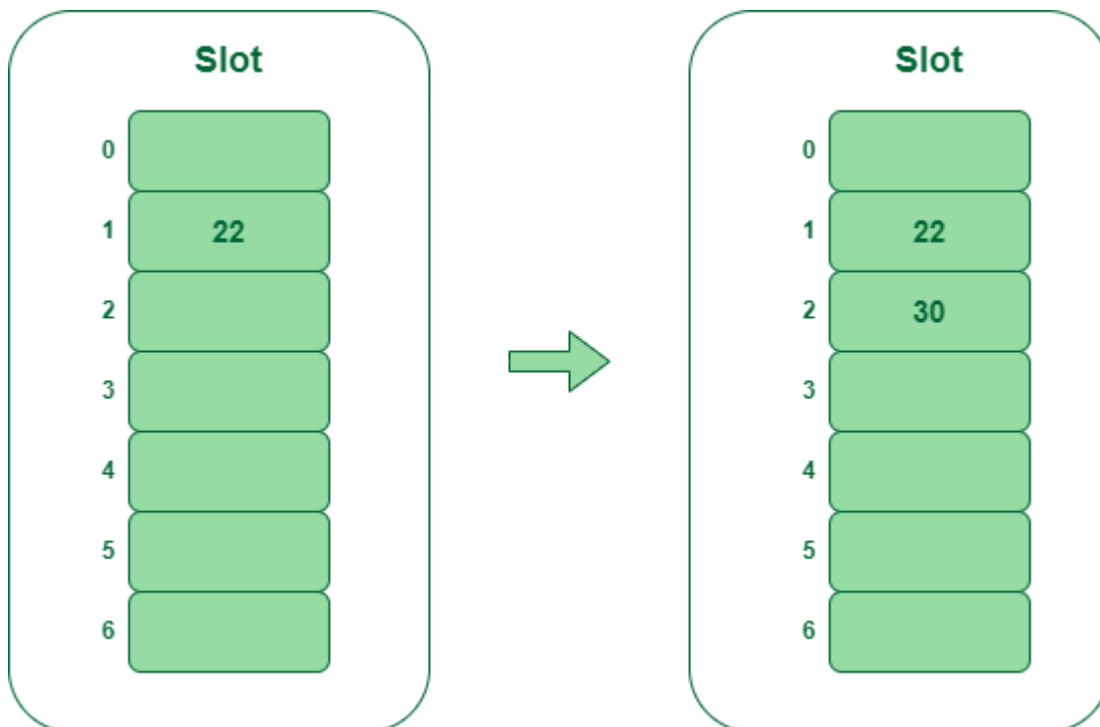
Example: Let us consider table Size = 7, hash function as  $\text{Hash}(x) = x \% 7$  and collision resolution strategy to be  $f(i) = i^2$ . Insert = 22, 30, and 50

- **Step 1:** Create a table of size 7.



Hash table

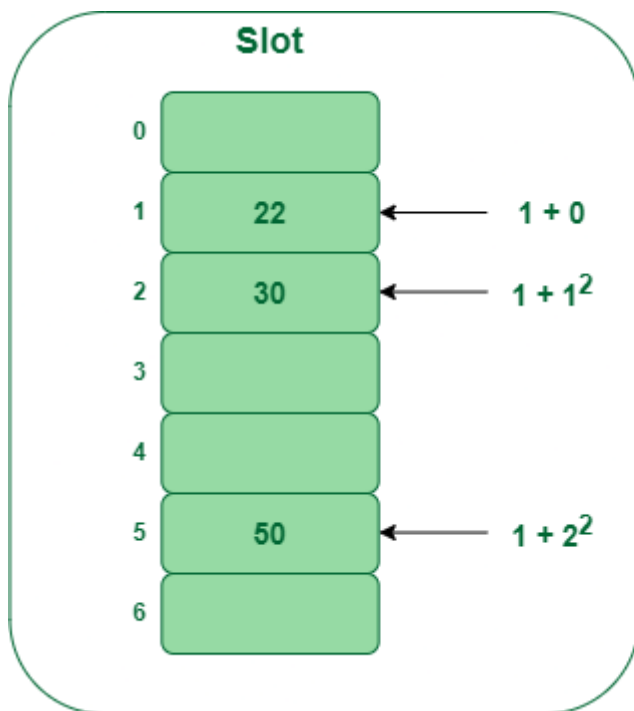
- **Step 2** – Insert 22 and 30
  - $\text{Hash}(22) = 22 \% 7 = 1$ , Since the cell at index 1 is empty, we can easily insert 22 at slot 1.
  - $\text{Hash}(30) = 30 \% 7 = 2$ , Since the cell at index 2 is empty, we can easily insert 30 at slot 2.



Insert key 22 and 30 in the hash table

- **Step 3:** Inserting 50

- $\text{Hash}(25) = 50 \% 7 = 1$
- In our hash table slot 1 is already occupied. So, we will search for slot  $1+1^2$ , i.e.  $1+1 = 2$ ,
- Again slot 2 is found occupied, so we will search for cell  $1+2^2$ , i.e.  $1+4 = 5$ ,
- Now, cell 5 is not occupied so we will place 50 in slot 5.



Insert key 50 in the hash table

## 2.c) Double Hashing

Double hashing is a collision resolving technique in [Open Addressed](#) Hash tables. Double hashing make use of two hash function,

- The first hash function is  **$h_1(k)$**  which takes the key and gives out a location on the hash table. But if the new location is not occupied or empty then we can easily place our key.
- But in case the location is occupied (collision) we will use secondary hash-function  **$h_2(k)$**  in combination with the first hash-function  **$h_1(k)$**  to find the new location on the hash table.

This combination of hash functions is of the form

$$h(k, i) = (h_1(k) + i * h_2(k)) \% n$$

where

- $i$  is a non-negative integer that indicates a collision number,
- $k$  = element/key which is being hashed
- $n$  = hash table size.

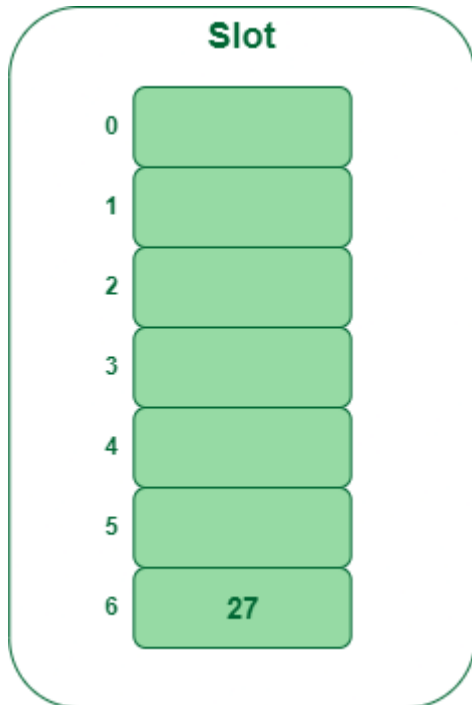
## Complexity of the Double hashing algorithm:

Time complexity:  $O(n)$



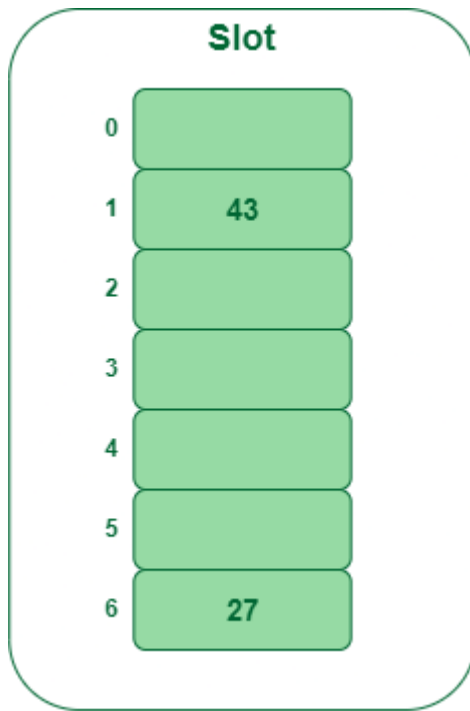
**Example:** Insert the keys 27, 43, 692, 72 into the Hash Table of size 7. where first hash-function is  $h_1(k) = k \bmod 7$  and second hash-function is  $h_2(k) = 1 + (k \bmod 5)$

- **Step 1:** Insert 27
  - $27 \% 7 = 6$ , location 6 is empty so insert 27 into 6 slot.



Insert key 27 in the hash table

- **Step 2:** Insert 43
  - $43 \% 7 = 1$ , location 1 is empty so insert 43 into 1 slot.

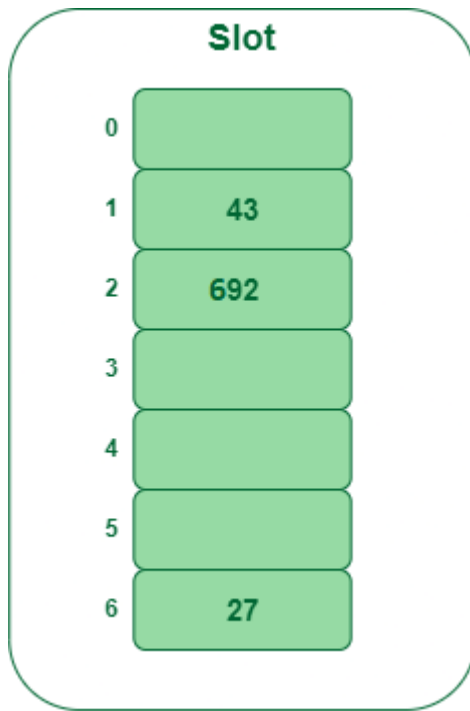


Insert key 43 in the hash table

- **Step 3:** Insert 692
  - $692 \% 7 = 6$ , but location 6 is already being occupied and this is a collision
  - So we need to resolve this collision using double hashing.

$$\begin{aligned}h_{\text{new}} &= [h_1(692) + i * (h_2(692))] \% 7 \\&= [6 + 1 * (1 + 692 \% 5)] \% 7 \\&= 9 \% 7 \\&= 2\end{aligned}$$

Now, as 2 is an empty slot,  
so we can insert 692 into 2nd slot.

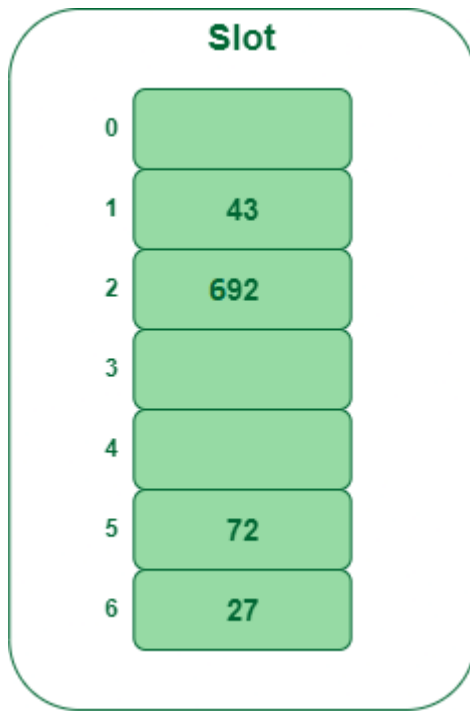


Insert key 692 in the hash table

- **Step 4:** Insert 72
  - $72 \% 7 = 2$ , but location 2 is already being occupied and this is a collision.
  - So we need to resolve this collision using double hashing.

$$\begin{aligned}h_{\text{new}} &= [h_1(72) + i * (h_2(72))] \% 7 \\&= [2 + 1 * (1 + 72 \% 5)] \% 7 \\&= 5 \% 7 \\&= 5,\end{aligned}$$

Now, as 5 is an empty slot,  
so we can insert 72 into 5th slot.



Insert key 72 in the hash table

### What is meant by Load Factor in Hashing?

The [load factor](#) of the hash table can be defined as the number of items the hash table contains divided by the size of the hash table. Load factor is the decisive parameter that is used when we want to rehash the previous hash function or want to add more elements to the existing hash table.

It helps us in determining the efficiency of the hash function i.e. it tells whether the hash function which we are using is distributing the keys uniformly or not in the hash table.

Load Factor = Total elements in hash table / Size of hash table

### What is Rehashing?

As the name suggests, [rehashing](#) means hashing again. Basically, when the load factor increases to more than its predefined value (the default value of the load factor is 0.75), the complexity increases. So to overcome this, the size of the array is increased (doubled) and all the values are hashed again and stored in the new double-sized array to maintain a low load factor and low complexity.

### Applications of Hash Data structure

- Hash is used in databases for indexing.
- Hash is used in disk-based data structures.
- In some programming languages like Python, JavaScript hash is used to implement objects.

### Real-Time Applications of Hash Data structure

- Hash is used for cache mapping for fast access to the data.

- Hash can be used for password verification.
- Hash is used in cryptography as a message digest.
- Rabin-Karp algorithm for pattern matching in a string.
- Calculating the number of different substrings of a string.

### Advantages of Hash Data structure

- Hash provides better synchronization than other data structures.
- Hash tables are more efficient than search trees or other data structures
- Hash provides constant time for searching, insertion, and deletion operations on average.

### Disadvantages of Hash Data structure

- Hash is inefficient when there are many collisions.
- Hash collisions are practically not avoided for a large set of possible keys.
- Hash does not allow null values.

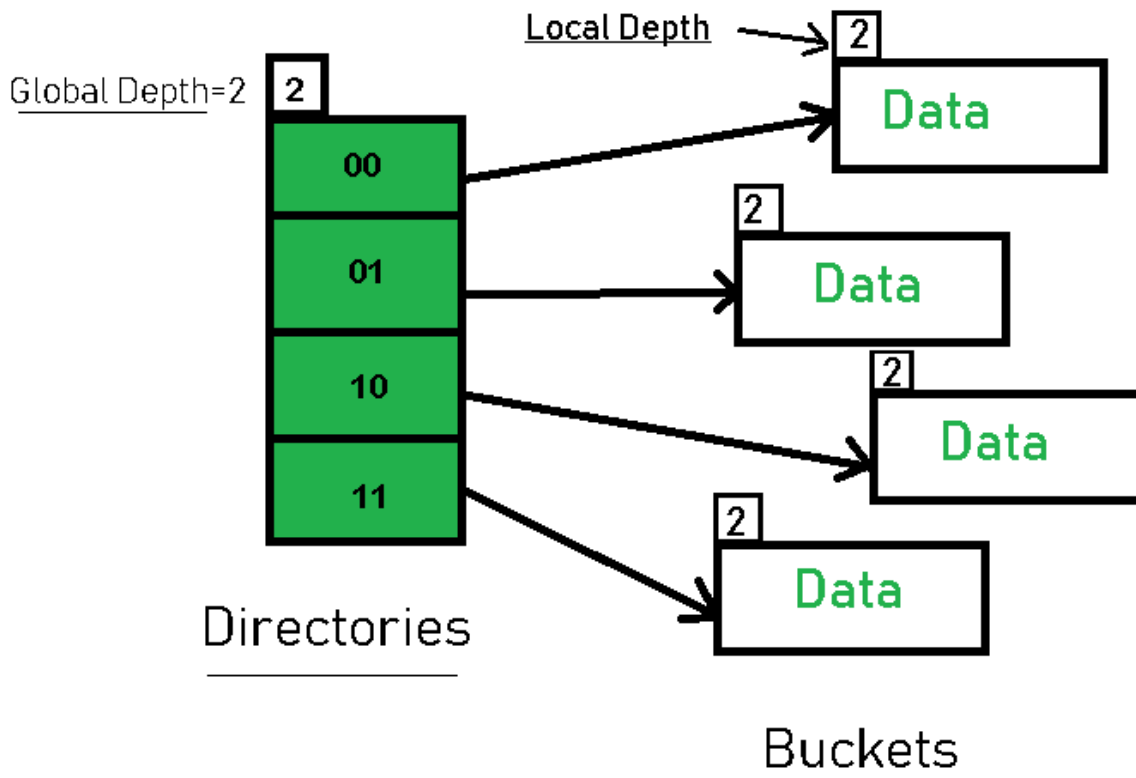
## Extendible Hashing (Dynamic approach to DBMS)

**Extendible Hashing** is a dynamic hashing method wherein directories, and buckets are used to hash data. It is an aggressively flexible method in which the hash function also experiences dynamic changes.

**Main features of Extendible Hashing:** The main features in this hashing technique are:

- **Directories:** The directories store addresses of the buckets in pointers. An id is assigned to each directory which may change each time when Directory Expansion takes place.
- **Buckets:** The buckets are used to hash the actual data.

### Basic Structure of Extendible Hashing:

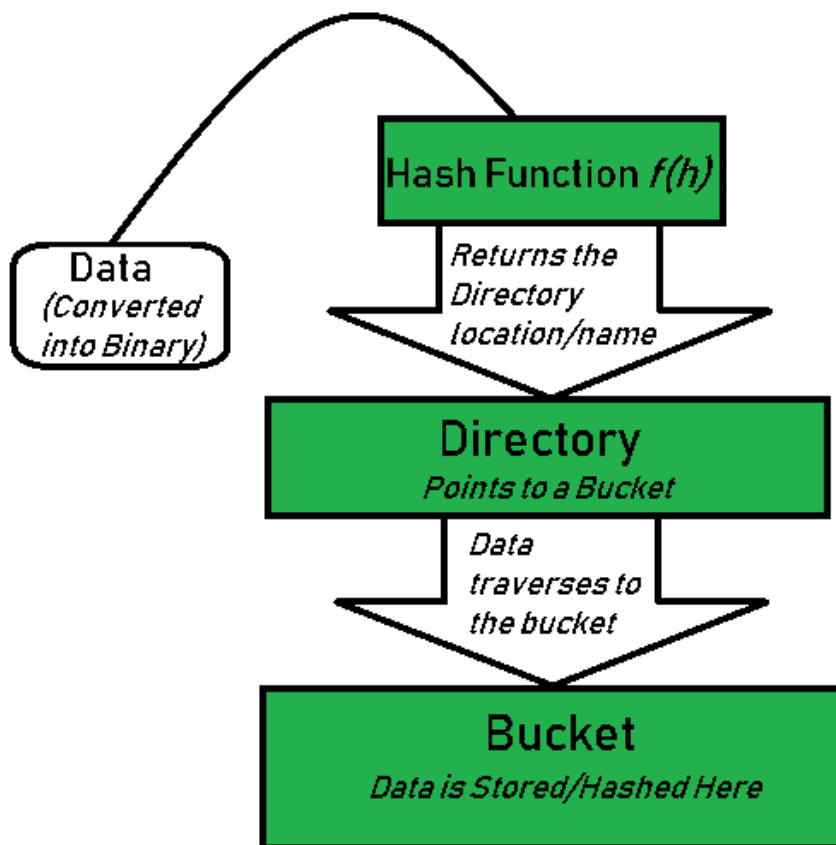


## Extendible Hashing

### Frequently used terms in Extendible Hashing:

- **Directories:** These containers store pointers to buckets. Each directory is given a unique id which may change each time when expansion takes place. The hash function returns this directory id which is used to navigate to the appropriate bucket. Number of Directories =  $2^{\text{Global Depth}}$ .
- **Buckets:** They store the hashed keys. Directories point to buckets. A bucket may contain more than one pointers to it if its local depth is less than the global depth.
- **Global Depth:** It is associated with the Directories. They denote the number of bits which are used by the hash function to categorize the keys. Global Depth = Number of bits in directory id.
- **Local Depth:** It is the same as that of Global Depth except for the fact that Local Depth is associated with the buckets and not the directories. Local depth in accordance with the global depth is used to decide the action that to be performed in case an overflow occurs. Local Depth is always less than or equal to the Global Depth.
- **Bucket Splitting:** When the number of elements in a bucket exceeds a particular size, then the bucket is split into two parts.
- **Directory Expansion:** Directory Expansion Takes place when a bucket overflows. Directory Expansion is performed when the local depth of the overflowing bucket is equal to the global depth.

### Basic Working of Extendible Hashing:

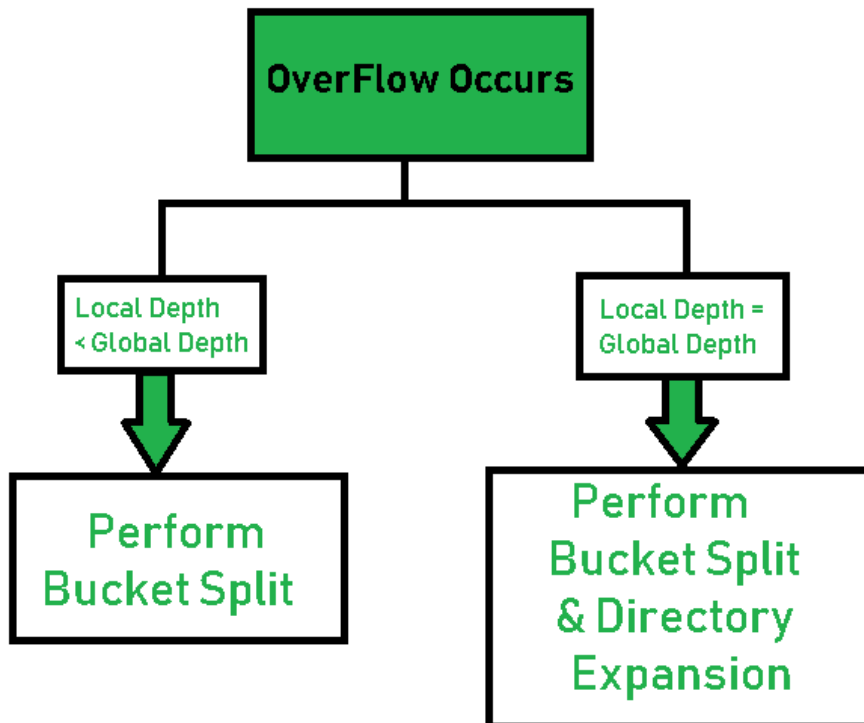


- **Step 1 – Analyze Data Elements:** Data elements may exist in various forms eg. Integer, String, Float, etc.. Currently, let us consider data elements of type integer. eg: 49.
- **Step 2 – Convert into binary format:** Convert the data element in Binary form. For string elements, consider the ASCII equivalent integer of the starting character and then convert the integer into binary form. Since we have 49 as our data element, its binary form is 110001.
- **Step 3 – Check Global Depth of the directory.** Suppose the global depth of the Hash-directory is 3.
- **Step 4 – Identify the Directory:** Consider the ‘Global-Depth’ number of LSBs in the binary number and match it to the directory id.  
Eg. The binary obtained is: 110001 and the global-depth is 3. So, the hash function will return 3 LSBs of 110001 viz. 001.
- **Step 5 – Navigation:** Now, navigate to the bucket pointed by the directory with directory-id 001.
- **Step 6 – Insertion and Overflow Check:** Insert the element and check if the bucket overflows. If an overflow is encountered, go to **step 7** followed by **Step 8**, otherwise, go to **step 9**.
- **Step 7 – Tackling Over Flow Condition during Data Insertion:** Many times, while inserting data in the buckets, it might happen that the Bucket overflows. In such cases, we need to follow an appropriate procedure to avoid mishandling of data.  
First, Check if the local depth is less than or equal to the global depth. Then choose one of the cases below.
  - **Case1:** If the local depth of the overflowing Bucket is equal to the global depth, then Directory Expansion, as well as Bucket Split, needs to be performed. Then increment the

global depth and the local depth value by 1. And, assign appropriate pointers.

Directory expansion will double the number of directories present in the hash structure.

- **Case2:** In case the local depth is less than the global depth, then only Bucket Split takes place. Then increment only the local depth value by 1. And, assign appropriate pointers.



- **Step 8 – Rehashing of Split Bucket Elements:** The Elements present in the overflowing bucket that is split are rehashed w.r.t the new global depth of the directory.
- **Step 9 –** The element is successfully hashed.

#### **Example based on Extendible Hashing:**

Now, let us consider a prominent example of hashing the following elements:

**16,4,6,22,24,10,31,7,9,20,26.**

**Bucket Size:** 3 (Assume)

**Hash Function:** Suppose the global depth is X. Then the Hash Function returns X LSBs.

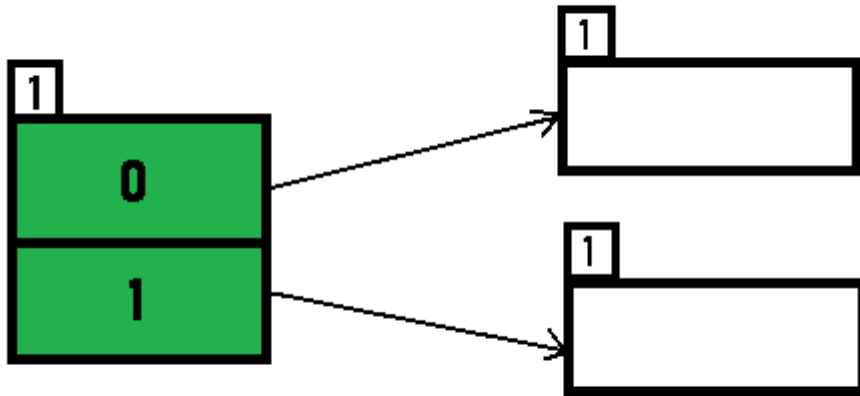
- **Solution:** First, calculate the binary forms of each of the given numbers.
  - 16- 10000
  - 4- 00100
  - 6- 00110
  - 22- 10110
  - 24- 11000
  - 10- 01010
  - 31- 11111
  - 7- 00111
  - 9- 01001



20- 10100

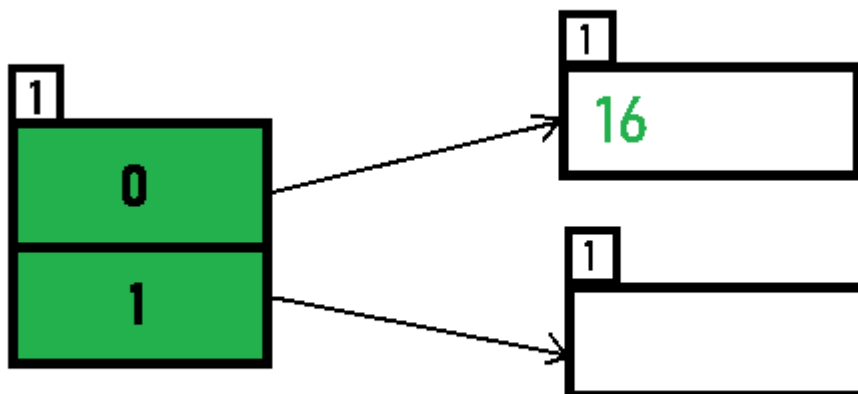
26- 11010

- Initially, the global-depth and local-depth is always 1. Thus, the hashing frame looks like this:



- Inserting 16:**

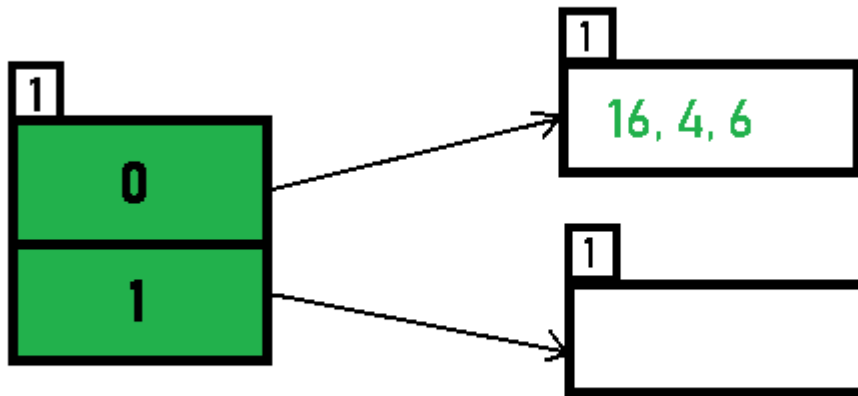
The binary format of 16 is 10000 and global-depth is 1. The hash function returns 1 LSB of 10000 which is 0. Hence, 16 is mapped to the directory with id=0.



*Hash(16) = 10000*

- Inserting 4 and 6:**

Both 4(100) and 6(110) have 0 in their LSB. Hence, they are hashed as follows:



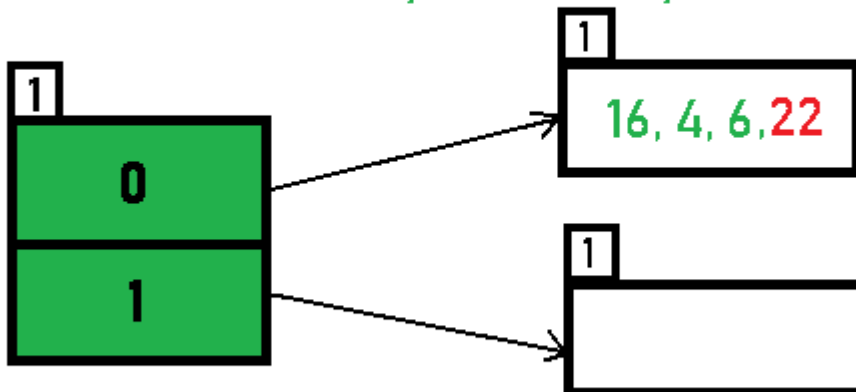
$Hash(4) = 100$

$Hash(6) = 110$

- **Inserting 22:** The binary form of 22 is 10110. Its LSB is 0. The bucket pointed by directory 0 is already full. Hence, Over Flow occurs.

## Overflow Condition

*Here, Local Depth = Global Depth*

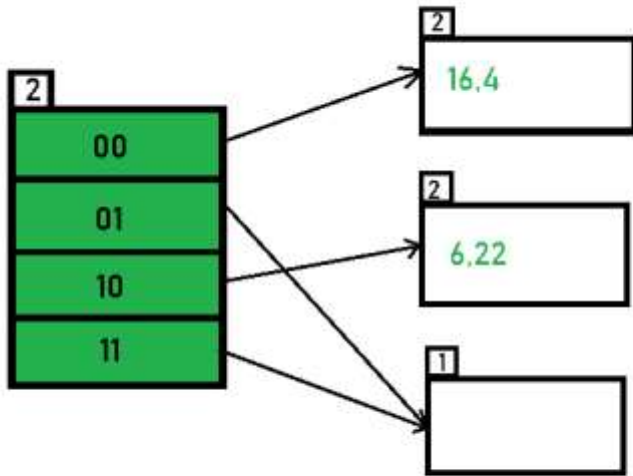


$Hash(22) = 10110$

- As directed by **Step 7-Case 1**, Since Local Depth = Global Depth, the bucket splits and directory expansion takes place. Also, rehashing of numbers present in the overflowing bucket takes place after the split. And, since the global depth is incremented by 1, now, the global depth is 2. Hence, 16, 4, 6, 22 are now rehashed w.r.t 2 LSBs.  
16(10000), 4(100), 6(110), 22(10110) ]

•

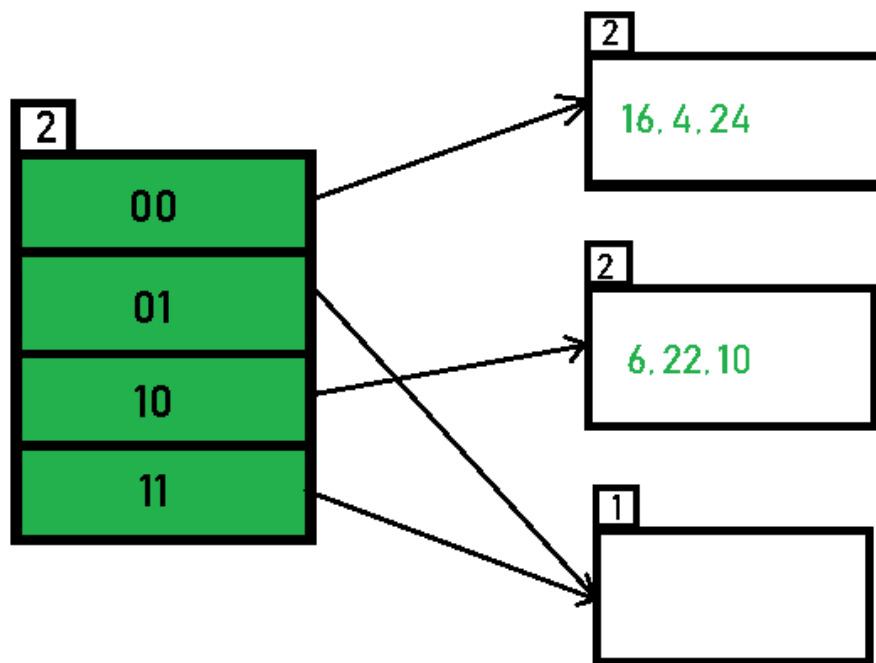
*After Bucket Split and Directory Expansion*



•

\*Notice that the bucket which was underflow has remained untouched. But, since the number of directories has doubled, we now have 2 directories 01 and 11 pointing to the same bucket. This is because the local-depth of the bucket has remained 1. And, any bucket having a local depth less than the global depth is pointed-to by more than one directories.

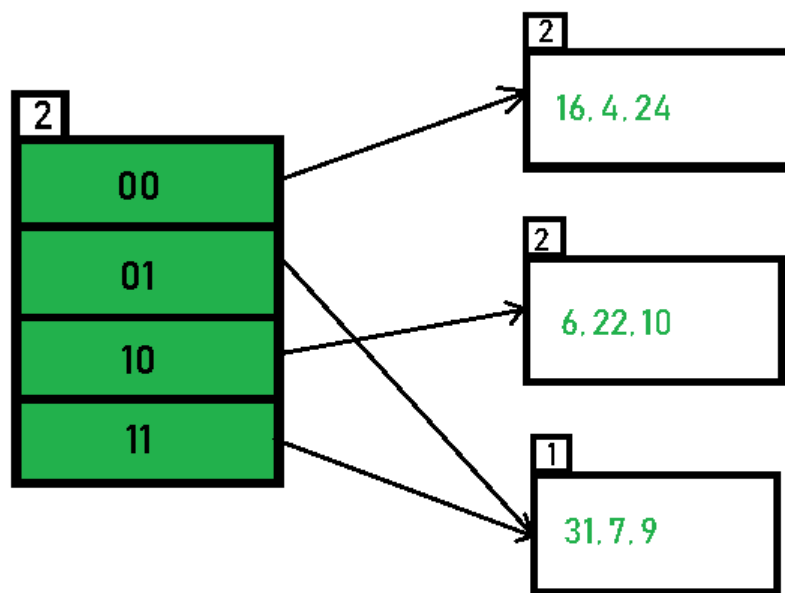
- **Inserting 24 and 10:** 24(11000) and 10 (1010) can be hashed based on directories with id 00 and 10. Here, we encounter no overflow condition.



$Hash(24) = 11000$

$Hash(10) = 1010$

- **Inserting 31,7,9:** All of these elements[ 31(11111), 7(111), 9(1001) ] have either 01 or 11 in their LSBs. Hence, they are mapped on the bucket pointed out by 01 and 11. We do not encounter any overflow condition here.



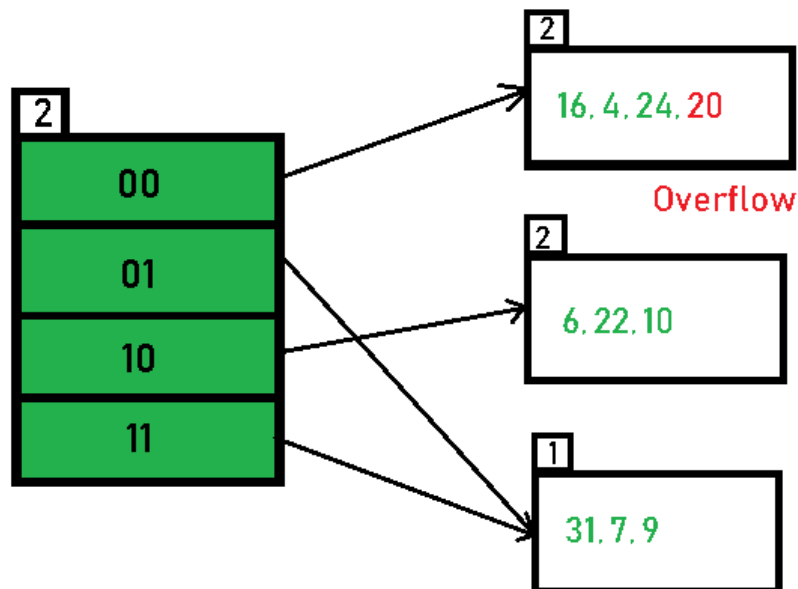
$Hash(31) = 1111$

$Hash(7) = 111$

$Hash(9) = 1001$

- **Inserting 20:** Insertion of data element 20 (10100) will again cause the overflow problem.

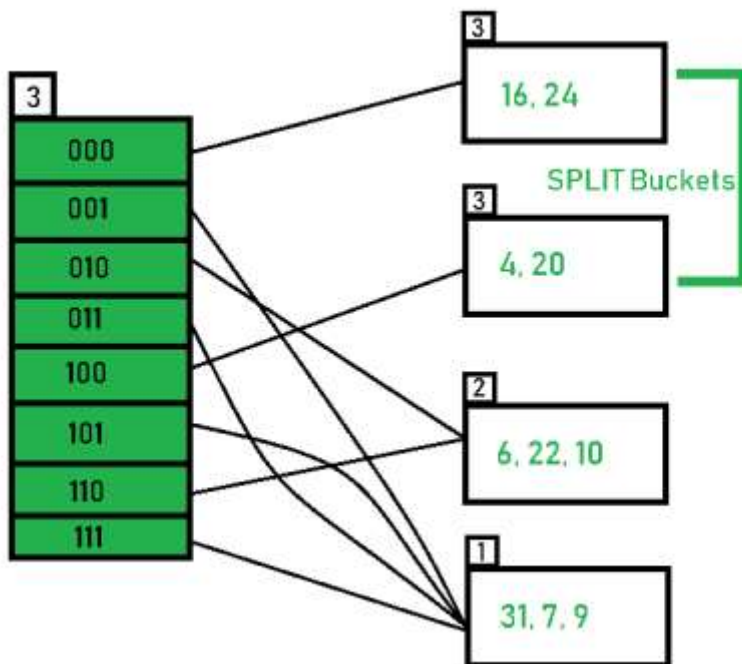
### Overflow, Local Depth = Global Depth



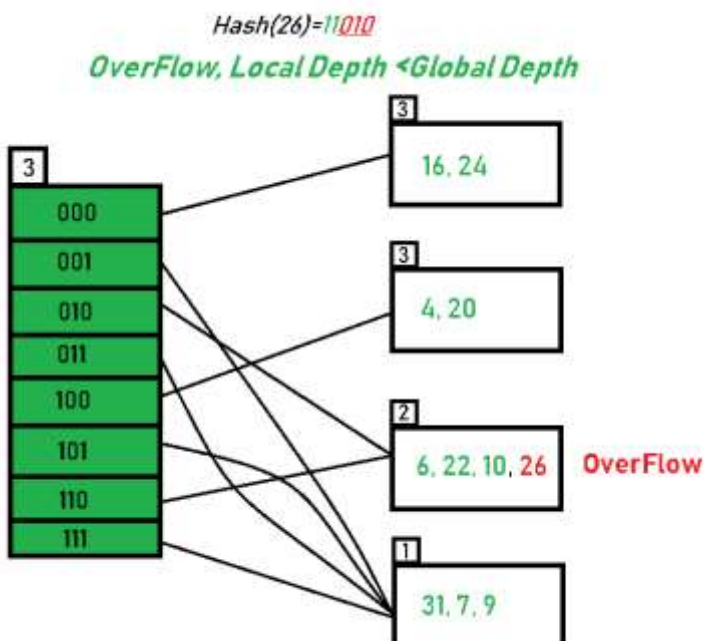
$Hash(20) = 10100$

- 20 is inserted in bucket pointed out by 00. As directed by **Step 7-Case 1**, since the **local depth of the bucket = global-depth**, directory expansion (doubling) takes place along with bucket

splitting. Elements present in overflowing bucket are rehashed with the new global depth. Now, the new Hash table looks like this:



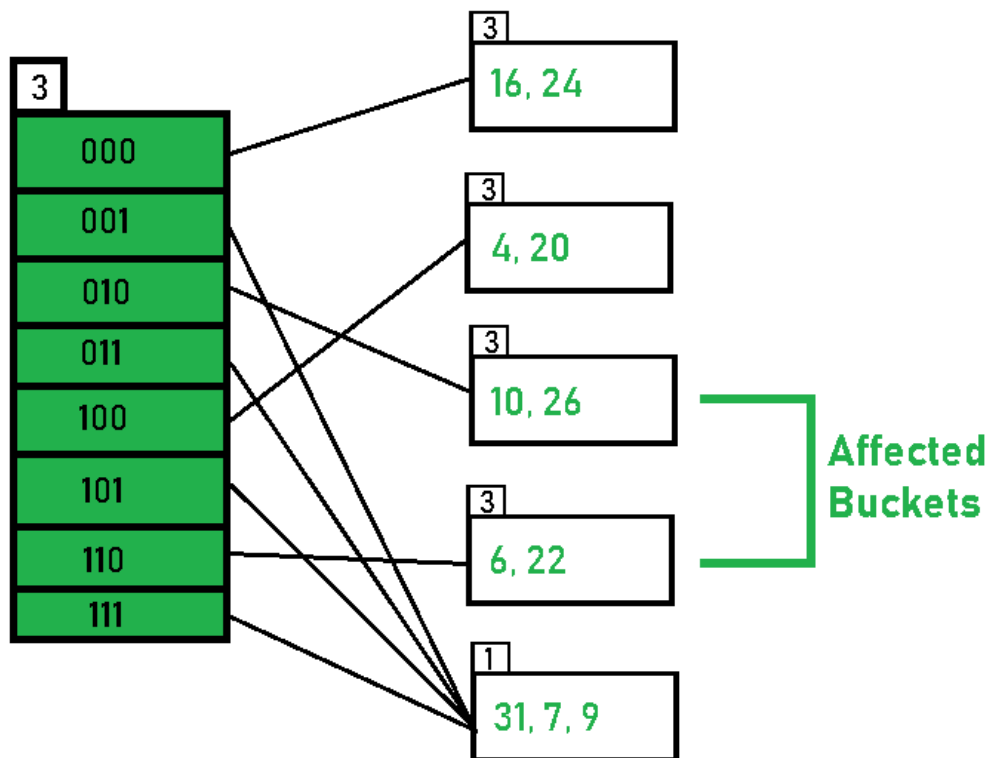
- **Inserting 26:** Global depth is 3. Hence, 3 LSBs of 26(11010) are considered. Therefore 26 best fits in the bucket pointed out by directory 010.



- The bucket overflows, and, as directed by **Step 7-Case 2**, since the **local depth of bucket** < **Global depth** (2<3), directories are not doubled but, only the bucket is split and

elements are rehashed.

Finally, the output of hashing the given list of numbers is obtained.



- **Hashing of 11 Numbers is Thus Completed.**

#### **Key Observations:**

1. A Bucket will have more than one pointers pointing to it if its local depth is less than the global depth.
2. When overflow condition occurs in a bucket, all the entries in the bucket are rehashed with a new local depth.
3. If Local Depth of the overflowing bucket
4. The size of a bucket cannot be changed after the data insertion process begins.

#### **Advantages:**

1. Data retrieval is less expensive (in terms of computing).
2. No problem of Data-loss since the storage capacity increases dynamically.
3. With dynamic changes in hashing function, associated old values are rehashed w.r.t the new hash function.

#### **Limitations Of Extendible Hashing:**

1. The directory size may increase significantly if several records are hashed on the same directory while keeping the record distribution non-uniform.
2. Size of every bucket is fixed.
3. Memory is wasted in pointers when the global depth and local depth difference becomes drastic.
4. This method is complicated to code.

### **Data Structures used for implementation:**

1. [B+ Trees](#)
2. [Array](#)
3. [Linked List](#)

### **Perfect Hashing Function:**

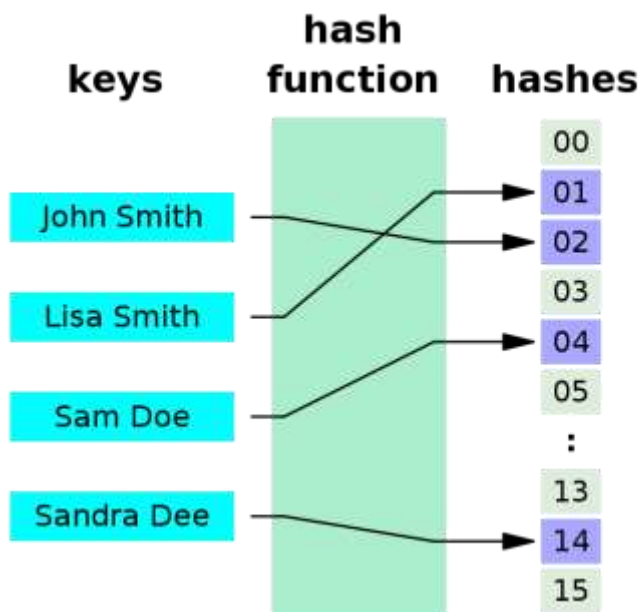
A perfect hash function is one that maps the set of actual key values to the table without any collisions. In [computer science](#), a **perfect hash function**  $h$  for a set  $S$  is a [hash function](#) that maps distinct elements in  $S$  to a set of  $m$  integers, with no [collisions](#).

Perfect hash functions may be used to implement a [lookup table](#) with constant worst-case access time. A perfect hash function can, as any [hash function](#), be used to implement [hash tables](#), with the advantage that no [collision resolution](#) has to be implemented. In addition, if the keys are not in the data and if it is known that queried keys will be valid, then the keys do not need to be stored in the lookup table, saving space.

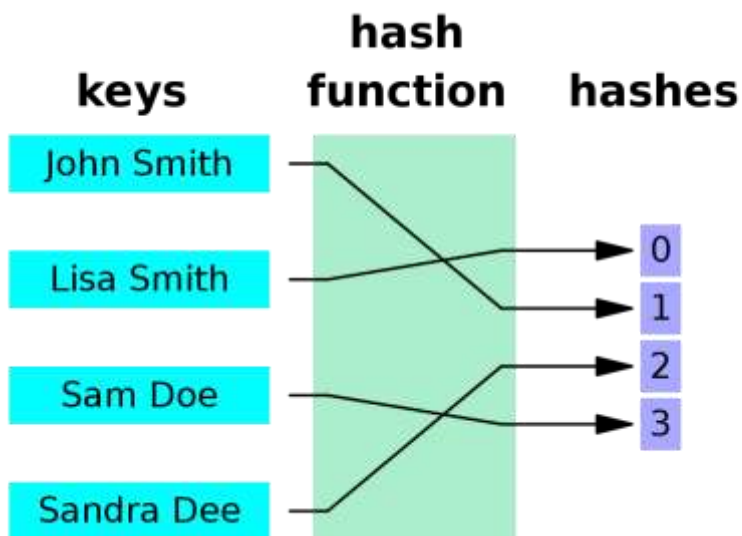
Disadvantages of perfect hash functions are that  $S$  needs to be known for the construction of the perfect hash function. Non-dynamic perfect hash functions need to be re-constructed if  $S$  changes. For frequently changing  $S$  [dynamic perfect hash functions](#) may be used at the cost of additional space. The space requirement to store the perfect hash function is in  $O(n)$ .

The important performance parameters for perfect hash functions are the evaluation time, which should be constant, the construction time, and the representation size.





A minimal perfect hash function does so using a table that has only as many slots as there are key values to be hashed.



A perfect hash function with values in a limited range can be used for efficient lookup operations, by placing keys from  $S$  (or other associated values) in a [lookup table](#) indexed by the output of the function. One can then test whether a key is present in  $S$ , or look up a value associated with that key, by looking for it at its cell of the table. Each such lookup takes [constant time](#) in the [worst case](#). With perfect hashing, the associated data can be read or written with a single access to the table.

## 5.5 Sorting Algorithms

### Selection Sort:

**Selection sort** is a simple and efficient sorting algorithm that works by repeatedly selecting the smallest (or largest) element from the unsorted portion of the list and moving it to the sorted portion of the list.

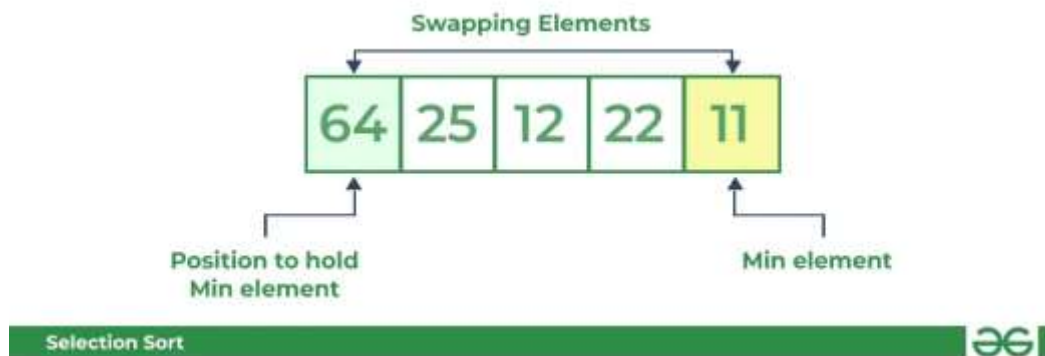
The algorithm repeatedly selects the smallest (or largest) element from the unsorted portion of the list and swaps it with the first element of the unsorted part. This process is repeated for the remaining unsorted portion until the entire list is sorted.

#### How does Selection Sort Algorithm work?

Lets consider the following array as an example: `arr[] = {64, 25, 12, 22, 11}`

#### **First pass:**

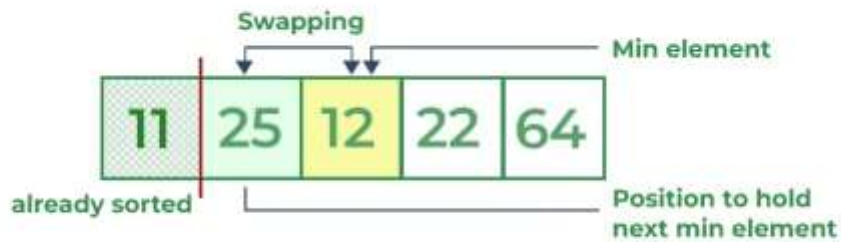
- For the first position in the sorted array, the whole array is traversed from index 0 to 4 sequentially. The first position where **64** is stored presently, after traversing whole array it is clear that **11** is the lowest value.
- Thus, replace 64 with 11. After one iteration **11**, which happens to be the least value in the array, tends to appear in the first position of the sorted list.



Selection Sort Algorithm | Swapping 1st element with the minimum in array

#### **Second Pass:**

- For the second position, where 25 is present, again traverse the rest of the array in a sequential manner.
- After traversing, we found that **12** is the second lowest value in the array and it should appear at the second place in the array, thus swap these values.



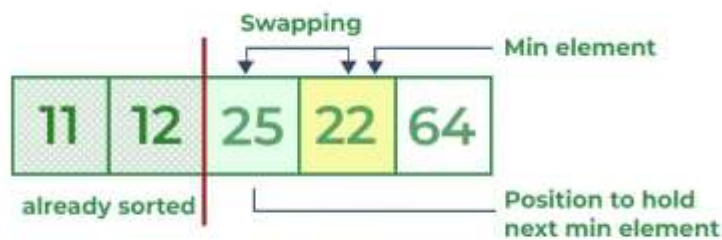
Selection Sort



Selection Sort Algorithm | swapping  $i=1$  with the next minimum element

### Third Pass:

- Now, for third place, where **25** is present again traverse the rest of the array and find the third least value present in the array.
- While traversing, **22** came out to be the third least value and it should appear at the third place in the array, thus swap **22** with element present at third position.



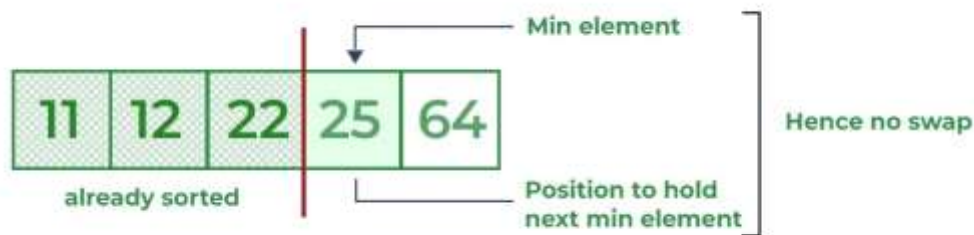
Selection Sort



Selection Sort Algorithm | swapping  $i=2$  with the next minimum element

### Fourth pass:

- Similarly, for fourth position traverse the rest of the array and find the fourth least element in the array
- As **25** is the 4th lowest value hence, it will place at the fourth position.



Selection Sort



Selection Sort Algorithm | swapping  $i=3$  with the next minimum element

#### Fifth Pass:

- At last the largest value present in the array automatically get placed at the last position in the array
- The resulted array is the sorted array.



Selection Sort



Selection Sort Algorithm | Required sorted array

Below is the implementation of the above approach:

```
// C++ program for implementation of
```

```
// selection sort
```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Function for Selection sort
```

```
void selectionSort(int arr[], int n)
```

```
{
```

```

int i, j, min_idx;

// One by one move boundary of
// unsorted subarray
for (i = 0; i < n - 1; i++) {

    // Find the minimum element in
    // unsorted array
    min_idx = i;
    for (j = i + 1; j < n; j++) {
        if (arr[j] < arr[min_idx])
            min_idx = j;
    }

    // Swap the found minimum element
    // with the first element
    if (min_idx != i)
        swap(arr[min_idx], arr[i]);
    }
}

// Function to print an array
void printArray(int arr[], int size)
{
    int i;
    for (i = 0; i < size; i++) {
        cout << arr[i] << " ";
        cout << endl;
    }
}

```

```
// Driver program
int main()
{
    int arr[] = { 64, 25, 12, 22, 11 };
    int n = sizeof(arr) / sizeof(arr[0]);

    // Function Call
    selectionSort(arr, n);
    cout << "Sorted array: \n";
    printArray(arr, n);
    return 0;
}

// This is code is contributed by rathbhupendra
```

## Output

Sorted array:  
11 12 22 25 64

## [Complexity Analysis of Selection Sort](#)

**Time Complexity:** The time complexity of Selection Sort is  $O(N^2)$  as there are two nested loops:

- One loop to select an element of Array one by one =  $O(N)$
- Another loop to compare that element with every other Array element =  $O(N)$
- Therefore overall complexity =  $O(N) * O(N) = O(N*N) = O(N^2)$

**Auxiliary Space:**  $O(1)$  as the only extra memory used is for temporary variables while swapping two values in Array. The selection sort never makes more than  $O(N)$  swaps and can be useful when memory writing is costly.

## [Advantages of Selection Sort Algorithm](#)

- Simple and easy to understand.
- Works well with small datasets.

## [Disadvantages of the Selection Sort Algorithm](#)

- Selection sort has a time complexity of  $O(n^2)$  in the worst and average case.
- Does not work well on large datasets.
- Does not preserve the relative order of items with equal keys which means it is not stable.

## [Frequently Asked Questions on Selection Sort](#)

### **Q1. Is Selection Sort Algorithm stable?**

The default implementation of the Selection Sort Algorithm is **not stable**. However, it can be made stable. Please see the [stable Selection Sort](#) for details.

### **Q2. Is Selection Sort Algorithm in-place?**

Yes, Selection Sort Algorithm is an in-place algorithm, as it does not require extra space.

## **Bubble Sort:**

**Bubble Sort** is the simplest [sorting algorithm](#) that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large data sets as its average and worst-case time complexity is quite high.

## [Bubble Sort Algorithm](#)

In this algorithm,

- traverse from left and compare adjacent elements and the higher one is placed at right side.
- In this way, the largest element is moved to the rightmost end at first.
- This process is then continued to find the second largest and place it and so on until the data is sorted.

## [How does Bubble Sort Work?](#)

Let us understand the working of bubble sort with the help of the following illustration:

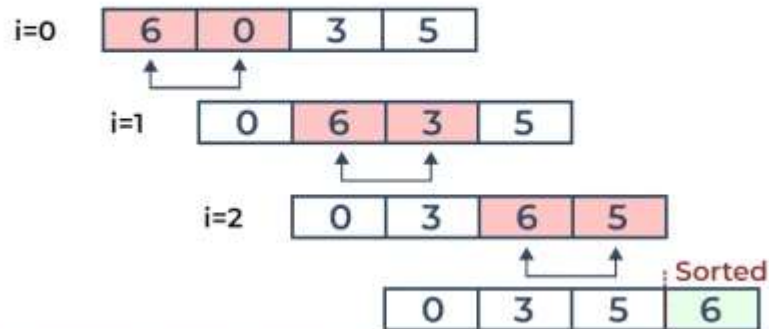
**Input:** arr[] = {6, 3, 0, 5}

### **First Pass:**

The largest element is placed in its correct position, i.e., the end of the array.

STEP  
01

Placing the 1<sup>st</sup> largest element at Correct position



Bubble sort



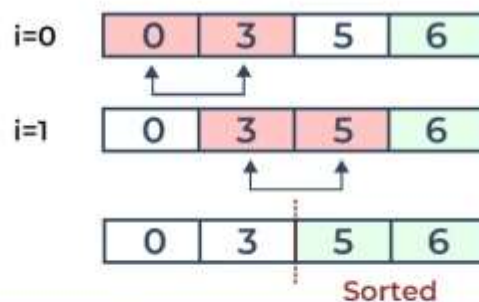
Bubble Sort Algorithm : Placing the largest element at correct position

**Second Pass:**

Place the second largest element at correct position

STEP  
02

Placing 2<sup>nd</sup> largest element at Correct position



Bubble sort



Bubble Sort Algorithm : Placing the second largest element at correct position

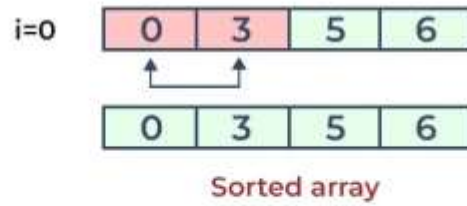
**Third Pass:**

Place the remaining two elements at their correct positions.



STEP  
03

## Placing 3<sup>rd</sup> largest element at Correct position



Bubble sort



Bubble Sort Algorithm : Placing the remaining elements at their correct positions

### Implementation of Bubble Sort

Below is the implementation of the bubble sort. It can be optimized by stopping the algorithm if the inner loop didn't cause any swap.

// Optimized implementation of Bubble sort

```
#include <stdbool.h>
```

```
#include <stdio.h>
```

```
void swap(int* xp, int* yp)
```

```
{
```

```
    int temp = *xp;
```

```
    *xp = *yp;
```

```
    *yp = temp;
```

```
}
```

// An optimized version of Bubble Sort

```
void bubbleSort(int arr[], int n)
```

```
{
```

```
    int i, j;
```

```
    bool swapped;
```

```
    for (i = 0; i < n - 1; i++) {
```

```
swapped = false;
for (j = 0; j < n - i - 1; j++) {
    if (arr[j] > arr[j + 1]) {
        swap(&arr[j], &arr[j + 1]);
        swapped = true;
    }
}

// If no two elements were swapped by inner loop,
// then break
if (swapped == false)
    break;
}
```

```
// Function to print an array
void printArray(int arr[], int size)
{
    int i;
    for (i = 0; i < size; i++)
        printf("%d ", arr[i]);
}
```

```
// Driver program to test above functions
int main()
{
    int arr[] = { 64, 34, 25, 12, 22, 11, 90 };
    int n = sizeof(arr) / sizeof(arr[0]);
    bubbleSort(arr, n);
    printf("Sorted array: \n");
    printArray(arr, n);
}
```

```
    return 0;
}
```

## Output

Sorted array:  
11 12 22 25 34 64 90

## Complexity Analysis of Bubble Sort:

**Time**

**Complexity:**

$O(N^2)$

**Auxiliary Space:**  $O(1)$

## Advantages of Bubble Sort:

- Bubble sort is easy to understand and implement.
- It does not require any additional memory space.
- It is a stable sorting algorithm, meaning that elements with the same key value maintain their relative order in the sorted output.

## Disadvantages of Bubble Sort:

- Bubble sort has a time complexity of  $O(N^2)$  which makes it very slow for large data sets.
- Bubble sort is a comparison-based sorting algorithm, which means that it requires a comparison operator to determine the relative order of elements in the input data set. It can limit the efficiency of the algorithm in certain cases.

## Some FAQs related to Bubble Sort:

What is the Boundary Case for Bubble sort?

Bubble sort takes minimum time (Order of  $n$ ) when elements are already sorted. Hence it is best to check if the array is already sorted or not beforehand, to avoid  $O(N^2)$  time complexity.

## Does sorting happen in place in Bubble sort?

Yes, Bubble sort performs the swapping of adjacent pairs without the use of any major data structure. Hence Bubble sort algorithm is an in-place algorithm.

## Is the Bubble sort algorithm stable?

Yes, the bubble sort algorithm is stable.

## Where is the Bubble sort algorithm used?

Due to its simplicity, bubble sort is often used to introduce the concept of a sorting algorithm. In computer graphics, it is popular for its capability to detect a tiny error (like a swap of just two elements) in almost-sorted arrays and fix it with just linear complexity ( $2n$ ).

## **Insertion Sort:**

**Insertion sort** is a simple sorting algorithm that works similar to the way you sort playing cards in your hands. The array is virtually split into a sorted and an unsorted part. Values from the unsorted part are picked and placed at the correct position in the sorted part.

### Insertion Sort Algorithm

To sort an array of size N in ascending order iterate over the array and compare the current element (key) to its predecessor, if the key element is smaller than its predecessor, compare it to the elements before. Move the greater elements one position up to make space for the swapped element.

### Working of Insertion Sort algorithm

Consider an example: arr[]: {12, 11, 13, 5, 6}

**12    11    13    5    6**

#### **First Pass:**

- Initially, the first two elements of the array are compared in insertion sort.

**12    11    13    5    6**

- Here, 12 is greater than 11 hence they are not in the ascending order and 12 is not at its correct position. Thus, swap 11 and 12.
- So, for now 11 is stored in a sorted sub-array.

**11    12    13    5    6**

#### **Second Pass:**

- Now, move to the next two elements and compare them

**11    12    13    5    6**

- Here, 13 is greater than 12, thus both elements seems to be in ascending order, hence, no swapping will occur. 12 also stored in a sorted sub-array along with 11

#### **Third Pass:**

- Now, two elements are present in the sorted sub-array which are **11** and **12**
- Moving forward to the next two elements which are 13 and 5

**11    12    13    5    6**

- Both 5 and 13 are not present at their correct place so swap them

11    12    5    **13**    6

- After swapping, elements 12 and 5 are not sorted, thus swap again

11    5    **12**    13    6

- Here, again 11 and 5 are not sorted, hence swap again

5    **11**    12    13    6

- Here, 5 is at its correct position

#### **Fourth Pass:**

- Now, the elements which are present in the sorted sub-array are **5, 11** and **12**
- Moving to the next two elements 13 and 6

5    11    12    **13**    **6**

- Clearly, they are not sorted, thus perform swap between both

5    11    12    **6**    **13**

- Now, 6 is smaller than 12, hence, swap again

5    11    **6**    **12**    13

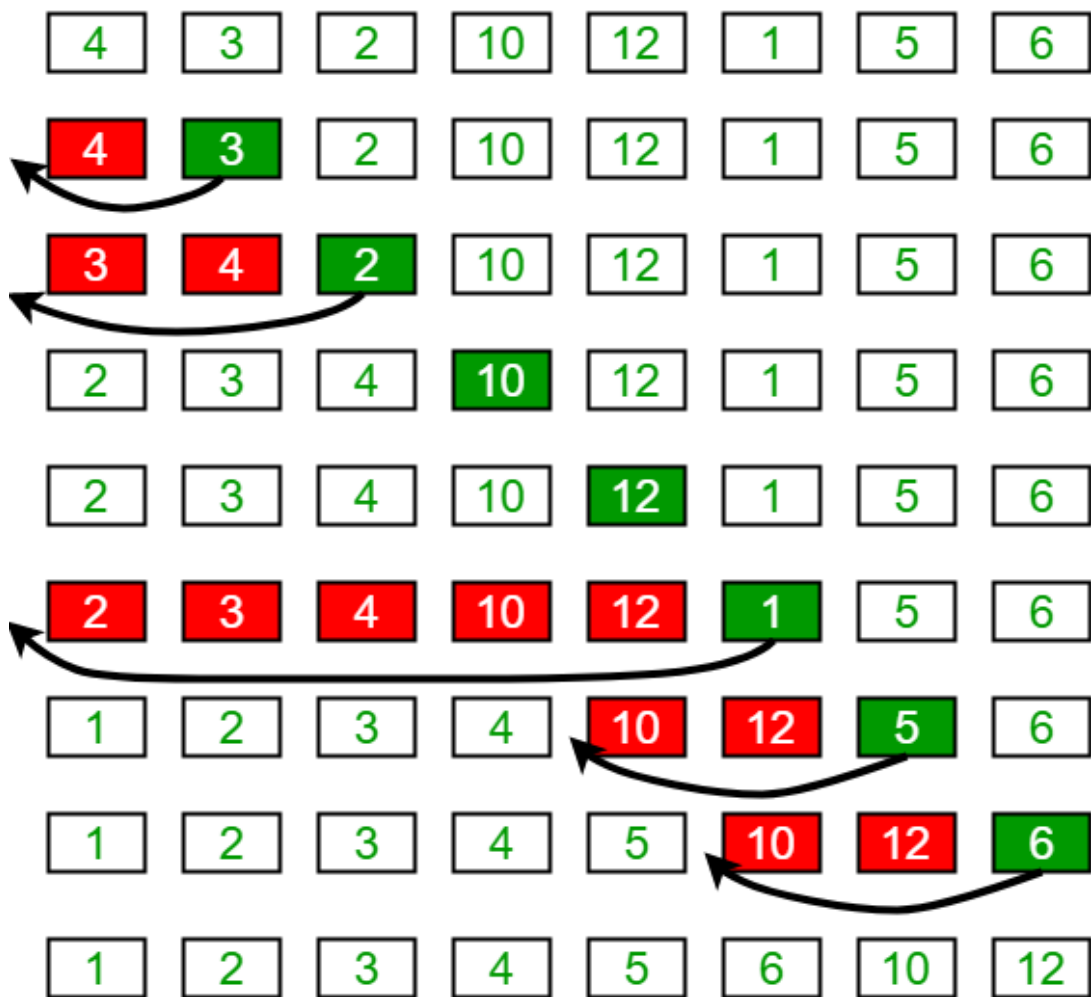
- Here, also swapping makes 11 and 6 unsorted hence, swap again

5    **6**    **11**    12    13

- Finally, the array is completely sorted.

#### **Illustrations:**

## Insertion Sort Execution Example



### [Implementation of Insertion Sort Algorithm](#)

Below is the implementation of the iterative approach:

```
// C++ program for insertion sort
```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Function to sort an array using insertion sort
```

```
void insertionSort(int arr[], int n)
```

```

{
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;

        // Move elements of arr[0..i-1], that are greater than key, to one position ahead of their
        // current position
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

```

// A utility function to print an array of size n

```
void printArray(int arr[], int n)
```

```

{
    int i;
    for (i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

```

// Driver code

```
int main()
```

```

{
    int arr[] = { 12, 11, 13, 5, 6 };
    int N = sizeof(arr) / sizeof(arr[0]);
}

```

```
insertionSort(arr, N);  
printArray(arr, N);  
  
return 0;  
}  
// This code is contributed by rathbhupendra
```

### Output

5 6 11 12 13

**Time Complexity:**  $O(N^2)$

**Auxiliary Space:**  $O(1)$

### Complexity Analysis of Insertion Sort:

#### Time Complexity of Insertion Sort

- The **worst-case** time complexity of the Insertion sort is  $O(N^2)$
- The **average case** time complexity of the Insertion sort is  $O(N^2)$
- The time complexity of the **best case** is  $O(N)$ .

#### Space Complexity of Insertion Sort

The auxiliary space complexity of Insertion Sort is  $O(1)$

### Characteristics of Insertion Sort

- This algorithm is one of the simplest algorithms with a simple implementation
- Basically, Insertion sort is efficient for small data values
- Insertion sort is adaptive in nature, i.e. it is appropriate for data sets that are already partially sorted.

### Frequently Asked Questions on Insertion Sort

#### Q1. What are the Boundary Cases of the Insertion Sort algorithm?

Insertion sort takes the maximum time to sort if elements are sorted in reverse order. And it takes minimum time (Order of  $n$ ) when elements are already sorted.

#### Q2. What is the Algorithmic Paradigm of the Insertion Sort algorithm?

The Insertion Sort algorithm follows an incremental approach.

#### Q3. Is Insertion Sort an in-place sorting algorithm?

Yes, insertion sort is an in-place sorting algorithm.



#### Q4. Is Insertion Sort a stable algorithm?

Yes, insertion sort is a stable sorting algorithm.

#### Q5. When is the Insertion Sort algorithm used?

Insertion sort is used when number of elements is small. It can also be useful when the input array is almost sorted, and only a few elements are misplaced in a complete big array.

5.5.1 Insertion Sort

5.5.2 Selection Sort

5.5.3 Bubble Sort

5.5.4 Merge Sort

## Merge Sort algorithm

The MergeSort function keeps on splitting an array into two halves until a condition is met where we try to perform MergeSort on a subarray of size 1, i.e.,  $p == r$ .

And then, it combines the individually sorted subarrays into larger arrays until the whole array is merged.

- ALGORITHM-MERGE SORT
- 1. If  $p < r$
- 2. Then  $q \rightarrow (p + r)/2$
- 3. MERGE-SORT (A, p, q)
- 4. MERGE-SORT (A, q+1, r)
- 5. MERGE (A, p, q, r)

Here we called **MergeSort(A, 0, length(A)-1)** to sort the complete array.

As you can see in the image given below, the merge sort algorithm recursively divides the array into halves until the base condition is met, where we are left with only 1 element in the array. And then, the merge function picks up the sorted sub-arrays and merge them back to sort the entire array. The following figure illustrates the dividing (splitting) procedure.

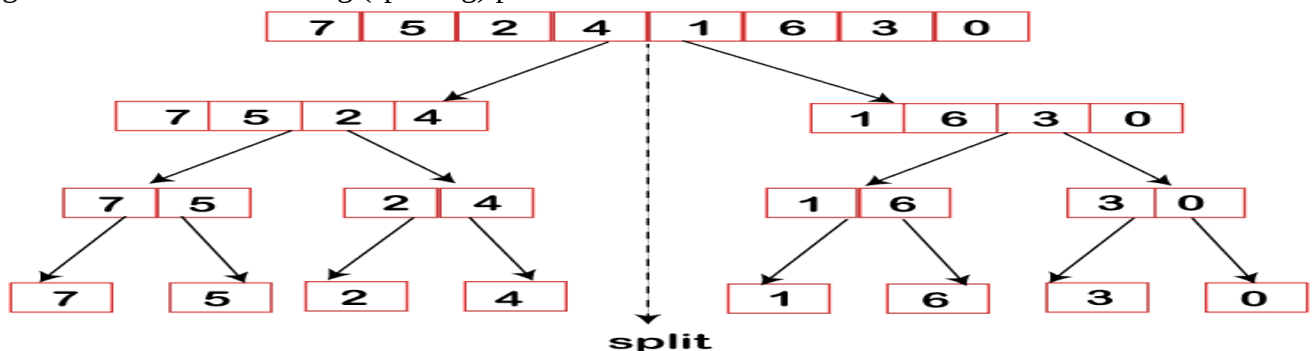


Figure 1: Merge Sort Divide Phase

The following figure illustrates the dividing (splitting) procedure.

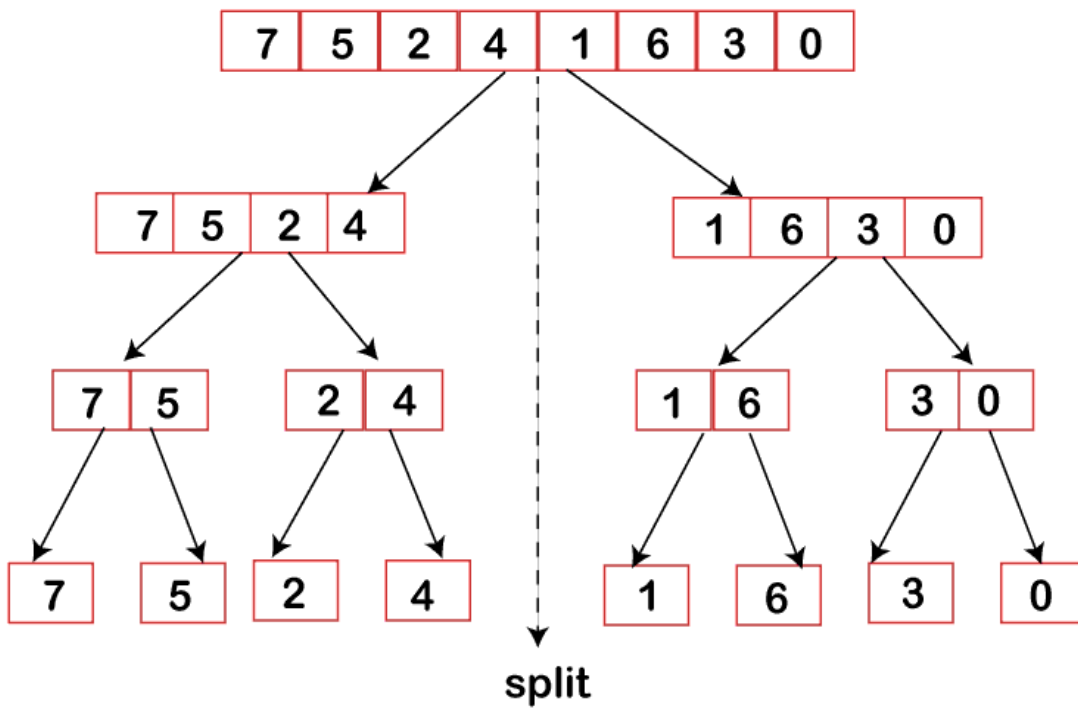


Figure 1: Merge Sort Divide Phase

- FUNCTIONS: MERGE (A, p, q, r)
- 
- 1.  $n_1 = q - p + 1$
- 2.  $n_2 = r - q$
- 3. create arrays  $L[1 \dots n_1 + 1]$  and  $R[1 \dots n_2 + 1]$
- 4. for  $i \leftarrow 1$  to  $n_1$
- 5. do  $L[i] \leftarrow A[p + i - 1]$
- 6. for  $j \leftarrow 1$  to  $n_2$
- 7. do  $R[j] \leftarrow A[q + j]$
- 8.  $L[n_1 + 1] \leftarrow \infty$
- 9.  $R[n_2 + 1] \leftarrow \infty$
- 10.  $I \leftarrow 1$
- 11.  $J \leftarrow 1$
- 12. For  $k \leftarrow p$  to  $r$
- 13. Do if  $L[I] \leq R[J]$
- 14. then  $A[k] \leftarrow L[I]$
- 15.  $I \leftarrow I + 1$
- 16. else  $A[k] \leftarrow R[J]$
- 17.  $J \leftarrow J + 1$

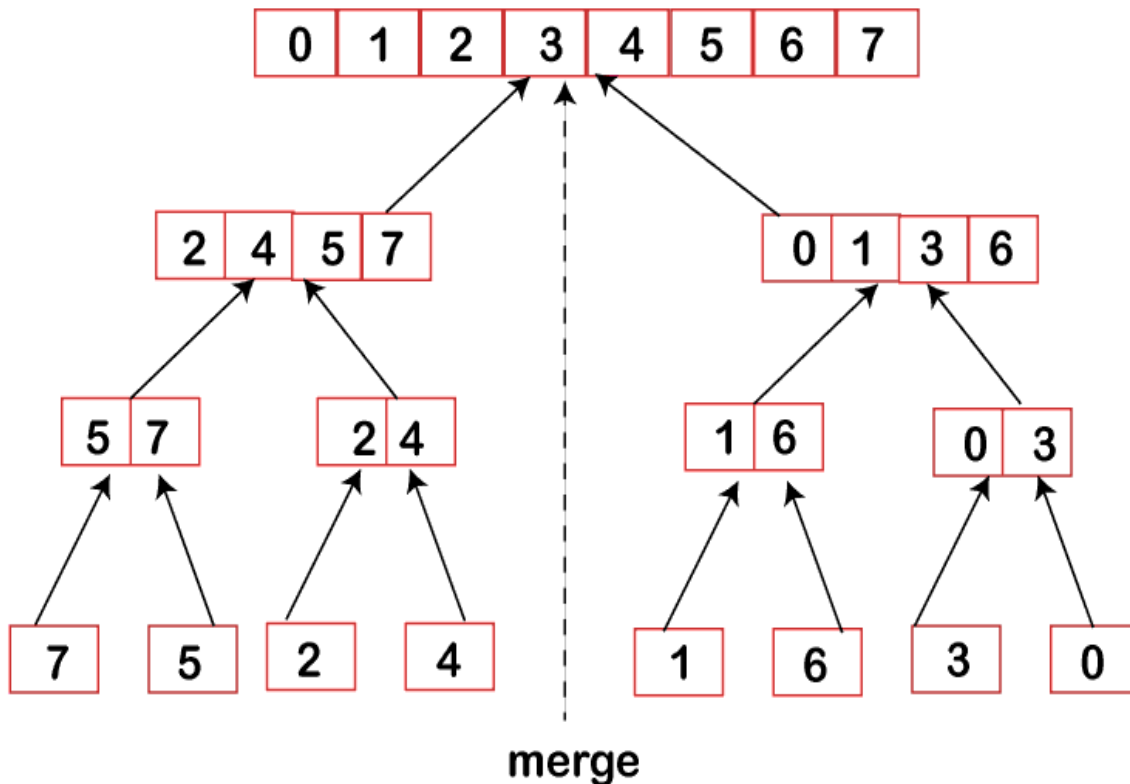


Figure 2: Merge Sort Combine Phase

## The merge step of Merge Sort

Mainly the recursive algorithm depends on a base case as well as its ability to merge back the results derived from the base cases. Merge sort is no different algorithm, just the fact here the **merge** step possesses more importance.

To any given problem, the merge step is one such solution that combines the two individually sorted lists(arrays) to build one large sorted list(array).

The merge sort algorithm upholds three pointers, i.e., one for both of the two arrays and the other one to preserve the final sorted array's current index.

- Did you reach the end of the array?
- No:
  - Firstly, start with comparing the current elements of both the arrays.
  - Next, copy the smaller element into the sorted array.
  - Lastly, move the pointer of the element containing a smaller element.
- Yes:
  - Simply copy the rest of the elements of the non-empty array

## Merge( ) Function Explained Step-By-Step

Consider the following example of an unsorted array, which we are going to sort with the help of the Merge Sort algorithm.

A= (36,25,40,2,7,80,15)

**Step1:** The merge sort algorithm iteratively divides an array into equal halves until we achieve an atomic value. In case if there are an odd number of elements in an array, then one of the halves will have more elements than the other half.

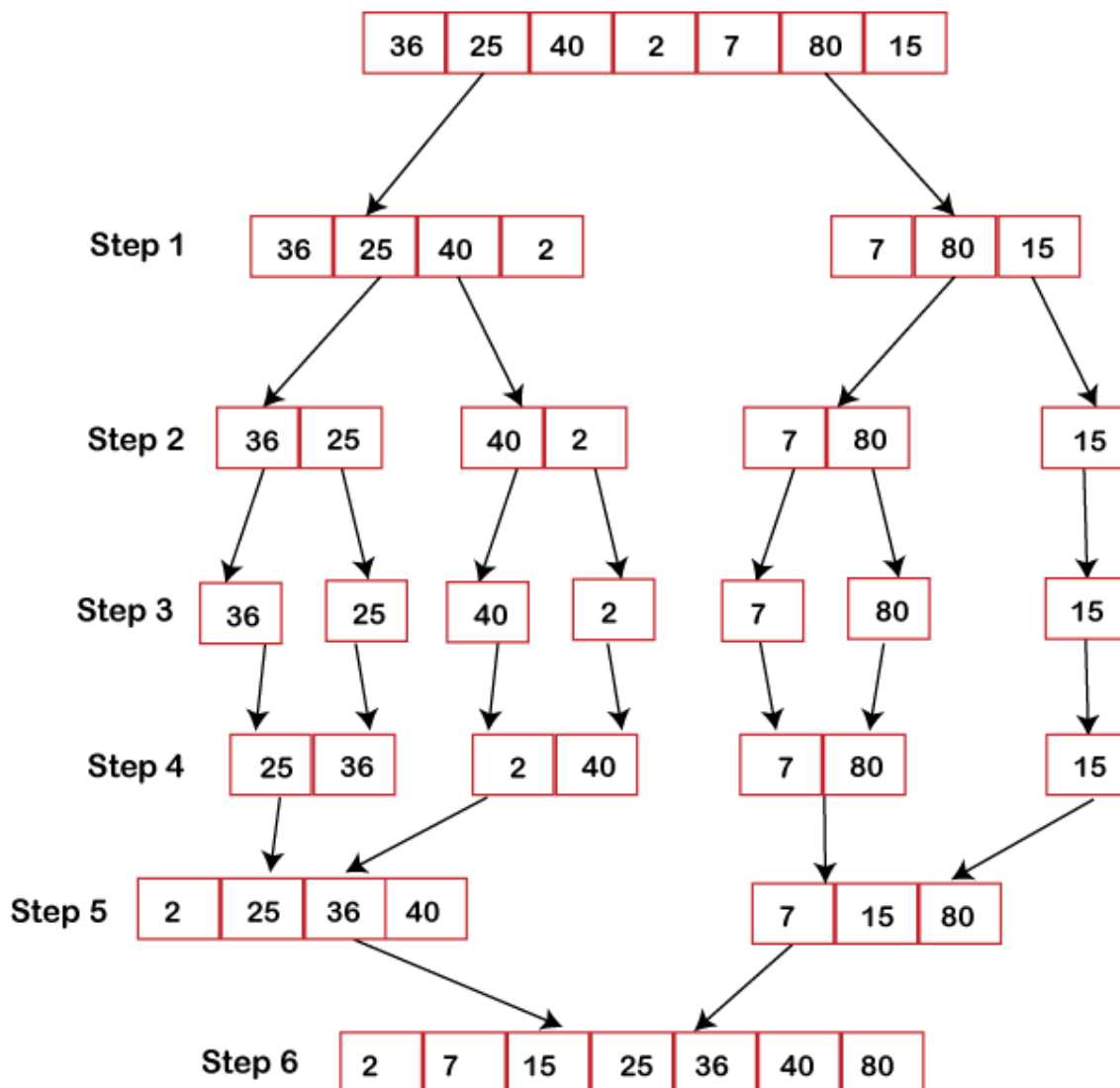
**Step2:** After dividing an array into two subarrays, we will notice that it did not hamper the order of elements as they were in the original array. After now, we will further divide these two arrays into other halves.

**Step3:** Again, we will divide these arrays until we achieve an atomic value, i.e., a value that cannot be further divided.

**Step4:** Next, we will merge them back in the same way as they were broken down.

**Step5:** For each list, we will first compare the element and then combine them to form a new sorted list.

**Step6:** In the next iteration, we will compare the lists of two data values and merge them back into a list of found data values, all placed in a sorted manner.



Hence the array is sorted.

## Analysis of Merge Sort:

Let  $T(n)$  be the total time taken by the Merge Sort algorithm.

- Sorting two halves will take at the most  $2T\left(\frac{n}{2}\right)$  time.
- When we merge the sorted lists, we come up with a total  $n-1$  comparison because the last element which is left will need to be copied down in the combined list, and there will be no comparison.

Thus, the relational formula will be

$$T(n) = 2T\left(\frac{n}{2}\right) + n - 1$$

But we ignore '-1' because the element will take some time to be copied in merge lists.

$$\text{So } T(n) = 2T\left(\frac{n}{2}\right) + n \dots \text{equation 1}$$

**Note: Stopping Condition  $T(1) = 0$  because at last, there will be only 1 element left that need to be copied, and there will be no comparison.**

Putting  $n = \frac{n}{2}$  in place of  $n$  in .....equation 1

$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{2^2}\right) + \frac{n}{2} \dots \text{equation 2}$$

Put 2 equation in 1 equation

$$T(n) = 2 \left[ 2T\left(\frac{n}{2^2}\right) + \frac{n}{2} \right] + n$$

$$= 2^2 T\left(\frac{n}{2^2}\right) + \frac{2n}{2} + n$$

$$T(n) = 2^2 T\left(\frac{n}{2^2}\right) + 2n \dots \text{equation 3}$$

Putting  $n = \frac{n}{2^2}$  in equation 1

$$T\left(\frac{n}{2^2}\right) = 2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2} \dots \text{equation 4}$$

Putting 4 equation in 3 equation

$$T(n) = 2^2 \left[ 2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2} \right] + 2n$$

$$T(n) = 2^3 T\left(\frac{n}{2^3}\right) + n + 2n$$

$$T(n) = 2^3 T\left(\frac{n}{2^3}\right) + 3n \dots \text{equation 5}$$

From eq 1, eq 3, eq 5 .....we get

$$T(n) = 2^i T\left(\frac{n}{2^i}\right) + in \dots \text{equation 6}$$

From Stopping Condition:

$$\frac{n}{2^i} = 1 \text{ And } T\left(\frac{n}{2^i}\right) = 0$$

$$n = 2^i$$

Apply log both sides:

$$\log n = \log 2^i$$

$$\log n = i \log 2$$

$$\frac{\log n}{\log 2} = i$$

$$\log_2 n = i$$

From 6 equation

$$T(n) = 2^i T\left(\frac{n}{2^i}\right) + in$$

$$= 2^i \times 0 + \log_2 n \cdot n$$

$$= T(n) = n \cdot \log n$$

**Best Case Complexity:** The merge sort algorithm has a best-case time complexity of  **$O(n \cdot \log n)$**  for the already sorted array.

**Average Case Complexity:** The average-case time complexity for the merge sort algorithm is  **$O(n \cdot \log n)$** , which happens when 2 or more elements are jumbled, i.e., neither in the ascending order nor in the descending order.

**Worst Case Complexity:** The worst-case time complexity is also  **$O(n \cdot \log n)$** , which occurs when we sort the descending order of an array into the ascending order.

**Space Complexity:** The space complexity of merge sort is  **$O(n)$** .

## Merge Sort Applications

The concept of merge sort is applicable in the following areas:

- Inversion count problem
- External sorting
- E-commerce applications

### 5.5.5 Quick Sort

## Quick sort

It is an algorithm of Divide & Conquer type.

**Divide:** Rearrange the elements and split arrays into two sub-arrays and an element in between such that each element in left sub array is less than or equal to the average element and each element in the right sub- array is larger than the middle element.

**Conquer:** Recursively, sort two sub arrays.

**Combine:** Combine the already sorted array.

## Algorithm:

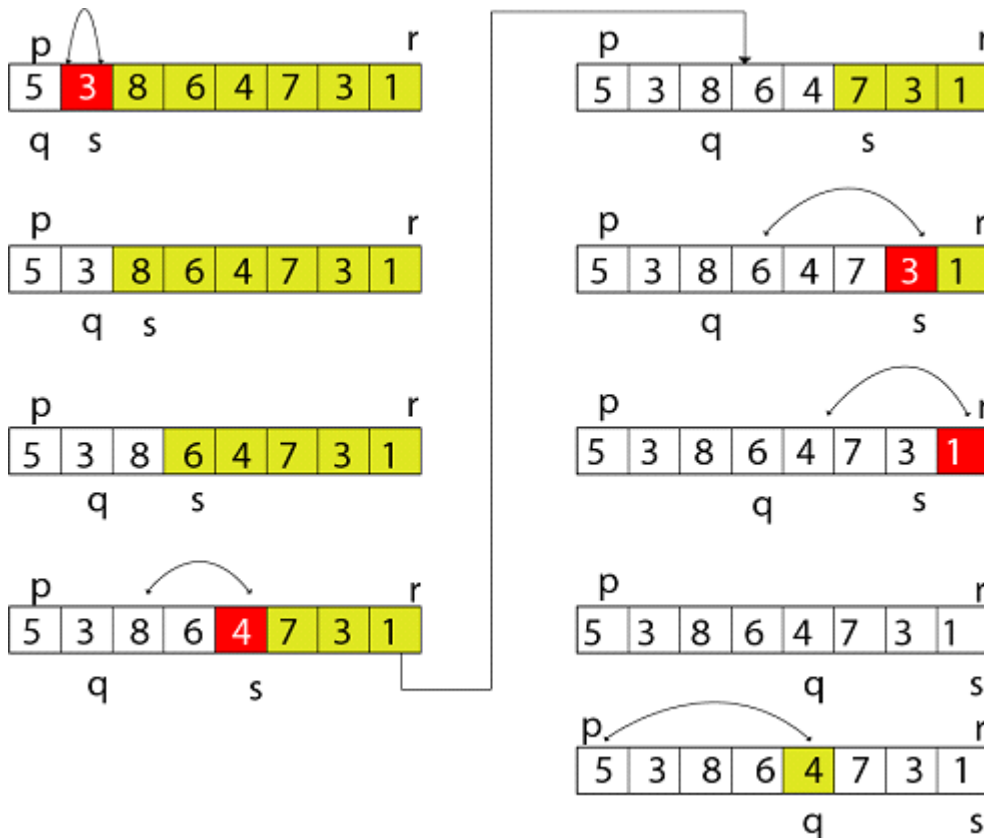
- QUICKSORT (array A, int m, int n)
- 1 if ( $n > m$ )
- 2 then
- 3  $i \leftarrow$  a random index from  $[m, n]$
- 4 swap  $A[i]$  with  $A[m]$
- 5  $o \leftarrow$  PARTITION ( $A, m, n$ )
- 6 QUICKSORT ( $A, m, o - 1$ )
- 7 QUICKSORT ( $A, o + 1, n$ )

## Partition Algorithm:

Partition algorithm rearranges the sub arrays in a place.

- PARTITION (array A, int m, int n)
- 1  $x \leftarrow A[m]$
- 2  $o \leftarrow m$
- 3 for  $p \leftarrow m + 1$  to  $n$
- 4 do if ( $A[p] < x$ )
- 5 then  $o \leftarrow o + 1$
- 6 swap  $A[o]$  with  $A[p]$
- 7 swap  $A[m]$  with  $A[o]$
- 8 return  $o$

**Figure:** shows the execution trace partition algorithm



### Example of Quick Sort:

- 44 33 11 55 77 90 40 60 99 22 88

Let **44** be the **Pivot** element and scanning done from right to left

Comparing **44** to the right-side elements, and if right-side elements are **smaller** than **44**, then swap it. As **22** is smaller than **44** so swap them.

22 33 11 55 77 90 40 60 99 **44** 88

Now comparing **44** to the left side element and the element must be **greater** than **44** then swap them. As **55** are greater than **44** so swap them.

22 33 11 **44** 77 90 40 60 99 **55** 88

Recursively, repeating steps 1 & steps 2 until we get two lists one left from pivot element **44** & one right from pivot element.

22 33 11 **40** 77 90 **44** 60 99 55 88

### Swap with 77:

22 33 11 40 **44** 90 **77** 60 99 55 88

Now, the element on the right side and left side are greater than and smaller than **44** respectively.

Now we get two sorted lists:



|                 |    |    |    |                 |    |    |    |    |    |    |
|-----------------|----|----|----|-----------------|----|----|----|----|----|----|
| 22              | 33 | 11 | 40 | <b>44</b>       | 90 | 77 | 66 | 99 | 55 | 88 |
| <b>Sublist1</b> |    |    |    | <b>Sublist2</b> |    |    |    |    |    |    |

And these sublists are sorted under the same process as above done.

These two sorted sublists side by side.

|                          |           |           |    |                 |           |           |               |                 |    |           |
|--------------------------|-----------|-----------|----|-----------------|-----------|-----------|---------------|-----------------|----|-----------|
| <b>22</b>                | 33        | 11        | 40 | <b>44</b>       | <b>90</b> | 77        | 60            | 99              | 55 | 88        |
| 11                       | 33        | <b>22</b> | 40 | <b>44</b>       | 88        | 77        | 60            | 99              | 55 | <b>90</b> |
| 11                       | <b>22</b> | 33        | 40 | <b>44</b>       | 88        | 77        | 60            | <b>90</b>       | 55 | <b>99</b> |
| <b>First sorted list</b> |           |           |    | 88              | 77        | 60        | <b>55</b>     | <b>90</b>       | 99 |           |
|                          |           |           |    | <b>Sublist3</b> |           |           |               | <b>Sublist4</b> |    |           |
|                          |           |           |    | 55              | 77        | 60        | <b>88</b>     | 90              | 99 |           |
|                          |           |           |    |                 |           |           | <b>Sorted</b> |                 |    |           |
|                          |           |           |    | 55              | <b>77</b> | <b>60</b> |               |                 |    |           |
|                          |           |           |    | 55              | 60        | 77        |               |                 |    |           |
|                          |           |           |    | <b>Sorted</b>   |           |           |               |                 |    |           |

### Merging Sublists:

|    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|
| 11 | 22 | 33 | 40 | 44 | 55 | 60 | 77 | 88 | 90 | 99 |
|----|----|----|----|----|----|----|----|----|----|----|

### SORTED LISTS

**Worst Case Analysis:** It is the case when items are already in sorted form and we try to sort them again. This will takes lots of time and space.

### Equation:

- $T(n) = T(1) + T(n-1) + n$

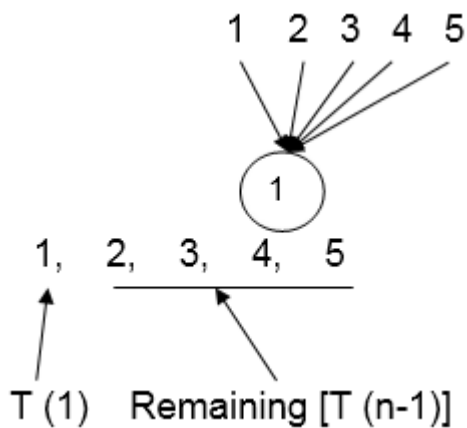
**T (1)** is time taken by pivot element.

**T (n-1)** is time taken by remaining element except for pivot element.

**N:** the number of comparisons required to identify the exact position of itself (every element)

If we compare first element pivot with other, then there will be 5 comparisons.

t means there will be n comparisons if there are n items.



### Relational Formula for Worst Case:

$$T(n) = T(1) + T(n-1) + n \dots \dots \dots (1)$$

$$T(n-1) = T(1) + T(n-1-1) + (n-1)$$

By putting (n-1) in place of n in equation 1

**Put T (n-1) in equation 1**

$$T(n) = T(1) + T(1) + T(n-2) + (n-1) + n \dots \dots \dots (ii)$$

$$T(n) = 2T(1) + T(n-2) + (n-1) + n$$

$$T(n-2) = T(1) + T(n-3) + (n-2)$$

By putting (n-2) in place of n in equation 1

**Put T (n-2) in equation (ii)**

$$T(n) = 2T(1) + T(1) + T(n-3) + (n-2) + (n-1) + n$$

$$T(n) = 3T(1) + T(n-3) + (n-2) + (n-1) + n$$

$$T(n-3) = T(1) + T(n-4) + n-3$$

By putting (n-3) in place of n in equation 1

$$T(n) = 3T(1) + T(1) + T(n-4) + (n-3) + (n-2) + (n-1) + n$$

$$= 4T(1) + T(n-4) + (n-3) + (n-2) + (n-1) + n \dots \dots \dots (iii)$$

$$T(n) = (n-1) T(1) + T(n-(n-1)) + (n-(n-2)) + (n-(n-3)) + (n-(n-4)) + n$$

$$T(n) = (n-1) T(1) + T(1) + 2 + 3 + 4 + \dots \dots \dots n$$

$$T(n) = (n-1) T(1) + T(1) + 2 + 3 + 4 + \dots \dots \dots + n + 1 - 1$$

[Adding 1 and subtracting 1 for making AP series]

$$T(n) = (n-1) T(1) + T(1) + 1 + 2 + 3 + 4 + \dots \dots \dots + n - 1$$

$$T(n) = (n-1) T(1) + T(1) + \frac{n(n+1)}{2} - 1$$

**Stopping Condition: T (1) =0**

Because at last there is only one element left and no comparison is required.

$$T(n) = (n-1)(0) + 0 + \frac{n(n+1)}{2} - 1$$

$$T(n) = \frac{n^2 + n - 2}{2}$$

$$T(n) = O(n^2)$$

Avoid all the terms except higher terms  $n^2$

**Worst Case Complexity of Quick Sort is  $T(n) = O(n^2)$**

#### 5.5.6 Counting Sort

## Counting Sort

It is a linear time sorting algorithm which works faster by not making a comparison. It assumes that the number to be sorted is in range 1 to k where k is small.

Basic idea is to determine the "rank" of each number in the final sorted array.

### Counting Sort uses three arrays:

- A [1, n] holds initial input.
- B [1, n] holds sorted output.
- C [1, k] is the array of integer. C [x] is the rank of x in A where  $x \in [1, k]$

Firstly C [x] to be a number of elements of A [j] that is equal to x

- Initialize C to zero
- For each j from 1 to n increment C [A[j]] by 1

We set B[C [x]] = A[j]

If there are duplicates, we decrement C [i] after copying.

- Counting Sort (array P, array Q, int k)
  1. For  $i \leftarrow 1$  to k
  2. do C [i]  $\leftarrow 0$  [  $\theta(k)$  times]
  3. for j  $\leftarrow 1$  to length [A]
  4. do C[A[j]]  $\leftarrow C[A[j]] + 1$  [  $\theta(n)$  times]
  5. // C [i] now contain the number of elements equal to i
  6. for i  $\leftarrow 2$  to k
  7. do C [i]  $\leftarrow C[i] + C[i-1]$  [  $\theta(k)$  times]
  8. //C[i] now contain the number of elements  $\leq i$
  9. for j  $\leftarrow$  length [A] down to 1 [  $\theta(n)$  times]
  10. do B[C[A[j]]]  $\leftarrow A[j]$
  11. C[A[j]]  $\leftarrow C[A[j]] - 1$

**Explanation:**

**Step1:** for loop initialize the array R to '0'. But there is a contradict in the first step initialize of loop variable 1 to k or 0 to k. As 0&1 are based on the minimum value comes in array A (input array). Basically, we start I with the value which is minimum in input array 'A'

For loops of steps **3 to 4** inspects each input element. If the value of an input element is 'i', we increment C [i]. Thus, after step 5, C [i] holds the number of input element equal to I for each integer i=0, 1, 2.....k

Step **6 to 8** for loop determines for each i=0, 1.....how many input elements are less than or equal to i

For loop of step **9 to 11** place each element A [j] into its correct sorted position in the output array B. for each A [j],the value C [A[j]] is the correct final position of A [j] in the output array B, since there are C [A[j]] element less than or equal to A [i].

Because element might not be distinct, so we decrement C[A[j]] each time we place a value A [j] into array B decrement C[A[j]] causes the next input element with a value equal to A [j], to go to the position immediately before A [j] in the output array.

## Analysis of Running Time:

- For a loop of step 1 to 2 take  $\theta(k)$  times
- For a loop of step 3 to 4 take  $\theta(n)$  times
- For a loop of step 6 to 7 take  $\theta(k)$  times
- For a loop of step 9 to 11 take  $\theta(n)$  times

Overall time is  $\theta(k+n)$  time.

### Note:

- Counting Sort has the important property that it is stable: numbers with the same value appears in the output array in the same order as they do in the input array.
- Counting Sort is used as a subroutine in Radix Sort.

**Example:** Illustration the operation of Counting Sort in the array.

- A= ( 7,1,3,1,2,4,5,7,2,4,3)

### Solution:

#### Fig: Initial A and C Arrays

- For j=1 to 11
- J=1, C [1, k] =

#### Fig: A [1] = 7 Processed

- J=2, C [1, k] =

#### Fig: A [2] = 1 Processed

- J=3, C [1, k]

**Fig: A [3] = 3 Processed**

- J=4, C [1, k]

**Fig: A [4] = 1 Processed**

- J=5, C [1, k]

**Fig: A [5] = 2 Processed**

**UPDATED C is:**

**Fig: C now contains a count of elements of A**

**Note: here the item of 'A' one by one get scanned and will become a position in 'C' and how many times the item get accessed will be mentioned in an item in 'C' Array and it gets updated or counter increased by 1 if any item gets accessed again.**

Now, the for loop i= 2 to 7 will be executed having statement:

- $C[i] = C[i] + C[i-1]$

By applying these conditions we will get C updated as i stated from 2 up to 7

$C[2] = C[2] + C[1]$      $C[3] = C[3] + C[2]$   
 $C[2] = 2 + 2$          $C[3] = 2 + 4$   
 **$C[2] = 4$**          **$C[3] = 6$**

$C[4] = C[4] + C[3]$      $C[5] = C[5] + C[4]$   
 $C[4] = 2 + 6$          $C[5] = 1 + 8$   
 **$C[4] = 8$**          **$C[5] = 9$**

$C[6] = C[6] + C[5]$      $C[7] = C[7] + C[6]$   
 $C[6] = 0 + 9$          $C[7] = 2 + 9$   
 **$C[6] = 9$**          **$C[7] = 11$**

**Thus the Updated C is:**

**Fig: C set to rank each number of A**

Now, we will find the new array B

Now two Conditions will apply:

- $B[C[A[j]]] \leftarrow A[j]$
- $C[A[j]] \leftarrow C[A[j]] - 1$

We decrease counter one by one by '1'

We start scanning of the element in A from the last position.

Element in A became a position in C

- For  $j \leftarrow 11$  to 1

#### Step 1

$B[C[A[11]]] = A[11]$      $C[A[11]] = C[A[11]] - 1$

$B[C[3]] = 3$              $C[3] = C[3] - 1$

**$B[6] = 3$              $C[3] = 5$**

**Fig: A [11] placed in Output array B**

#### Step 2

$B[C[A[10]]] = A[10]$      $C[A[10]] = C[A[10]] - 1$

$B[C[4]] = 4$              $C[4] = C[4] - 1$

**$B[8] = 4$              $C[4] = 7$**

**Fig: A [10] placed in Output array B**

#### Step 3

$B[C[A[9]]] = A[9]$      $C[A[9]] = C[A[9]] - 1$

$B[C[2]] = A[2]$              $C[2] = C[2] - 1$

**$B[4] = 2$              $C[2] = 3$**

**Fig: A [9] placed in Output array B**

#### Step 4

$B[C[A[8]]] = A[8]$      $C[A[8]] = C[A[8]] - 1$

$B[C[7]] = 7$              $C[A[8]] = C[7] - 1$

**$B[11] = 7$              $C[7] = 10$**

**Fig: A [8] placed in Output array B**

#### Step 5

$B[C[A[7]]] = A[7]$      $C[A[7]] = C[A[7]] - 1$

$B[C[5]] = 5$              $C[5] = C[5] - 1$

**$B[9] = 5$              $C[5] = 8$**

**Fig: A [7] placed in Output array B**

#### Step 6

$B[C[A[6]]] = A[6]$      $C[A[6]] = C[A[6]] - 1$

$B[C[4]] = 4$              $C[4] = C[4] - 1$

**$B[7] = 4$              $C[4] = 6$**

**Fig: A [6] placed in Output array B**

**Step 7**

$B[C[A[5]]] = A[5]$        $C[A[5]] = C[A[5]] - 1$   
 $B[C[2]] = 2$                $C[2] = C[2] - 1$   
 **$B[3] = 2$                $C[2] = 2$**

**Fig: A [5] placed in Output array B**

**Step 8**

$B[C[A[4]]] = A[4]$        $C[A[4]] = C[A[4]] - 1$   
 $B[C[1]] = 1$                $C[1] = C[1] - 1$   
 **$B[2] = 1$                $C[1] = 1$**

**Fig: A [4] placed in Output array B**

**Step 9**

$B[C[A[3]]] = A[3]$        $C[A[3]] = C[A[3]] - 1$   
 $B[C[3]] = 3$                $C[3] = C[3] - 1$   
 **$B[5] = 3$                $C[3] = 4$**

**Fig: A [3] placed in Output array B**

**Step 10**

$B[C[A[2]]] = A[2]$        $C[A[2]] = C[A[2]] - 1$   
 $B[C[1]] = 1$                $C[1] = C[1] - 1$   
 **$B[1] = 1$                $C[1] = 0$**

**Fig: A [2] placed in Output array B**

**Step 11**

$B[C[A[1]]] = A[1]$        $C[A[1]] = C[A[1]] - 1$   
 $B[C[7]] = 7$                $C[7] = C[7] - 1$   
 **$B[10] = 7$                $C[7] = 9$**

**Fig: B now contains the final sorted data.**

# Heap Sort Algorithm

In this article, we will discuss the Heapsort Algorithm. Heap sort processes the elements by creating the min-heap or max-heap using the elements of the given array. Min-heap or max-heap represents the ordering of array in which the root element represents the minimum or maximum element of the array.

Heap sort basically recursively performs two main operations -

- Build a heap H, using the elements of array.
- Repeatedly delete the root element of the heap formed in 1<sup>st</sup> phase.

Before knowing more about the heap sort, let's first see a brief description of **Heap**.

## What is a heap?

A heap is a complete binary tree, and the binary tree is a tree in which the node can have the utmost two children. A complete binary tree is a binary tree in which all the levels except the last level, i.e., leaf node, should be completely filled, and all the nodes should be left-justified.

## What is heap sort?

Heapsort is a popular and efficient sorting algorithm. The concept of heap sort is to eliminate the elements one by one from the heap part of the list, and then insert them into the sorted part of the list.

Heapsort is the in-place sorting algorithm.

Now, let's see the algorithm of heap sort.

## Algorithm

1. HeapSort(arr)
2. BuildMaxHeap(arr)
3. for **i** = **length**(arr) to 2
4.     swap arr[1] with arr[i]
5.     heap\_size[arr] = heap\_size[arr] ? 1
6.     MaxHeapify(arr,1)
7. End

### **BuildMaxHeap(arr)**

1. BuildMaxHeap(arr)



2.    heap\_size(arr) = length(arr)
3.    for i = length(arr)/2 to 1
4.    MaxHeapify(arr,i)
5.    End

#### **MaxHeapify(arr,i)**

1.    MaxHeapify(arr,i)
2.    L = left(i)
3.    R = right(i)
4.    if L ? heap\_size[arr] and arr[L] > arr[i]
5.    largest = L
6.    else
7.    largest = i
8.    if R ? heap\_size[arr] and arr[R] > arr[largest]
9.    largest = R
10.   if largest != i
11.   swap arr[i] with arr[largest]
12.   MaxHeapify(arr,largest)
13.   End

## Working of Heap sort Algorithm

Now, let's see the working of the Heapsort Algorithm.

ADVERTISEMENT

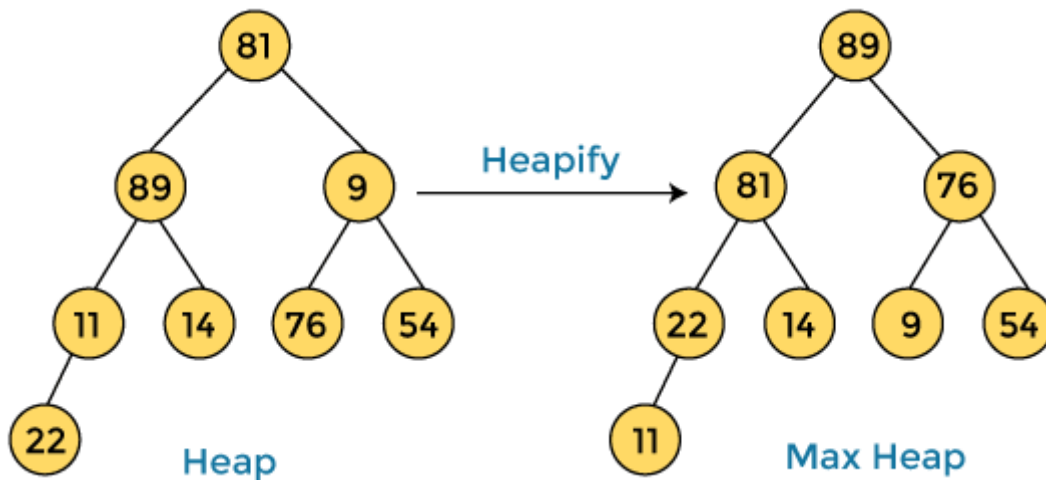
In heap sort, basically, there are two phases involved in the sorting of elements. By using the heap sort algorithm, they are as follows -

- The first step includes the creation of a heap by adjusting the elements of the array.
- After the creation of heap, now remove the root element of the heap repeatedly by shifting it to the end of the array, and then store the heap structure with the remaining elements.

Now let's see the working of heap sort in detail by using an example. To understand it more clearly, let's take an unsorted array and try to sort it using heap sort. It will make the explanation clearer and easier.

|    |    |   |    |    |    |    |    |
|----|----|---|----|----|----|----|----|
| 81 | 89 | 9 | 11 | 14 | 76 | 54 | 22 |
|----|----|---|----|----|----|----|----|

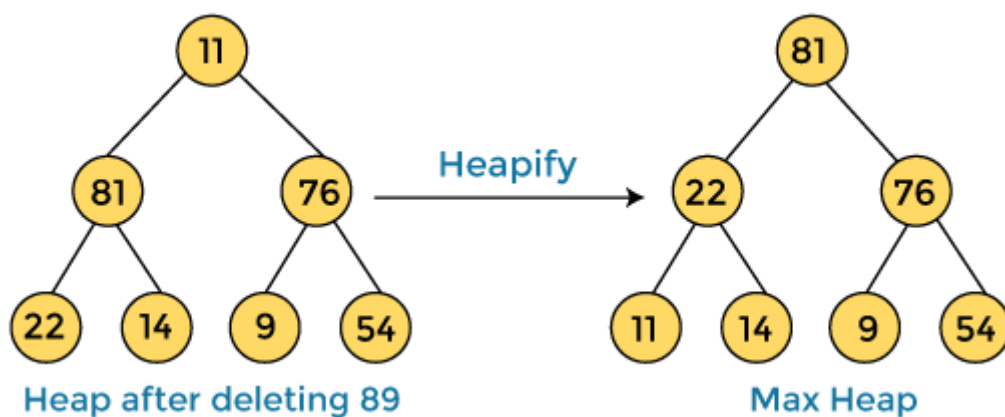
First, we have to construct a heap from the given array and convert it into max heap.



After converting the given heap into max heap, the array elements are -

|    |    |    |    |    |   |    |    |
|----|----|----|----|----|---|----|----|
| 89 | 81 | 76 | 22 | 14 | 9 | 54 | 11 |
|----|----|----|----|----|---|----|----|

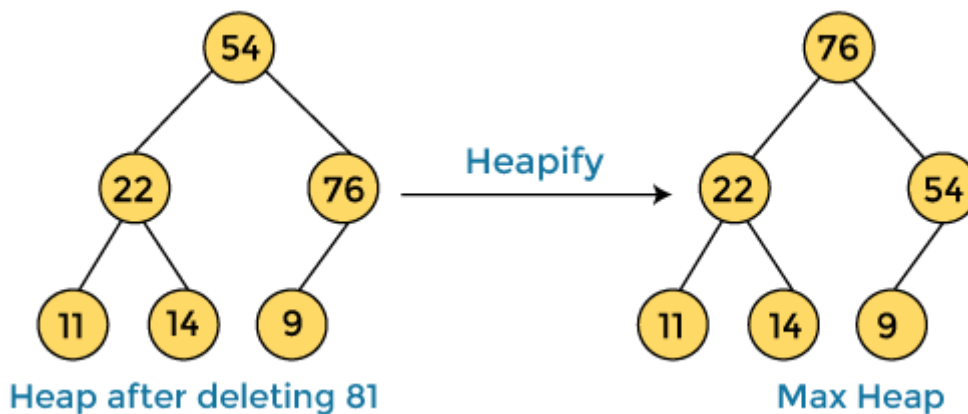
Next, we have to delete the root element (**89**) from the max heap. To delete this node, we have to swap it with the last node, i.e. (**11**). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element **89** with **11**, and converting the heap into max-heap, the elements of array are -

|    |    |    |    |    |   |    |    |
|----|----|----|----|----|---|----|----|
| 81 | 22 | 76 | 11 | 14 | 9 | 54 | 89 |
|----|----|----|----|----|---|----|----|

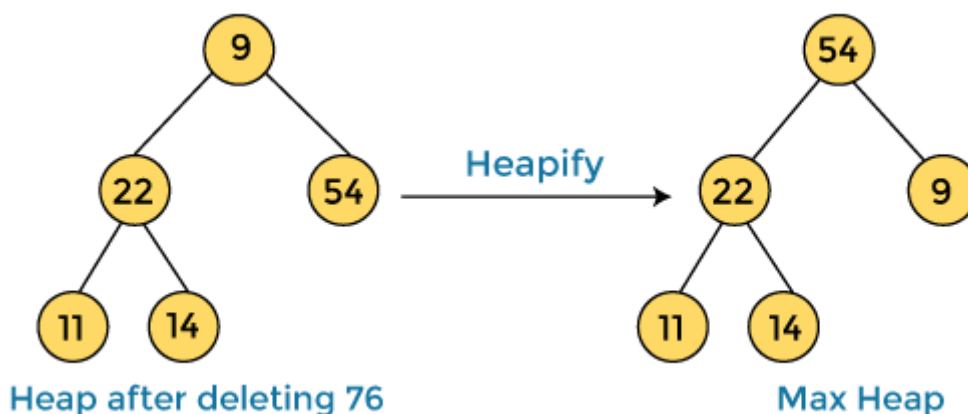
In the next step, again, we have to delete the root element (**81**) from the max heap. To delete this node, we have to swap it with the last node, i.e. (**54**). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element **81** with **54** and converting the heap into max-heap, the elements of array are -

|    |    |    |    |    |   |    |    |
|----|----|----|----|----|---|----|----|
| 76 | 22 | 54 | 11 | 14 | 9 | 81 | 89 |
|----|----|----|----|----|---|----|----|

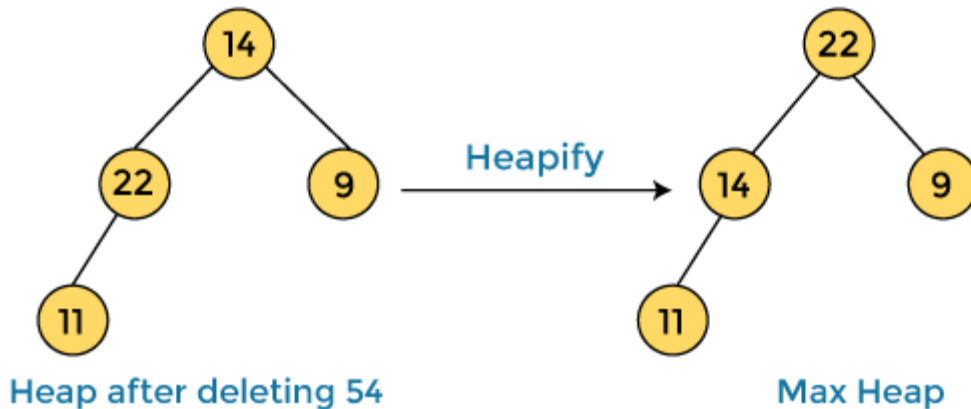
In the next step, we have to delete the root element (**76**) from the max heap again. To delete this node, we have to swap it with the last node, i.e. (**9**). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element **76** with **9** and converting the heap into max-heap, the elements of array are -

|    |    |   |    |    |    |    |    |
|----|----|---|----|----|----|----|----|
| 54 | 22 | 9 | 11 | 14 | 76 | 81 | 89 |
|----|----|---|----|----|----|----|----|

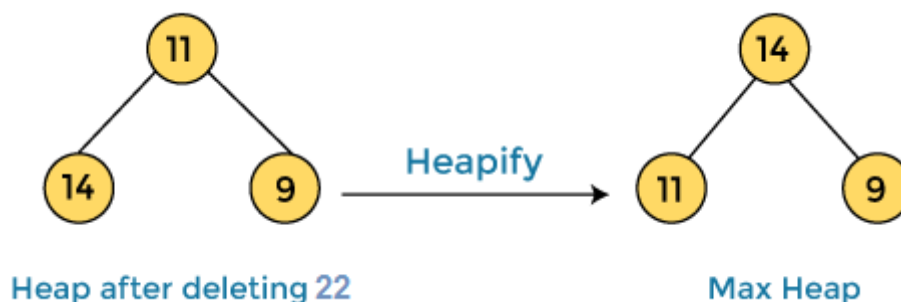
In the next step, again we have to delete the root element (**54**) from the max heap. To delete this node, we have to swap it with the last node, i.e. (**14**). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element **54** with **14** and converting the heap into max-heap, the elements of array are -

|    |    |   |    |    |    |    |    |
|----|----|---|----|----|----|----|----|
| 22 | 14 | 9 | 11 | 54 | 76 | 81 | 89 |
|----|----|---|----|----|----|----|----|

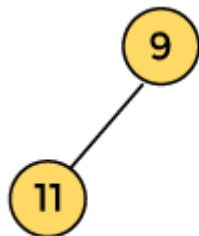
In the next step, again we have to delete the root element (**22**) from the max heap. To delete this node, we have to swap it with the last node, i.e. (**11**). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element **22** with **11** and converting the heap into max-heap, the elements of array are -

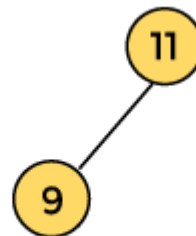
|    |    |   |    |    |    |    |    |
|----|----|---|----|----|----|----|----|
| 14 | 11 | 9 | 22 | 54 | 76 | 81 | 89 |
|----|----|---|----|----|----|----|----|

In the next step, again we have to delete the root element (**14**) from the max heap. To delete this node, we have to swap it with the last node, i.e. (**9**). After deleting the root element, we again have to heapify it to convert it into max heap.



Heap after deleting 14

Heapify



Max Heap

After swapping the array element **14** with **9** and converting the heap into max-heap, the elements of array are -

|    |   |    |    |    |    |    |    |
|----|---|----|----|----|----|----|----|
| 11 | 9 | 14 | 22 | 54 | 76 | 81 | 89 |
|----|---|----|----|----|----|----|----|

In the next step, again we have to delete the root element (**11**) from the max heap. To delete this node, we have to swap it with the last node, i.e. (**9**). After deleting the root element, we again have to heapify it to convert it into max heap.



Heap after deleting 11

Heapify



Max Heap

After swapping the array element **11** with **9**, the elements of array are -

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 11 | 14 | 22 | 54 | 76 | 81 | 89 |
|---|----|----|----|----|----|----|----|

ADVERTISEMENT

Now, heap has only one element left. After deleting it, heap will be empty.



Remove 9



Empty

After completion of sorting, the array elements are -

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 11 | 14 | 22 | 54 | 76 | 81 | 89 |
|---|----|----|----|----|----|----|----|

Now, the array is completely sorted.

## Heap sort complexity

Now, let's see the time complexity of Heap sort in the best case, average case, and worst case. We will also see the space complexity of Heapsort.

### 1. Time Complexity

| Case         | Time Complexity |
|--------------|-----------------|
| Best Case    | $O(n \log n)$   |
| Average Case | $O(n \log n)$   |
| Worst Case   | $O(n \log n)$   |

- **Best Case Complexity** - It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of heap sort is  **$O(n \log n)$** .
- **Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of heap sort is  **$O(n \log n)$** .
- **Worst Case Complexity** - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of heap sort is  **$O(n \log n)$** .

The time complexity of heap sort is  **$O(n \log n)$**  in all three cases (best case, average case, and worst case). The height of a complete binary tree having  $n$  elements is  **$\log n$** .

### 2. Space Complexity

|                         |      |
|-------------------------|------|
| <b>Space Complexity</b> | O(1) |
| <b>Stable</b>           | NO   |

- The space complexity of Heap sort is O(1).

## Implementation of Heapsort

Now, let's see the programs of Heap sort in different programming languages.

**Program:** Write a program to implement heap sort in C language.

```

1. #include <stdio.h>
2. /* function to heapify a subtree. Here 'i' is the
3. index of root node in array a[], and 'n' is the size of heap. */
4. void heapify(int a[], int n, int i)
5. {
6.     int largest = i; // Initialize largest as root
7.     int left = 2 * i + 1; // left child
8.     int right = 2 * i + 2; // right child
9.     // If left child is larger than root
10.    if (left < n && a[left] > a[largest])
11.        largest = left;
12.    // If right child is larger than root
13.    if (right < n && a[right] > a[largest])
14.        largest = right;
15.    // If root is not largest
16.    if (largest != i) {
17.        // swap a[i] with a[largest]
18.        int temp = a[i];
19.        a[i] = a[largest];
20.        a[largest] = temp;
21.
22.        heapify(a, n, largest);
23.    }
24.}
25. /*Function to implement the heap sort*/
26. void heapSort(int a[], int n)
27. {

```

```

28.  for (int i = n / 2 - 1; i >= 0; i--)
29.      heapify(a, n, i);
30.  // One by one extract an element from heap
31.  for (int i = n - 1; i >= 0; i--) {
32.      /* Move current root element to end*/
33.      // swap a[0] with a[i]
34.      int temp = a[0];
35.      a[0] = a[i];
36.      a[i] = temp;
37.
38.      heapify(a, i, 0);
39.  }
40.}
41./* function to print the array elements */
42.void printArr(int arr[], int n)
43.{
44.    for (int i = 0; i < n; ++i)
45.    {
46.        printf("%d", arr[i]);
47.        printf(" ");
48.    }
49.
50.}
51.int main()
52.{
53.    int a[] = {48, 10, 23, 43, 28, 26, 1};
54.    int n = sizeof(a) / sizeof(a[0]);
55.    printf("Before sorting array elements are - \n");
56.    printArr(a, n);
57.    heapSort(a, n);
58.    printf("\nAfter sorting array elements are - \n");
59.    printArr(a, n);
60.    return 0;
61.}

```

## Output



```
Before sorting array elements are -  
48 10 23 43 28 26 1  
After sorting array elements are -  
1 10 23 26 28 43 48
```

**Program:** Write a program to implement heap sort in C++.

5.5.8 in-place Sort

5.5.9 Stable Sort

## Unit 6:

### Analysis of Algorithms and Sorting

*“An algorithm can be defined as a finite set of steps, which has to be followed while carrying out a particular problem.”*

## Characteristics of Algorithms

- **Input:** It should externally supply zero or more quantities.
- **Output:** It results in at least one quantity.
- **Definiteness:** Each instruction should be clear and ambiguous.
- **Finiteness:** An algorithm should terminate after executing a finite number of steps.
- **Effectiveness:** Every instruction should be fundamental to be carried out, in principle, by a person using only pen and paper.
- **Feasible:** It must be feasible enough to produce each instruction.
- **Flexibility:** It must be flexible enough to carry out desired changes with no efforts.
- **Efficient:** The term efficiency is measured in terms of time and space required by an algorithm to implement. Thus, an algorithm must ensure that it takes little time and less memory space meeting the acceptable limit of development time.
- **Independent:** An algorithm must be language independent, which means that it should mainly focus on the input and the procedure required to derive the output instead of depending upon the language.

## Advantages of an Algorithm

- **Effective Communication:** Since it is written in a natural language like English, it becomes easy to understand the step-by-step delineation of a solution to any particular problem.
- **Easy Debugging:** A well-designed algorithm facilitates easy debugging to detect the logical errors that occurred inside the program.

- **Easy and Efficient Coding:** An algorithm is nothing but a blueprint of a program that helps develop a program.
- **Independent of Programming Language:** Since it is a language-independent, it can be easily coded by incorporating any high-level language.

## Disadvantages of an Algorithm

- Developing algorithms for complex problems would be time-consuming and difficult to understand.
- It is a challenging task to understand complex logic through algorithms.

## Need of Algorithm

1. To understand the basic idea of the problem.
2. To find an approach to solve the problem.
3. To improve the efficiency of existing techniques.
4. To understand the basic principles of designing the algorithms.
5. To compare the performance of the algorithm with respect to other techniques.
6. It is the best method of description without describing the implementation detail.
7. The Algorithm gives a clear description of requirements and goal of the problem to the designer.
8. A good design can produce a good solution.
9. To understand the flow of the problem.
10. To measure the behavior (or performance) of the methods in all cases (best cases, worst cases, average cases)
11. With the help of an algorithm, we can also identify the resources (memory, input-output) cycles required by the algorithm.
12. With the help of algorithm, we convert art into a science.
13. To understand the principle of designing.
14. We can measure and analyze the complexity (time and space) of the problems concerning input size without implementing and running it; it will reduce the cost of design.

## Analysis of algorithm

The analysis is a process of estimating the efficiency of an algorithm. There are two fundamental parameters based on which we can analysis the algorithm:

- **Space Complexity:** The space complexity can be understood as the amount of space required by an algorithm to run to completion.

- **Time Complexity:** Time complexity is a function of input size  $n$  that refers to the amount of time needed by an algorithm to run to completion.

In this course, we will be focusing more on time rather than space because time is instead a more limiting parameter in terms of the hardware. It is not easy to take a computer and change its speed. So, if we are running an algorithm on a particular platform, we are more or less stuck with the performance that platform can give us in terms of speed.

However, on the other hand, memory is relatively more flexible. We can increase the memory as when required by simply adding a memory card. So, we will focus on time than that of the space.

The running time is measured in terms of a particular piece of hardware, not a robust measure. When we run the same algorithm on a different computer or use different programming languages, we will encounter that the same algorithm takes a different time.

Generally, we make three types of analysis, which is as follows:

- **Worst-case time complexity:** For ' $n$ ' input size, the worst-case time complexity can be defined as the maximum amount of time needed by an algorithm to complete its execution. Thus, it is nothing but a function defined by the maximum number of steps performed on an instance having an input size of  $n$ .
- **Average case time complexity:** For ' $n$ ' input size, the average-case time complexity can be defined as the average amount of time needed by an algorithm to complete its execution. Thus, it is nothing but a function defined by the average number of steps performed on an instance having an input size of  $n$ .
- **Best case time complexity:** For ' $n$ ' input size, the best-case time complexity can be defined as the minimum amount of time needed by an algorithm to complete its execution. Thus, it is nothing but a function defined by the minimum number of steps performed on an instance having an input size of  $n$ .

## Complexity of Algorithm

The term algorithm complexity measures how many steps are required by the algorithm to solve the given problem. It evaluates the order of count of operations executed by an algorithm as a function of input data size.

To assess the complexity, the order (approximation) of the count of operation is always considered instead of counting the exact steps.

$O(f)$  notation represents the complexity of an algorithm, which is also termed as an Asymptotic notation or "**Big O**" notation. Here the  $f$  corresponds to the function whose size is the same as that of the input data. The complexity of the asymptotic computation  $O(f)$  determines in which order the resources such as CPU time, memory, etc. are consumed by the algorithm that is articulated as a function of the size of the input data.

The complexity can be found in any form such as constant, logarithmic, linear,  $n \cdot \log(n)$ , quadratic, cubic, exponential, etc. It is nothing but the order of constant, logarithmic, linear and so on, the number of steps encountered for the completion of a particular algorithm. To make it even more precise, we often call the complexity of an algorithm as "running time".

# Typical Complexities of an Algorithm

- **Constant Complexity:**

It imposes a complexity of  $O(1)$ . It undergoes an execution of a constant number of steps like 1, 5, 10, etc. for solving a given problem. The count of operations is independent of the input data size.

- **Logarithmic Complexity:**

It imposes a complexity of  $O(\log(N))$ . It undergoes the execution of the order of  $\log(N)$  steps. To perform operations on  $N$  elements, it often takes the logarithmic base as 2.

For  $N = 1,000,000$ , an algorithm that has a complexity of  $O(\log(N))$  would undergo 20 steps (with a constant precision). Here, the logarithmic base does not hold a necessary consequence for the operation count order, so it is usually omitted.

- **Linear Complexity:**

- It imposes a complexity of  $O(N)$ . It encompasses the same number of steps as that of the total number of elements to implement an operation on  $N$  elements.

For example, if there exist 500 elements, then it will take about 500 steps. Basically, in linear complexity, the number of elements linearly depends on the number of steps. For example, the number of steps for  $N$  elements can be  $N/2$  or  $3*N$ .

- It also imposes a run time of  $O(n*\log(n))$ . It undergoes the execution of the order  $N*\log(N)$  on  $N$  number of elements to solve the given problem.
- For a given 1000 elements, the linear complexity will execute 10,000 steps for solving a given problem.
- **Quadratic Complexity:** It imposes a complexity of  $O(n^2)$ . For  $N$  input data size, it undergoes the order of  $N^2$  count of operations on  $N$  number of elements for solving a given problem. If  $N = 100$ , it will endure 10,000 steps. In other words, whenever the order of operation tends to have a quadratic relation with the input data size, it results in quadratic complexity. For example, for  $N$  number of elements, the steps are found to be in the order of  $3*N^2/2$ .
- **Cubic Complexity:** It imposes a complexity of  $O(n^3)$ . For  $N$  input data size, it executes the order of  $N^3$  steps on  $N$  elements to solve a given problem. For example, if there exist 100 elements, it is going to execute 1,000,000 steps.
- **Exponential Complexity:** It imposes a complexity of  $O(2^n)$ ,  $O(N!)$ ,  $O(n^k)$ , .... For  $N$  elements, it will execute the order of count of operations that is exponentially dependable on the input data size.

## How to approximate the time taken by the Algorithm?

So, to find it out, we shall first understand the types of the algorithm we have. There are two types of algorithms:

- **Iterative Algorithm:** In the iterative approach, the function repeatedly runs until the condition is met or it fails. It involves the looping construct.
- **Recursive Algorithm:** In the recursive approach, the function calls itself until the condition is met. It integrates the branching structure.

However, it is worth noting that any program that is written in iteration could be written as recursion. Likewise, a recursive program can be converted to iteration, making both of these algorithms equivalent to each other.

But to analyze the iterative program, we have to count the number of times the loop is going to execute, whereas in the recursive program, we use recursive equations, i.e., we write a function of  $F(n)$  in terms of  $F(n/2)$ .

Suppose the program is neither iterative nor recursive. In that case, it can be concluded that there is no dependency of the running time on the input data size, i.e., whatever is the input size, the running time is going to be a constant value. Thus, for such programs, the complexity will be  **$O(1)$** .

## For Iterative Programs

Consider the following programs that are written in simple English and does not correspond to any syntax.

### Example1:

In the first example, we have an integer  $i$  and a for loop running from  $i$  equals 1 to  $n$ . Now the question arises, how many times does the name get printed?

- A()
- {
- int  $i$ ;
- for ( $i=1$  to  $n$ )
- printf("Edward");
- }

Since  $i$  equals 1 to  $n$ , so the above program will print Edward,  $n$  number of times. Thus, the complexity will be  **$O(n)$** .

### Example2:

- A()
- {
- int  $i, j$ ;
- for ( $i=1$  to  $n$ )
- for ( $j=1$  to  $n$ )
- printf("Edward");
- }

In this case, firstly, the outer loop will run  $n$  times, such that for each time, the inner loop will also run  $n$  times. Thus, the time complexity will be  **$O(n^2)$** .

### Example3:

- A()
- {
- $i = 1; S = 1;$
- while ( $S \leq n$ )
- {

- i++;
- SS = S + i;
- printf("Edward");
- }
- }

As we can see from the above example, we have two variables; i, S and then we have while  $S \leq n$ , which means S will start at 1, and the entire loop will stop whenever S value reaches a point where S becomes greater than n.

Here i is incrementing in steps of one, and S will increment by the value of i, i.e., the increment in i is linear. However, the increment in S depends on the i.

Initially;

i=1, S=1

After 1<sup>st</sup> iteration;

i=2, S=3

After 2<sup>nd</sup> iteration;

i=3, S=6

After 3<sup>rd</sup> iteration;

i=4, S=10 ... and so on.

Since we don't know the value of n, so let's suppose it to be k. Now, if we notice the value of S in the above case is increasing; for i=1, S=1; i=2, S=3; i=3, S=6; i=4, S=10; ...

Thus, it is nothing but a series of the sum of first n natural numbers, i.e., by the time i reaches k, the value of S will be  $k(k+1)/2$ .

To stop the loop,  $\frac{k(k+1)}{2}$  has to be greater than n, and when we solve this equation, we will get  $\frac{k^2+k}{2} > n$ . Hence, it can be concluded that we get a complexity of  $O(\sqrt{n})$  in this case.

## For Recursive Program

Consider the following recursive programs.

### Example1:

- A(n)
- {
- if (n>1)
- return (A(n-1))
- }

Solution;

Here we will see the simple Back Substitution method to solve the above problem.

$$T(n) = 1 + T(n-1) \quad \dots \text{Eqn. (1)}$$

**Step1:** Substitute  $n-1$  at the place of  $n$  in Eqn. (1)

$$T(n-1) = 1 + T(n-2) \quad \dots \text{Eqn. (2)}$$

**Step2:** Substitute  $n-2$  at the place of  $n$  in Eqn. (1)

$$T(n-2) = 1 + T(n-3) \quad \dots \text{Eqn. (3)}$$

**Step3:** Substitute Eqn. (2) in Eqn. (1)

$$T(n) = 1 + 1 + T(n-2) = 2 + T(n-2) \quad \dots \text{Eqn. (4)}$$

**Step4:** Substitute eqn. (3) in Eqn. (4)

$$T(n) = 2 + 1 + T(n-3) = 3 + T(n-3) = \dots = k + T(n-k) \quad \dots \text{Eqn. (5)}$$

Now, according to Eqn. (1), i.e.  $T(n) = 1 + T(n-1)$ , the algorithm will run until  $n > 1$ . Basically,  $n$  will start from a very large number, and it will decrease gradually. So, when  $T(n) = 1$ , the algorithm eventually stops, and such a terminating condition is called anchor condition, base condition or stopping condition.

Thus, for  $k = n-1$ , the  $T(n)$  will become.

**Step5:** Substitute  $k = n-1$  in eqn. (5)

$$T(n) = (n-1) + T(n-(n-1)) = (n-1) + T(1) = n-1+1$$

Hence,  $T(n) = n$  or  **$O(n)$** .

## Algorithm Design Techniques

The following is a list of several popular design approaches:

**1. Divide and Conquer Approach:** It is a top-down approach. The algorithms which follow the divide & conquer techniques involve three steps:

- Divide the original problem into a set of subproblems.
- Solve every subproblem individually, recursively.
- Combine the solution of the subproblems (top level) into a solution of the whole original problem.

**2. Greedy Technique:** Greedy method is used to solve the optimization problem. An optimization problem is one in which we are given a set of input values, which are required either to be maximized or minimized (known as objective), i.e. some constraints or conditions.

- Greedy Algorithm always makes the choice (greedy criteria) looks best at the moment, to optimize a given objective.
- The greedy algorithm doesn't always guarantee the optimal solution however it generally produces a solution that is very close in value to the optimal.

**3. Dynamic Programming:** Dynamic Programming is a bottom-up approach we solve all possible small problems and then combine them to obtain solutions for bigger problems.

This is particularly helpful when the number of copying subproblems is exponentially large. Dynamic Programming is frequently related to **Optimization Problems**.

**4. Branch and Bound:** In Branch & Bound algorithm a given subproblem, which cannot be bounded, has to be divided into at least two new restricted subproblems. Branch and Bound algorithm are methods for global optimization in non-convex problems. Branch and Bound algorithms can be slow, however in the worst case they require effort that grows exponentially with problem size, but in some cases we are lucky, and the method coverage with much less effort.

**5. Backtracking Algorithm:** Backtracking Algorithm tries each possibility until they find the right one. It is a depth-first search of the set of possible solution. During the search, if an alternative doesn't work, then backtrack to the choice point, the place which presented different alternatives, and tries the next alternative.

**6. Randomized Algorithm:** A randomized algorithm is defined as an algorithm that is allowed to access a source of independent, unbiased random bits, and it is then allowed to use these random bits to influence its computation. A randomized algorithm uses a random number at least once during the computation make a decision.

**Example 1:** In Quick Sort, using a random number to choose a pivot.

**Example 2:** Trying to factor a large number by choosing a random number as possible divisors.

## Loop invariants

This is a justification technique. We use loop invariant that helps us to understand why an algorithm is correct. To prove statement  $S$  about a loop is correct, define  $S$  concerning series of smaller statement  $S_0, S_1, \dots, S_k$  where,

- The initial claim  $S_0$  is true before the loop begins.
- If  $S_{i-1}$  is true before iteration  $i$  begin, then one can show that  $S_i$  will be true after iteration  $i$  is over.
- The final statement  $S_k$  implies the statement  $S$  that we wish to justify as being true. analysis

## Asymptotic Analysis of algorithms (Growth of function)

Resources for an algorithm are usually expressed as a function regarding input. Often this function is messy and complicated to work. To study Function growth efficiently, we reduce the function down to the important part.

$$\text{Let } f(n) = an^2 + bn + c$$

In this function, the  $n^2$  term dominates the function that is when  $n$  gets sufficiently large.

Dominate terms are what we are interested in reducing a function, in this; we ignore all constants and coefficient and look at the highest order term concerning  $n$ .

### Asymptotic notation:

The word **Asymptotic** means approaching a value or curve arbitrarily closely (i.e., as some sort of limit is taken).

### Asymptotic analysis



It is a technique of representing limiting behavior. The methodology has the applications across science. It can be used to analyze the performance of an algorithm for some large data set. In computer science in the analysis of algorithms, considering the performance of algorithms when applied to very large input datasets

The simplest example is a function  $f(n) = n^2 + 3n$ , the term  $3n$  becomes insignificant compared to  $n^2$  when  $n$  is very large. The function " $f(n)$ " is said to be **asymptotically equivalent** to  $n^2$  as  $n \rightarrow \infty$ ", and here is written symbolically as  $f(n) \sim n^2$ .

**Asymptotic notations** are used to write fastest and slowest possible running time for an algorithm. These are also referred to as 'best case' and 'worst case' scenarios respectively.

"In asymptotic notations, we derive the complexity concerning the size of the input. (Example in terms of  $n$ )"

"These notations are important because without expanding the cost of running the algorithm, we can estimate the complexity of the algorithms."

## Why is Asymptotic Notation Important?

1. They give simple characteristics of an algorithm's efficiency.
2. They allow the comparisons of the performances of various algorithms.

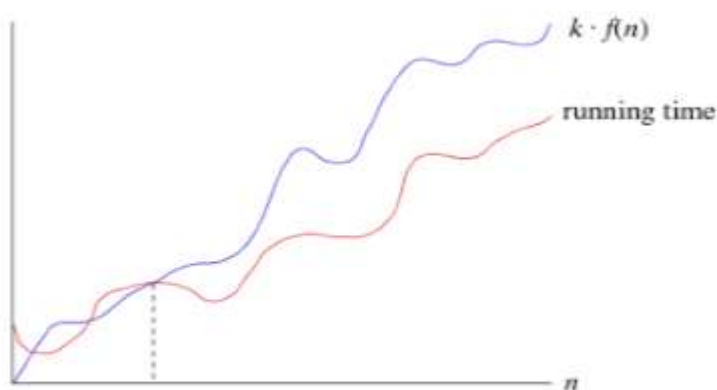
## Asymptotic Notations:

Asymptotic Notation is a way of comparing function that ignores constant factors and small input sizes. Three notations are used to calculate the running time complexity of an algorithm:

**1. Big-oh notation:** Big-oh is the formal method of expressing the upper bound of an algorithm's running time. It is the measure of the longest amount of time. The function  $f(n) = O(g(n))$  [read as "f of n is big-oh of g of n"] if and only if exist positive constant  $c$  and such that

- $f(n) \leq k \cdot g(n)$  for  $n > n_0$  in all case

Hence, function  $g(n)$  is an upper bound for function  $f(n)$ , as  $g(n)$  grows faster than  $f(n)$



**ASYMPTOTIC UPPER BOUND**

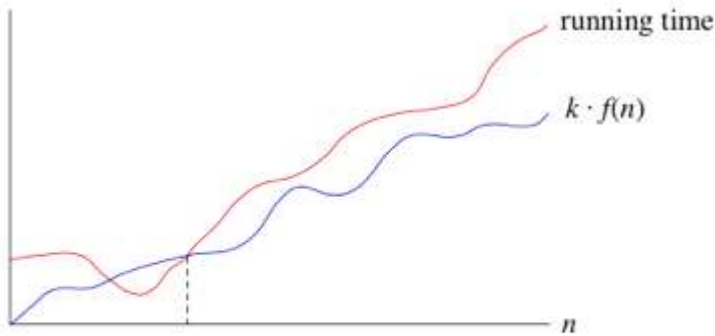
**For Example:**

- 1.  $3n+2=O(n)$  as  $3n+2 \leq 4n$  for all  $n \geq 2$
- 2.  $3n+3=O(n)$  as  $3n+3 \leq 4n$  for all  $n \geq 3$

Hence, the complexity of **f(n)** can be represented as  $O(g(n))$

**2. Omega () Notation:** The function  $f(n) = \Omega(g(n))$  [read as "f of n is omega of g of n"] if and only if there exists positive constant  $c$  and  $n_0$  such that

$$f(n) \geq k \cdot g(n) \text{ for all } n, n \geq n_0$$



### ASYMPTOTIC LOWER BOUND

**For Example:**

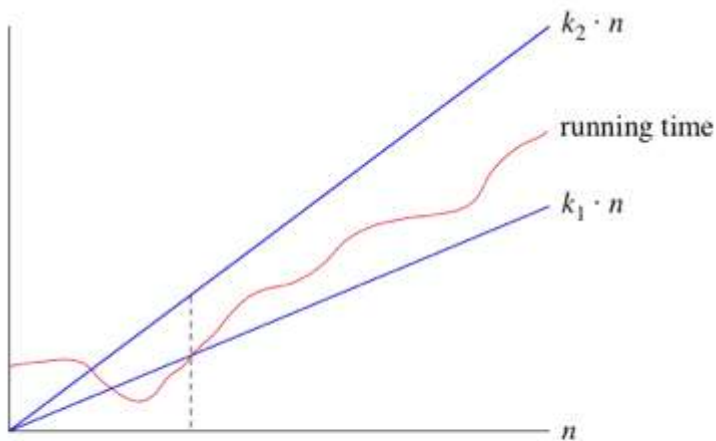
$$\begin{aligned} f(n) &= 8n^2 + 2n - 3 \geq 8n^2 - 3 \\ &= 7n^2 + (n^2 - 3) \geq 7n^2 (g(n)) \end{aligned}$$

Thus,  $k_1=7$

Hence, the complexity of **f(n)** can be represented as  $\Omega(g(n))$

**3. Theta (θ):** The function  $f(n) = \theta(g(n))$  [read as "f is the theta of g of n"] if and only if there exists positive constant  $k_1$ ,  $k_2$  and  $k_0$  such that

$$k_1 \cdot g(n) \leq f(n) \leq k_2 \cdot g(n) \text{ for all } n, n \geq n_0$$



### ASYMPTOTIC TIGHT BOUND

#### For Example:

$3n+2 = \theta(n)$  as  $3n+2 \geq 3n$  and  $3n+2 \leq 4n$ , for  $n$   
 $k_1=3, k_2=4$ , and  $n_0=2$

Hence, the complexity of  $f(n)$  can be represented as  $\theta(g(n))$ .

The Theta Notation is more precise than both the big-oh and Omega notation. The function  $f(n) = \theta(g(n))$  if  $g(n)$  is both an upper and lower bound.

## 6.1 Complexity Measures of an Algorithm

## 6.2 Asymptotic Notations

## 6.3 Analysis of Sorting Algorithms

### 6.3.1 Selection Sort

### 6.3.2 bubble Sort

### 6.3.3 Insertion Sort

### 6.3.4 Heap Sort

### 6.3.5 Quick Sort

## 6.4 Analysis of Searching Algorithms

### 6.4.1 linear Search

### 6.4.2 binary Search